CASAL2 User Manual

NIWA Technical Report 139
ISSN 1174-2631
2020

CASAL2 User Manual (modified 2020-08-07) for use with
`casal2-v2020-08-07 (rev. 2d6fe374)`

Citation: CASAL2 Development Team (2020). CASAL2 User Manual, v2020-08-07 (rev. 2d6fe374). National Institute of Water & Atmospheric Research Ltd. *NIWA Technical Report 139*. 260 p.

Contents

1 Introduction **1**
- 1.1 About CASAL2 . . . 1
- 1.2 Citing CASAL2 . . . 1
- 1.3 CASAL2 Contributors . . . 1
- 1.4 Software license . . . 2
- 1.5 Where to get CASAL2 . . . 2
- 1.6 System requirements . . . 2
- 1.7 Necessary files . . . 2
- 1.8 Getting help . . . 2
- 1.9 Technical details . . . 3

2 Model overview **5**

3 Running CASAL2 **7**
- 3.1 Using CASAL2 . . . 7
- 3.2 Redirecting standard output . . . 7
- 3.3 Command line arguments . . . 8
- 3.4 Constructing the CASAL2 input configuration files . . . 9
  - 3.4.1 Commands . . . 10
  - 3.4.2 Subcommands . . . 11
  - 3.4.3 The command block format . . . 12
  - 3.4.4 Commenting out lines . . . 12
  - 3.4.5 How to reference parameters . . . 13
- 3.5 Reading a command block . . . 13
- 3.6 Single-stepping CASAL2 . . . 15
- 3.7 CASAL2 exit status values . . . 15

4 Partition & Categories **17**
- 4.1 Specifying the partition using categories . . . 17
- 4.2 Shorthand syntax for categories . . . 18
- 4.3 Referencing vector and map parameters . . . 22
- 4.4 Examples? not needed? . . . 24

5 Specifying the population model **24**
- 5.1 Introduction . . . 24
- 5.2 Model scope and structure . . . 25
  - 5.2.1 The implicit annual cycle . . . 27
  - 5.2.2 The initialisation phases . . . 28
    - 5.2.2.1 Iterative Initialisation . . . 29
    - 5.2.2.2 Derived Initialisation . . . 30
    - 5.2.2.3 Cinitial Initialisation UNTESTED . . . 31

5.2.2.4 State_category_by_age UNTESTED . . . 32
5.3 Population processes . . . 32
5.3.1 Recruitment . . . 33
5.3.1.1 Constant recruitment . . . 33
5.3.1.2 Beverton-Holt recruitment . . . 34
5.3.2 Ageing . . . 38
5.3.3 Mortality . . . 38
5.3.3.1 Timing evaluation interval; timing the point when observations are fitted or derived quantities are evaluated . . . 39
5.3.3.2 Constant mortality rate UNTESTED . . . 40
5.3.3.3 Event and biomass-event mortality UNTESTED? . . . 41
5.3.3.4 Instantaneous mortality . . . 42
5.3.3.5 Instantaneous mortality with retained catch and discards . . . 44
5.3.3.6 Holling mortality rate UNTESTED . . . 47
5.3.3.7 Initialisation-event mortality UNTESTED . . . 48
5.3.3.8 Prey-suitability mortality UNTESTED . . . 49
5.3.4 Transition By Category . . . 51
5.3.4.1 Annual transition by category . . . 51
5.3.5 Tag Release events UNTESTED . . . 51
5.3.6 Tag Loss UNTESTED . . . 53
5.4 Derived quantities . . . 53
5.5 Age-length relationship . . . 54
5.5.0.1 None . . . 54
5.5.0.2 von Bertalanffy . . . 54
5.5.0.3 Schnute . . . 55
5.5.0.4 Data: `data` . . . 55
5.6 Length-weight relationship . . . 55
5.6.0.1 None . . . 56
5.6.0.2 Basic . . . 56
5.7 Age-weight relationship UNTESTED . . . 56
5.8 Weightless model UNTESTED . . . 57
5.9 Maturity, in models without maturing in the partition . . . 57
5.10 Selectivities . . . 57
5.10.1 Constant . . . 58
5.10.2 Knife-edge . . . 59
5.10.3 All-values . . . 59
5.10.4 All-values-bounded . . . 59
5.10.5 Increasing . . . 59
5.10.6 Logistic . . . 60
5.10.7 Inverse logistic . . . 60
5.10.8 Logistic producing . . . 60
5.10.9 Double-normal . . . 61

5.10.10 Double-exponential . . . 61
5.11 Time-varying Parameters UNTESTED . . . 62
5.11.1 Constant (year blocks) . . . 63
5.11.2 Random Walk . . . 64
5.11.3 Annual shift . . . 64
5.11.4 Exogenous . . . 64
5.12 Equation Parser UNTESTED . . . 65
5.13 Specifying projections UNTESTED . . . 66
5.13.1 Projection methods . . . 67
5.13.1.1 Constant . . . 67
5.13.1.2 Empirical resampling . . . 68
5.13.1.3 Lognormal . . . 68
5.13.1.4 Lognormal-Empirical . . . 68
5.13.1.5 User Defined . . . 69
5.13.1.6 Catches . . . 70

6 The estimation section **71**
6.1 Role of the estimation section . . . 71
6.2 The objective function . . . 71
6.3 Specifying the parameters to be estimated . . . 72
6.4 Point estimation . . . 72
6.4.1 The numerical differences minimiser . . . 73
6.4.2 The differential evolution minimiser . . . 73
6.4.3 The BetaDiff minimiser . . . 74
6.4.4 The ADOL-C minimiser . . . 74
6.4.5 The CppAD minimiser . . . 75
6.4.6 The Dlib minimiser . . . 75
6.5 Posterior profiles . . . 75
6.6 Bayesian estimation . . . 76
6.7 Priors . . . 79
6.8 Penalties . . . 80
6.9 Additional Priors . . . 81
6.10 Parameter Transformations . . . 83

7 The observation section **87**
7.1 Observations . . . 87
7.1.1 Mortality block associated observations . . . 87
7.1.2 General process observations . . . 96
7.1.3 Specific process observations . . . 96
7.2 Likelihoods . . . 101
7.2.1 Likelihoods for proportions-at-age observations . . . 101
7.2.2 Likelihoods for abundance and biomass observations . . . 102

7.2.3 Likelihoods for tag recapture by age and length observations 104
7.2.4 Likelihoods for proportions-by-category observations 104
7.3 Process error 105
7.4 Calculating nuisance q parameters 106
7.5 Ageing error 108
7.6 Simulating observations 108
7.7 Pseudo-observations 109
7.8 Residuals 110

8 The report section **111**
8.1 Report command block format 111
8.2 Report block output format 111
8.3 Print the partition at the end of an initialisation 112
8.4 Print the partition 112
8.5 Print the age length and length weight values 112
8.6 Print a process summary 112
8.7 Print derived quantities 112
8.8 Print the estimated parameters 112
8.9 Print the estimated parameters in a vector format 113
8.10 Print the objective function 113
8.11 Print the covariance matrix 113
8.12 Print observations, fits, and residuals 113
8.13 Print simulated observations 113
8.14 Print the ageing error misclassification matrix 114
8.15 Print selectivities 114
8.16 Print the random number seed 114
8.17 Print the results of an MCMC 114
8.18 Print the MCMC samples as they are calculated 114
8.19 Print the MCMC objective function values as they are calculated 114
8.20 Print time varying parameters 114
8.21 Tabular reporting format 114

9 Population command and subcommand syntax **117**
9.1 Model structure 117
9.2 Initialisation 118
9.2.1 Cinitial 119
9.2.2 Derived 119
9.2.3 Iterative 119
9.2.4 State Category By Age 120
9.3 Categories 120
9.4 Time-steps 121
9.5 Processes 121

9.5.1 Ageing . . . 121
9.5.2 Growth Basic . . . 121
9.5.3 Maturation . . . 122
9.5.4 Mortality Constant Rate . . . 122
9.5.5 Mortality Event . . . 123
9.5.6 Mortality Event Biomass . . . 123
9.5.7 Mortality Holling Rate . . . 124
9.5.8 Mortality Initialisation Event . . . 125
9.5.9 Mortality Initialisation Event Biomass . . . 126
9.5.10 Mortality Instantaneous . . . 126
9.5.11 Mortality Instantaneous Retained . . . 127
9.5.12 Mortality Prey Suitability . . . 127
9.5.13 Recruitment Beverton Holt . . . 128
9.5.14 Recruitment Beverton Holt With Deviations . . . 130
9.5.15 Recruitment Constant . . . 131
9.5.16 Survival Constant Rate . . . 132
9.5.17 Tag By Age . . . 132
9.5.18 Tag By Length . . . 134
9.5.19 Tag Loss . . . 134
9.5.20 Transition Category . . . 135
9.5.21 Transition Category By Age . . . 136
9.6 Time varying parameters . . . 136
9.6.1 Annual Shift . . . 137
9.6.2 Constant . . . 137
9.6.3 Exogenous . . . 137
9.6.4 Linear . . . 138
9.6.5 Random Draw . . . 138
9.6.6 Random Walk . . . 138
9.7 Derived quantities . . . 139
9.7.1 Abundance . . . 140
9.7.2 Biomass . . . 140
9.8 Age-length relationship . . . 140
9.8.1 Data . . . 141
9.8.2 None . . . 141
9.8.3 Schnute . . . 141
9.8.4 Von Bertalanffy . . . 142
9.9 Length-weight . . . 142
9.9.1 Basic . . . 143
9.9.2 None . . . 143
9.10 Selectivities . . . 143
9.10.1 All Values . . . 144
9.10.2 All Values Bounded . . . 144

9.10.3 Constant . . . 144
9.10.4 Double Exponential . . . 145
9.10.5 Double Normal . . . 145
9.10.6 Increasing . . . 146
9.10.7 Inverse Logistic . . . 146
9.10.8 Knife Edge . . . 147
9.10.9 Logistic . . . 147
9.10.10 Logistic Producing . . . 147
9.11 Projections . . . 148
9.11.1 Constant . . . 148
9.11.2 Empirical Sampling . . . 149
9.11.3 Log Normal . . . 149
9.11.4 Log Normal Empirical . . . 149
9.11.5 User Defined . . . 149
9.12 Time-varying Parameters . . . 150
9.12.1 Annual Shift . . . 150
9.12.2 Constant . . . 151
9.12.3 Exogenous . . . 151
9.12.4 Linear . . . 151
9.12.5 Random Draw . . . 151
9.12.6 Random Walk . . . 152

10 Estimation command and subcommand syntax 152
10.1 Estimation methods . . . 152
10.1.1 Beta . . . 154
10.1.2 Lognormal . . . 154
10.1.3 Normal . . . 154
10.1.4 Normal By Stdev . . . 155
10.1.5 Normal Log . . . 155
10.1.6 Uniform . . . 155
10.1.7 Uniform Log . . . 155
10.2 Point estimation . . . 155
10.2.1 ADOLC . . . 156
10.2.2 Betadiff . . . 156
10.2.3 CppAD . . . 157
10.2.4 DE Solver . . . 158
10.2.5 DLib . . . 158
10.2.6 Gamma Diff . . . 159
10.3 Markov chain Monte Carlo (MCMC) . . . 160
10.3.1 Independence Metropolis . . . 160
10.4 Profiles . . . 162
10.5 Defining catchability constants . . . 162

10.5.1 Free . . . 163
10.5.2 Nuisance . . . 163
10.6 Defining penalties . . . 163
10.6.1 Process . . . 163
10.7 Defining priors on parameter ratios, differences, and means . . . 164
10.7.1 Beta . . . 164
10.7.2 Element Difference . . . 164
10.7.3 Log Normal . . . 165
10.7.4 Uniform Log . . . 165
10.7.5 Vector Average . . . 165
10.7.6 Vector Smoothing . . . 165
10.8 Defining estimation of transformations . . . 166
10.8.1 Average Difference . . . 166
10.8.2 Inverse . . . 166
10.8.3 Log . . . 167
10.8.4 Log Sum . . . 167
10.8.5 Orthogonal . . . 167
10.8.6 Simplex . . . 167
10.8.7 Square Root . . . 168
10.8.8 Sum To One . . . 168

11 Observation command and subcommand syntax 168
11.1 Observation types . . . 168
11.1.1 Abundance . . . 169
11.1.2 Biomass . . . 170
11.1.3 Process Removals By Age . . . 171
11.1.4 Process Removals By Age Retained . . . 172
11.1.5 Process Removals By Age Retained Total . . . 173
11.1.6 Process Removals By Length . . . 174
11.1.7 Process Removals By Length Retained . . . 174
11.1.8 Process Removals By Length Retained Total . . . 175
11.1.9 Proportions At Age . . . 176
11.1.10 Proportions At Length . . . 177
11.1.11 Proportions By Category . . . 178
11.1.12 Proportions Mature By Age . . . 178
11.1.13 Proportions Migrating . . . 179
11.1.14 Tag Recapture By Age . . . 180
11.1.15 Tag Recapture By Length . . . 181
11.2 Likelihoods . . . 182
11.2.1 Binomial . . . 182
11.2.2 Binomial Approx . . . 182
11.2.3 Dirichlet . . . 182

11.2.4 Log Normal . . . 182
11.2.5 Log Normal With q . . . 182
11.2.6 Multinomial . . . 182
11.2.7 Normal . . . 182
11.2.8 Pseudo . . . 182
11.3 Defining ageing error . . . 182
11.3.1 Data . . . 183
11.3.2 None . . . 183
11.3.3 Normal . . . 183
11.3.4 Off By One . . . 183
11.4 Simulating observations . . . 184
11.4.1 Constant . . . 184

12 Report command and subcommand syntax **184**
12.1 Report commands and subcommands . . . 184
12.1.1 Addressable . . . 185
12.1.2 Age Length . . . 185
12.1.3 Ageing Error Matrix . . . 186
12.1.4 Catchability . . . 186
12.1.5 Category Info . . . 186
12.1.6 Category List . . . 186
12.1.7 Correlation Matrix . . . 186
12.1.8 Covariance Matrix . . . 186
12.1.9 Derived Quantity . . . 186
12.1.10 Equation Test . . . 186
12.1.11 Estimate Summary . . . 187
12.1.12 Estimate Value . . . 187
12.1.13 Estimation Result . . . 187
12.1.14 Hessian Matrix . . . 187
12.1.15 Initialisation Partition . . . 187
12.1.16 Initialisation Partition Mean Weight . . . 187
12.1.17 MCMC Covariance . . . 187
12.1.18 MCMC Objective . . . 187
12.1.19 MCMC Sample . . . 187
12.1.20 MPD . . . 187
12.1.21 Objective Function . . . 187
12.1.22 Observation . . . 187
12.1.23 Output Parameters . . . 187
12.1.24 Partition . . . 187
12.1.25 Partition Biomass . . . 188
12.1.26 Partition Mean Length . . . 188
12.1.27 Partition Mean Weight . . . 188

12.1.28 Partition Year Cross Age Matrix 188
12.1.29 Process 188
12.1.30 Project 188
12.1.31 Random Number Seed 189
12.1.32 Selectivity 189
12.1.33 Simulated Observation 189
12.1.34 Standard Header 189
12.1.35 Time Varying 189

13 Including commands from other files **189**

14 Tips for setting up Casal2 model based on an existing CASAL model **191**

15 Syntax conventions, examples and niceties **193**
15.1 Input File Specification 193
15.1.1 Keywords And Reserved Characters 193
15.2 More examples of shorthand syntax and use of reserved and key characters 197
15.3 Processes 201
15.4 An example of a simple model 201

16 Post-processing output using R **205**

17 Troubleshooting **209**
17.1 Logging 209
17.2 Reporting errors 209
17.2.1 Guidelines for reporting an error with CASAL2 209

18 CASAL2 software license **211**

19 Acknowledgements **217**

20 References **219**

21 Index **221**

22 Quick reference **225**

1. Introduction

1.1. About CASAL2

CASAL2 is NIWA's open-source integrated assessment tool for modelling the population dynamics of marine species, including fishery stock assessments. CASAL2 expands functionality and increases maintainability relative to its predecessor, CASAL. CASAL2 can be used for quantitative assessments of marine populations, including fish, invertebrates, marine mammals and seabirds.

The CASAL2 software implements a generalised age-structured population model that allows for a great deal of choice in specifying the population dynamics, parameter estimation, and model outputs. CASAL2 is designed for flexibility. It can implement an age-structured model for a single population or multiple populations using user-defined categories such as area, sex, and maturity. These structural elements are generic and not predefined, and are easily constructed. CASAL2 models can be used for a single population with a single anthropogenic event (in a fish population model this would be a single fishery), or for multiple species and populations, areas, and/or anthropogenic or exploitation methods, and including predator-prey interactions.

In CASAL2 the processes in a time period and within an annual cycle are defined by the user. Observation data used for model fitting can be from many different sources, like, removals-at-size or -age from an anthropogenic or exploitation event (e.g., fishery or other human impact), research survey and other biomass indices, and mark-recapture data. Model parameters can be estimated using penalised maximum likelihood or Bayesian methods.

As well as the point estimates of the parameters, CASAL2 can calculate the likelihood or posterior distribution profiles, and can generate Bayesian posterior distributions using Markov chain Monte Carlo methods. CASAL2 can project population status using deterministic or stochastic population dynamics. CASAL2 can also simulate observations from a set of given model structures.

1.2. Citing CASAL2

The reference for this document is CASAL2 Development Team (2020). CASAL2 User Manual, v2020-08-07 (rev. 2d6fe374). National Institute of Water & Atmospheric Research Ltd. *NIWA Technical Report 139*. 260 p.

The peer-reviewed journal article reference for CASAL2 is (Doonan et al., 2016).

1.3. CASAL2 Contributors

The CASAL2 project was started by Alistair Dunn. The software architect is Scott Rasmussen.

Authors of this document include Ian Doonan, Craig Marsh, Kath Large, Teresa A'mar, and Alistair Dunn.

Contributors to the development include Scott Rasmussen, Alistair Dunn, Ian Doonan, Craig Marsh, Teresa A'mar, Kath Large, Sophie Mormede, and Samik Datta.

Other contributors include Matt Dunn, Jingjing Zhang, and Marco Kienzle.

1.4. Software license

This program and the accompanying materials are made available under the terms of the GNU General Public License version 2 which accompanies this software (see Section 18).

Copyright ©2016-2020, National Institute of Water & Atmospheric Research Ltd.. All rights reserved.

1.5. Where to get CASAL2

In the first instance, see `https://www.niwa.co.nz/` for information about CASAL2. The CASAL2 source code is hosted on GitHub, and can be found at `http://github.com/NIWAFisheriesModelling/CASAL2`.

There are installation packages available for Linux and Microsoft Windows. The package includes the CASAL2 executable, the **R** library, the User Manual and other documentation, example models, and other information. The installation packages can be downloaded at `https://github.com/NIWAFisheriesModelling/CASAL2/releases`.

1.6. System requirements

CASAL2 is available for most IBM compatible machines running 64-bit Linux and Microsoft Windows operating systems.

Several of CASAL2's tasks are computer intensive and a fast processor is recommended. Depending on the model implemented, some of the CASAL2 tasks can take a considerable amount of processing time (minutes to hours), and in extreme cases may take several days to complete an MCMC estimate.

Output files can be large, and the output from developing a model, sensitivity analyses, and running multiple MCMC chains can take up significant amounts of disk space. Depending on the number and type of user output requests, the output could range from a few hundred kilobytes to several hundred megabytes. When estimating model fits, several hundred megabytes of RAM may be required, depending on the spatial size of the model, number of categories, and complexity of processes and observations. For larger models, several gigabytes of RAM and disk space may occasionally be required.

1.7. Necessary files

For both 64-bit Linux and Microsoft Windows, only the binary executable `casal2` or `casal2.exe` is required to run CASAL2. No other software is required. CASAL2 is not available for 32-bit operating systems.

CASAL2 provides few functions for post-processing model output, and a package that allows tabulation and graphing of model outputs is recommended. Software such as **R** (R Core Team, 2014) is recommended for post-processing CASAL2 output. The `CASAL2` **R** package is provided for parsing the CASAL2 output into **R** (see Section 16), as well as providing some diagnostic and plotting functionality.

1.8. Getting help

TODO review with the Project Manager and Project Lead

CASAL2 is distributed as unsupported software. Please notify the CASAL2 Development Team of any issues with or errors in CASAL2. Please contact the `Casal2 Development Team`. See Section 17.2 for the template for reporting issues.

1.9. Technical details

CASAL2 was compiled on Linux using `gcc` (`http://gcc.gnu.org`), the C/C++ compiler developed by the GNU Project (`http://gcc.gnu.org`). The 64-bit Linux version was compiled using `gcc` version 4.8.5 20150623 on CentOS 7.7 Linux (`http://www.ubuntu.com/`).

The Microsoft Windows (`http://www.microsoft.com`) version was compiled using MingW (`http://www.mingw.org`) `gcc` 8.1.0 (`http://gcc.gnu.org`). The Microsoft Windows(`http://www.microsoft.com`) installer was built using the Inno Setup 5 (`http://www.jrsoftware.org/isdl.php`).

CASAL2 includes number of different minimisers; different minimisers may perform better for some models than others. The first three minimisers are non-automatic differentiation minimisers: the first is closely based on the main algorithm of Dennis Jr and Schnabel (1996), and uses finite difference gradients; the second is an implementation of the differential evolution solver (Storn and Price, 1995), based on code by Lester E. Godwin of PushCorp, Inc.; and the third is Dlib (King, 2009).

The three automatic differentiation minimisers are: ADOL-C, an auto differentiation minimiser (Walther et al., 1996); CppAD, an auto differentiation minimiser similar to ADOL-C (Wächter and Biegler, 2006); and the third is a modified version of an older version of ADOL-C (v1.8.4) that was used as the automatic differentiation minimiser in the first version of CASAL (Bull et al., 2012).

The random number generator used in CASAL2 uses an implementation of the Mersenne twister random number generator (Matsumoto and Nishimura, 1998). This functionality, the command line functionality, matrix operations, and a number of other functions use the Boost C++ library (Version 1.58.0).

The output from CASAL2 may differ slightly on the different operating systems and operating system versions due to different precision arithmetic or other platform-dependent implementation details. The source code for CASAL2 is available in the GitHub repository at `https://github.com/NIWAFisheriesModelling/CASAL2`.

Unit tests of the underlying CASAL2 code are carried out at build time, using the Google Test and Mock unit testing and mocking framework. The unit test framework aims to cover a significant proportion of the key functionality within the CASAL2 code base. The unit test code for CASAL2 is available as a part of the underlying source code.

2. Model overview

CASAL2 is a generalized age-structured population dynamics modelling framework of the integrated assessment type (Maunder and Punt, 2013). It allows several sources of information to be combined into a single analysis using a statistical framework so that error sources are fully propagated into the uncertainty in the results. The model follows cohorts as numbers-at-age through time recording changes from processes acting on them such as nature mortality and fishing.

CASAL2 is run from the console window on Microsoft Windows or from a terminal window on Linux. CASAL2 has two sources of information: the *input configuration file* which defines the model structure, provides observations, defines active parameters, and specifies outputs (reports); and run time actions (e.g., estimation of active parameters or projections) which are given by command line options and arguments (see Section 3 for details).

The model is defined in terms of the *state* at various points in the annual cycle within each year. The the annual cycle defines the time steps within a year and the order of processes. The state consists of two parts, the *partition*, and any defined *derived quantities*. The partition will typically change in each *time-step* of every year, depending on the *processes* defined for those time-steps in the model.

The *partition* is a representation of the population at each time step, and can be considered a matrix of the numbers of individuals within each category (i.e., row) and at each age. Categories define the important population structure, such as whether a fish is male or female, in a specific area, and its maturity state, but other divisions are also possible like stock, or even species. How many categories there are and what they represent are entirely defined by the user as CASAL2 does not have any hard-wired fisheries concepts. The latter is one reason for re-writing CASAL (CASAL2's precursor), but the increased flexibility does come at a cost of more complex input files complexity.

A *derived quantity* is a calculation of a selected part of the partition at some point in time, an example is spawning stock biomass (SSB). Unlike the partition, which is updated as each new time step, a derived quantity records a single value for each year of the model run. Hence, derived quantities build up a vector of values over the time period represented by the model. The derived quantity is then available to be reported or as an input into another process at some point in a subsequent year. The most common use is using SSB from a previous year in the stock-recruitment relationship to determine numbers into the first age of the partition. Another example might be having age -at-maturity inversely related to the largest size of recruitment in the previous four years.

Observations, their sampling distributions, relationship with the partition, and time sampled (time step and year) are specified in the input configuration file. Examples are abundance indices from a research trawl survey or age compositions from the commercial catch. The relevant state can now be linked to the observation which then can generate expected values for the observations. In broad terms, the model parameters can be optimised to provide the best fit between expected values and the observations. Best fit is judged by the lowest negative log-likelihood, for which a special case would be the sums-of-squares. The manner in which observations are evaluated and how the expected values are calculated by the model is described in Section 7.

The model scope and population structure are specified in the input configuration file and this is similar to that in CASAL (Bull et al., 2012), but with a diffident syntax. The model records the numbers of individuals by category and age (e.g., for a sex category, the numbers of males and females at age). In general, cohorts are added via a recruitment event, are aged annually, and are removed from the population via various forms of mortality (e.g., fishing or natural mortality). The population is assumed to be closed (i.e., no immigration or emigration from the modelled area). REPEAT?

By convention, the input configuration file is a file named config.csl2 which reads in four other

files so that assessment specifications are more digestible (one comprehensive file can be used instead and users do not have to use the name config.csl2). These subordinate files cover (again by convention): the model structure, population dynamics, etc. (Section 5); the methods of estimation, priors, penalties, and the model parameters to be estimated Section 6); the observation data, their associated likelihoods, and their timing (Section 7); and the outputs from the model (default is no outputs) (Section 8). The input configuration file completely describes a model implemented in CASAL2. See Sections 9, 10, 11, and 12 for details and specification of CASAL2's command and subcommand syntax within the input configuration file.

3. Running CASAL2

CASAL2 is run from a console window (i.e., the command line) on Microsoft Windows or from a terminal window on Linux. CASAL2 uses information from input data files -- the *input configuration file* being the input file that is supplied to CASAL2.

The input configuration file is required and defines the model structure, processes, observations, parameters (both the fixed parameters and the parameters to be estimated), and the requested reports (outputs).

By convention, the name of the input configuration file ends with the suffix `.csl2`. However, any suffix is acceptable. The default name for the input configuration file is *config.csl2* and if used it does not have to be specified to CASAL2. Note that the input configuration file can include other separate files so the specification can be split into digestible parts.

Command line arguments are used to specify the actions or *tasks* of CASAL2, e.g., to run a model with a set of parameter values, to estimate parameter values (either point estimates or MCMC), to project quantities, or to simulate observations.For example, `-r` is the *run* mode, `-e` is the *estimation* mode, and `-m` is the *MCMC* mode. The *command line arguments* are described in Section 3.3.

3.1. Using CASAL2

To use CASAL2, open a console window (i.e. the command prompt) window on Microsoft Windows or a terminal window on Linux. Navigate to the directory where the model input configuration files are located. Then enter `casal2` with arguments for a specific mode to start the CASAL2 mode running; see Section 3.3 for the list of possible arguments. CASAL2 will print output to the screen.

The CASAL2 executable and shared libraries (files with extension `.dll` or `.so`) must either be in the same directory as the input configuration files or in one of the directories in your operating system's `PATH` environment variable. The CASAL2 installer will update `PATH`; see your operating system documentation for help displaying or modifying `PATH`.

3.2. Redirecting standard output

CASAL2 uses the `standard output` stream to display runtime information. The standard error stream is used by CASAL2 to output the program exit status and runtime errors. We suggest redirecting both the standard output and standard error into files.

With the bash shell (on Linux systems), you can do this using the command structure

```
(casal2 [arguments] > run.out) >& run.err &
```

It may be useful to redirect the standard input, especially if you're using CASAL2 inside a batch job, i.e.

```
(casal2 [arguments] > run.out < /dev/null) >& run.err &
```

On Microsoft Windows systems, you can redirect to standard output using

```
casal2 [arguments] > run.out
```

And, on some Microsoft Windows systems (e.g., Windows 10), you can redirect to both standard output and standard error, using the syntax

```
casal2 [arguments] > run.out 2> run.err
```

CASAL2 outputs header information to the output (Figure 3.1). The header consists of the program name and version, the arguments passed to CASAL2 from the command line, the date and time that the program was called (derived from the system time), the user name, and the machine name (including the operating system and the process identification number). This information can be used to track outputs as well as identifying the version of CASAL2 used to run the model.

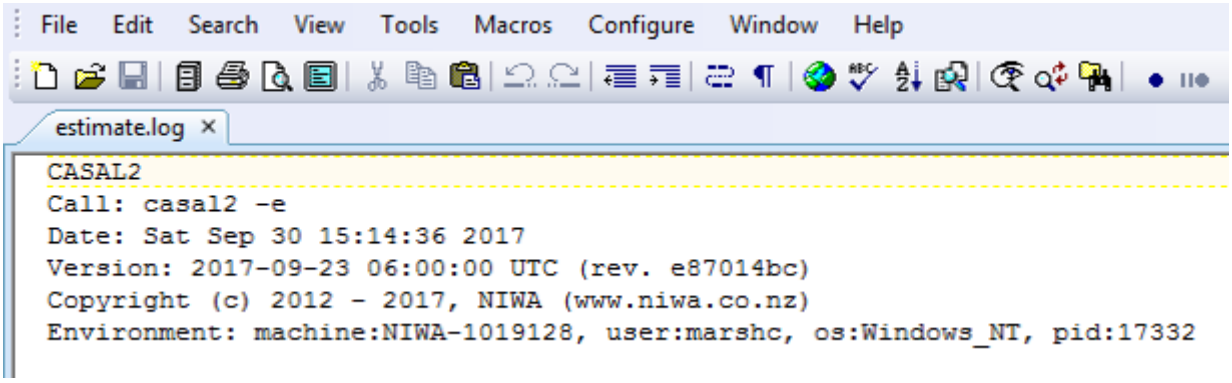

Figure 3.1: Example of output file header information.

3.3. Command line arguments

CASAL2 is called using:

casal2[-c *config_file*] [*task*] [*options*]

where

-c *config_file* Define the input configuration file for CASAL2 (if this argument is omitted, the default input configuration file is `config.csl2`)

and where *task* must be one of the following ([] indicates a secondary label to call the task, e.g. **-h** will execute the same task as **--help**),

-h [--help] Display help (this page)

-l [--licence] Display the reference for the software license (GPL v2)

-v [--version] Display the CASAL2 version number

-r [--run] *Run* the model once using the parameter values in the input configuration file, or optionally with the parameter values from the file specified with argument `-i filename`

-e [--estimate] Do a point *estimate* using the values in the input configuration file as the starting point for the parameters to be estimated, or optionally with the starting parameter values from the file specified with the argument `-i filename`. By default, it generates the file `mpd.out` with the estimated parameters and the covariance matrix; use `[--no-mpd]` to stop creating this file

[-p-profiling] Do a likelihood *profile* using the parameter values in the input configuration file as the starting point, or optionally with the starting parameter values from the file specified with the argument `-i filename`

-m [--mcmc] Do an *MCMC* chain. The the co-variance matrix is first estimated using the values in the input configuration file as the starting point for the parameters to be estimated, or optionally with the starting parameter values from the file specified with the argument `-i filename`. Use `-skip-estimation` to skip estimating the co-variance matrix and to read it in from mpd.out.

-f [--projection] *arg*. Project the model *forward* in time using the parameter values in the input configuration file as the starting point for the estimation, or optionally with the parameter values from the file specified with the argument `-i filename`. Repeat each parameter set *arg* times (default 1). Normally, the MCMC sample are used via `-i`

-s [--simulation] ***number*** *Simulate* the *number* of observation sets using values in the input configuration file as the parameter values, or optionally with the parameter values from the file specified with the argument `-i filename`

and where the following optional arguments [*options*] may be specified

-i [--input] ***filename*** *Input* one or more sets of free (estimated) parameter values from `filename` (see Section 12 for details about the format of `filename`)

-o [--output] ***filename*** *Output* a report of the free (estimated) parameter values in a format suitable for `-i filename` (see Section 12 for details about the format of `filename`)

-g [--seed] ***seed*** Initialise the random number *generator* with `seed`, a positive (long) integer value (note, if `-g` is not specified, then CASAL2 will generate a random number seed based on the computer clock time)

--loglevel arg = {trace, finest, fine, medium} (see Section 8)

--tabular Run with `-r` or `-f` command to print `@report` in tabular format (see Section 8)

--single-step Run with `-r` to pause the model and ask the user to specify parameters and their values to use for the next iteration (see Section 3.6)

[--resume] Resume a MCMC chain

[--objective-file] *arg*. Objective file for resuming an MCMC is *arg*

[--sample-file] *arg*. Sample file for resuming an MCMC is *arg*

[--skip-estimation] For `[--mcmc]` run, skip estimating the covariance matrix; take it from the file `mpd.out`. This is a way use a re-calculated covariance matrix based on a previous MCMC run and start a new chain

[--no-mpd] For `-e [--estimate]` run, do not create the MPD file, `mpd.out`

-q [--query] ***object type*** *Query* an object type to print an extract of the object description and parameter definitions. An object can be defined as `block.type`, e.g. `casal2 --query process.recruitment_constant` will query the constant recruitment block.

3.4. Constructing the CASAL2 input configuration files

By convention, the input configuration file is a stub that names four other files that cover four broad sections:

- the description of the population structure, dynamics, and parameters (`population.csl2`,Section 5),
- the estimation methods and estimated variables (`estimation.csl2`, Section 6),
- the observations and their associated properties and likelihoods (`observation.csl2`, `Section 7`), and
- the results that CASAL2 will output (`reports.csl2`, Section 8).

The way to do the above is to use the command `!include "filename"` (Figure 3.2). This command specifies that another file, `filename`, be read and processed, exactly as if its contents had been inserted into the main input configuration file at that point. The file name must be the complete file name with extension, and can use either the relative or absolute path as part of its name. Included files can also contain `!include` commands. See Section 13 for more detail.

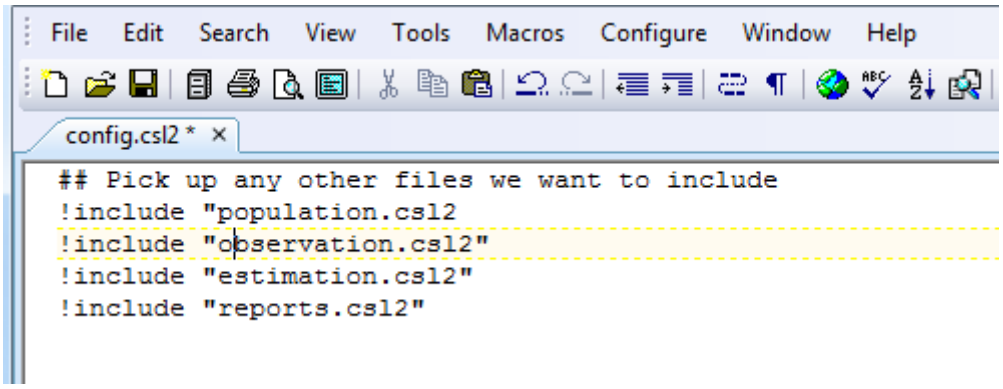

Figure 3.2: Example of using the input configuration file command `!include "filename"`.

The command and subcommand syntax to be used in each of these configuration sections are listed in Sections 9 (Population), 10 (Estimation), 11 (Observation) and 12 (Report).

3.4.1. Commands

CASAL2 has a range of commands that define the model structure, processes, parameters, observations, and how tasks are carried out. There are three types of commands

- Commands that have an argument and do not have subcommands (for example, `!include filename`)
- Commands that have a label and subcommands (for example `@process` must have a label and has subcommands)
- Commands that do not have either a label or argument, but have subcommands (for example `@model`)

Apart from `!include`, commands start with an `@` in column 1. Otherwise, inputs are free form. The `@` is important as it signals the start of a command block which is the basic unit in the input files (see sub-section **??**).

Commands that have a label must have a unique label, i.e., the label cannot be used on more than one command of that type so that it can be referenced in other parts of the input configuration file. The labels can contain alpha numeric characters, period ('.'), underscore ('_') and dash ('-'). Labels

must not contain whitespace (tabs or spaces) or other characters that are not letters, numbers, dash, period, or an underscore. For example,

```
@process NaturalMortality
or
!include MyModelSpecification.csl2
```

3.4.2. Subcommands

CASAL2 subcommands define options and parameter values related to a particular command. Subcommands always take an argument which is one of a specific *type*. The argument *types* acceptable for each subcommand are defined in Section 13, and are summarised below.

Like commands (`@command`), subcommands and their arguments are not order specific, except that that all subcommands of a given command must appear before the next `@command` block. CASAL2 may report an error if they are not supplied in this way. However, in some circumstances a different order may result in a valid, but unintended, set of actions, leading to possible errors in the expected results.

The argument type for a subcommand can be:

switch	true/false
integer	an integer number
integer vector	a vector of integer numbers
integer range	a range of integer numbers separated by a colon, e.g. 1994:1996 is expanded to an integer vector of values (1994 1995 1996)
constant	a real number (i.e., a double)
constant vector	a vector of real numbers (i.e., a vector of doubles)
estimable	a real number that can be estimated (i.e., a double)
estimable vector	a vector of real numbers that can be estimated (i.e., a vector of doubles)
addressable	a real number that can be referenced but not estimated (i.e., an addressable double)
addressable vector	a vector of real numbers that can be referenced but not estimated (i.e., a vector of addressable doubles)
string	a categorical (string) value
string vector	a vector of categorical values.

Switches are characteristics which are either true or false. Enter *true* as `true` or `t`, and *false* as `false` or `f`.

Integers must be entered as whole numbers without decimal points (i.e., if `year` is an integer then it is specified as `2008`, not `2008.0`)

Arguments of type integer vector, constant vector, estimable vector, addressable vector, or categorical vector must contain one or more entries on a row, separated by whitespace (tabs or spaces). Arguments of type integer range must contain a colon (`:`) and no whitespace (tabs or spaces).

Parameters are defined in the population section and can be specified as estimable with the subcommand type `estimable` or `estimable vector`. These parameters will be estimated if specified as such in the estimation section. If an estimable parameter is not specified in the estimation section it will instead be treated as a constant (or constant vector). In other words, only estimable parameters can be estimated and the parameter command must explicitly specify that the parameter is estimable with the `estimable` or `estimable vector` subcommand type. TODO: check

Parameters defined as addressable with the subcommand type `addressable` or `addressable`

`vector` are usually derived quantities and are not directly estimable. As such, they can be referenced by various processes, or have priors and/or penalties associated with them, but they do not directly contribute to any estimation within the model. WHAT IS ADDRESSABLE????

3.4.3. The command block format

The command block is a basic continuous unit within the input file(s). It begin with a line that starts with the symbol `@`, juxtaposition against a single command and followed, if required, by any unique label or argument. There is no indicator of the end of a command block. Each command block is delimited by the end of the file or the start of the next command block (which is marked by the `@` on the first character of a line). The `!include` command is the only exception to this rule (see Section 13 for details of the use of `!include`). The `!include` command can be used to include contents of another file within the scope of the command block.

Within the command block, any subcommands needed and their arguments are presented,

`@command`	`@command argument`	`@command` *`label`*
`subcommand argument`	`subcommand argument`	`subcommand argument`
`subcommand argument`	`subcommand argument`	`subcommand argument`
.	.	.
.	.	.
etc.	etc.	etc.

Subcommands can be in any order within the command block. Command blocks can be in any order within the input files, except `@model` which must be the first command block encountered by CASAL2 since this scopes the basic model dimensions (but excluding the `!include` command).

Blank lines are ignored, as is extra whitespace (tabs and spaces) between arguments. However, to start command block the `@` character must be the first character on the line and must not be preceded by any whitespace. Each input file must end with a carriage return.

Commands, subcommands, and arguments in the input configuration files are not case sensitive. However, labels and variable values are case sensitive. On Linux, filenames and paths are case sensitive (i.e., when using `!include` *`filename`*, the argument *`filename`* will be case sensitive).

3.4.4. Commenting out lines

Text on a line that starts with the symbol # is considered to be a comment and is ignored. To comment out a group of commands or subcommands, use # at the beginning of each line to be ignored.

Alternatively, to comment out an entire block or section, use /* at the beginning of a line to start the comment block, then end the block with */. All lines (including line breaks) between /* and */ inclusive are ignored.

```
# This line is a comment and will be ignored
@process NaturalMortality
m 0.2
/*
This block of text
is a comment and
```

```
will be ignored
*/
```

3.4.5. How to reference parameters

Parameters need an unique name so it can be referenced in other command blocks, e.g., specifying it to be estimated, to apply a penalty, specifying values to use in projections, making it time varying, or profiling.

When CASAL2 processes the input configuration file it translates each command block and each subcommand block into a CASAL2 object, each with a unique parameter name. For commands, this parameter name is simply the command label. For subcommands, the parameter name format is one of the following:

`command[label].subcommand` if the command has a label, or

`command.subcommand` if the command has no label, or

`command[label].subcommand{i}` if the command has a label and the subcommand arguments are a vector, and we are accessing the *i*th element of that vector.

`command[label].subcommand{i:j}` if the command has a label, and the subcommand arguments are a vector, and we are accessing the elements from *i* to *j* (inclusive) of that vector.

For example, the parameter name of the natural mortality rates subcommand `m` of the command `@process` with the label `NaturalMortality` and there is a different rate for `males` and `females` has the form to reference both rates as a vector is

`process[NaturalMortality].m`, but to reference just the male rate then the form is `process[NaturalMortality].m{male}`.

All labels (parameter names) are user specified. As such, naming conventions are non-restrictive and can be model specific.

3.5. Reading a command block

Here, we illustrate reading a command block using two important commands, `@process` and `@estimate`.

The command `@process` specifies a process that can be used in the model. The are a fixed set of predefined processes (subroutines in C++ code). The way to identify which one is by the `type` subcommand. Processes can take one or more parameters and some will need other data to be applied too. Some parameters are mandatory and others can take a default value if they are not specified. We have categories male and female, and two fisheries, line and pot. The command block starts with a@*process*:

```
@process Fishing
type mortality_instantaneous
```

This sets up a process block using the *mortality_instantaneous* process which simultaneously depletes the population by natural mortality and from two types of fishing. Its label is *Fishing*.

Next we specify the values for natural mortality (m) an argument to this process, to 0.17 and specify that fisheries acts on all categories. Note there are two values for natural mortality, one for each sex. The parameters m can be estimated, if required. The command block fragment:

```
m 0.17 0.17   # natural mortality  for each category
relative_m_by_age One One # natural mortality multiplier
categories *  # fishing acts on all categories ("*" shorthand for male female)
```

Catches are supplied via a *table* format using three columns: year and two for the fisheries, one for each, which take the labels *line* and *pot*. Column names are on the first line of the table and these columns can be in any order,

```
#catches
table catch         # define catches by fishery in table format
year line pot       #names columns so can identity catch for each fishery
2000 1000 2000      # catches by year
2001 500  1000
2002 1000 5000
end_table           # end of table marker
```

Other information needed are supplied in the methods table which has a fixed number of columns (again these can be in any order), one for each piece of information needed to model a fishery. The method column defines the fishery name which is used in the catch table and also in other observations like age composition from that fishery. The categories that the fishery operates on (all in this case, but it could be just males for one and females for the other) are in the category column, the fishing selectivity to be used is given as a selectivity block name which is define somewhere else in the files, *u_max* is the maximum exploitation rate in any year that is allowed, next comes the time step the fishing operates in, and lastly the block name of a penalty function that is used to penalise estimable parameter values that result in the supplied catch not being caught. Again, the penalty block is define elsewhere in the files. After the row with the column names, there follows one row for each fishery:

```
table method        # supply arguments and name selectivity etc
method   category  selectivity  u_max  time_step  penalty
pot         *          potFSel    0.7  1  CatchMustBeTaken1
line      *         lineFSel      0.7  1  CatchMustBeTaken1
end_table
```

To estimate natural mortality, you need to supply an @*estimate* block with a reference name back to *m* in the *Fishing* block. For @*estimate*, *type* specifies the prior to be used in the estimation, which in this case is a normal distribution:

```
@estimate estimate.m
type normal normal
parameter process[fishing].m  #
    \*Fishing is unique amongst the @process command blocks
    so     this defines the unique reference to the parameter m
    *\

mu  0.2 0.2   #argument to prior = mean
sd  0.02 0.02 #another argument to the prior = standard deviation
```

Note that there are two *m*s, one for each sex, so there has to be two priors. The *estiamte* label *estimate.m* is often redundant, but it may be needed, e.g., in a transformation command block.

To estimate a common *m* over both sexes, we estimate one *m*, say male, and use the *same* subcommand to apply the same value to the female *m*,

```
@estimate estimate.m
type normal
parameter process[fishing].m{1}  #male M since males are first in the category order
same process[fishing].m{2}       #{} is used to index one or more elements in a vector
mu  0.2    #argument to prior = mean
sd  0.02 #another argument to the prior = standard deviation
```

There are section detailing the equations involve and a separate syntax section that gives the possible `types` and their subcommands.

Note that the syntax sections are auto-generated from the code base (from the class constructors) so they give the definitive set of subcommands and their names.

3.6. Single-stepping CASAL2

TODO: move this section to somewhere else

Single-stepping means CASAL2 can 'pause' after each year in the annual cycle during a model run, write reports, then wait and process user input of updated estimable parameters for the next year (see the command line argument `--single-step`).

This enables CASAL2 to implement models for management simulations or scenarios that require feedback and can be used, for example, in operational management procedures (OMPs). The single-stepping process can be automated using **R**, so that CASAL2 may be used with **R** to update input harvest values (e.g., catches from a fishery in a fisheries model) to evaluate a particular harvest control rule.

3.7. CASAL2 exit status values

When CASAL2 is run, it will complete its task successfully or output errors. CASAL2 will return a single exit status value 'completed' to the standard output. Error messages will be printed to the console. When input file configuration errors are found, CASAL2 will print error messages, along with the associated filename(s) and line number(s) where the errors were identified, for example,

```
#1: At line 15 in Reports.csl2: Parameter '{' is not supported
```

4. Partition & Categories

Dividing the population into different categories is fundamental to modelling the dynamics of a fish stock. CASAL had a fixed set of hard-wired categories (e.g., factors like sex, area, or stock) and each category type had a predefined set of allowed processes (or transitions in CASAL-speak), e.g., immature fish moving into mature category (Bull et al., 2012). This made sense when CASAL was coded, but now it is seen as a limitation, e.g., changing sex was not allowed and there can only be male and female sexes, not an unknown sex that sometimes occurs in data. [TODO: shift to intro or drop?]

In CASAL2 , the concept of user defined categories was introduced to allow more flexibility in dividing up the population. Note that CASAL2 does not know about sex or area and their properties; these are explicitly built up by the user by specifying processes that act on the categories in the input files. The cost is that users need to follow good practice to achieve clarity and readability of the input files, i.e., poor specifications can result in obscure input files.

4.1. Specifying the partition using categories

A key element of the model is the partition which holds the current state of the population. The partition can be conceptualised as a matrix, where each row represents a category and the columns are the age classes (Figure 4.1). Each row represents all individuals in that category as a numbers-at-age vector. There must be at least one category defined for each model.

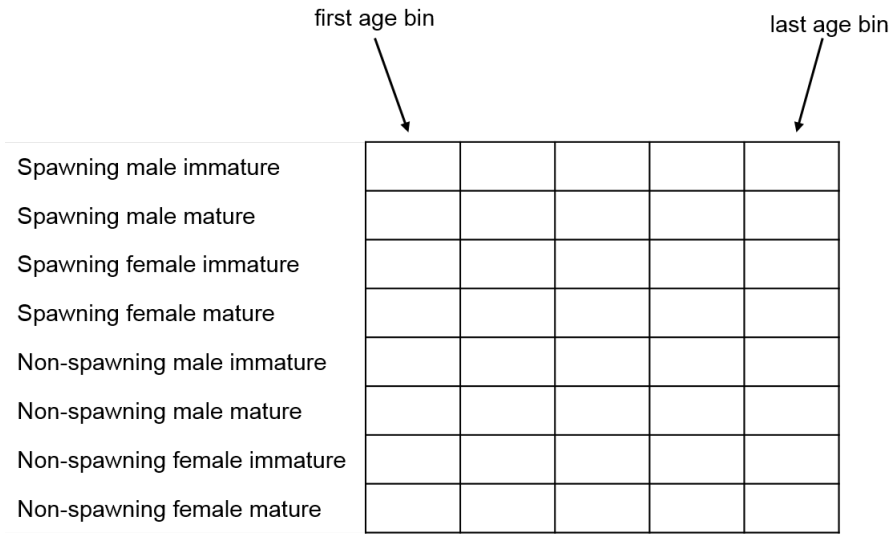

Figure 4.1: A visual representation of a partition.

The categories can include combinations of levels from one or more factors such as sex, maturity state, area, or even species. CASAL2 has no predefined categories; *all* categories are defined by the user. Note that the partition only has the current state of the model; past states are not kept (*See* the section on derived quantities about saving past summaries from the partition, p. 53).

To illustrate categories, consider a model of a fish population with two fisheries, one on spawning fish at the spawning grounds and another on the non-spawning population in the rest of the stock area. The mature fish will migrate to the spawning area, where the spawning fishery occurs. At the end of spawning, these fish, along with the recruits from the previous year, migrate back to the non-spawning area. The fish population can be represented by factors sex (levels *male* and

female), maturity (levels *immature* and *mature*), and area (levels *spawning* and *non-spawning*). So the partition has 8 rows of numbers-at-age, from 2 sexes $\times$ 2 maturity levels $\times$ 2 areas.

These categories are specified in a categories block which starts with a @*categories* line followed on the next line by a *format* subcommand that specifies the factors to use and their order. Factor names are user defined and have no intrinsic meaning to CASAL2). The command block is:

```
@categories
format area.sex.mature
names spawn.male.immature spawn.male.mature spawn.female.immature spawn.female.mature
    nonspawn.male.immature nonspawn.male.mature nonspawn.female.immature #all on one line
    nonspawn.female.mature
```

Note the "." syntax to separate the factor names.

Next comes the *names* subcommand which specifies the combinations of levels that makes up each category. In a sense, the *format* subcommand is not needed since the *names* subcommand can define all categories. However, *format* allows a more digestible and shorter syntax to define categories here and in other blocks such as matching observation to categories that provided the data (including combinations of categories, e.g., age compositions that combine both sexes).

For example, the *names* subcommand can be specified by:

```
names spawn,nonspawn.male,female.immature,mature
```

which defines the categories above in a more efficient manner, (again, note the "." to separate the factors and "," to separate the levels within each factor (*see* the next section for more details). A visualisation of the partition is in Figure 4.1.

When using the short-cut syntax in *names*, the order of level combinations is for the levels of the first factor to change the slowest, then the next factor will change faster, and so on with the last factor to changing levels the fastest. The order is important because linking categories to their characteristics, e.g., growth curve or selectivity, is done in other subcommands where these must be specified in the same order.

To exclude unused categories from the partition, the long form must be used in the *names* subcommand, e.g., to exclude *spawn.female.immature* and *spawn.male.immature* since they are never in the spawning area.

To make recruitment to enter the partition in the non-spawning area, use

```
@categories
format area.sex.mature
names spawn.male.mature spawn.female.mature nonspawn.male.immature nonspawn.male.mature
    nonspawn.female.immature
```

4.2. Shorthand syntax for categories

This can be skipped on the first reading.

Some specifications have long lists of categories or years or initial values for parameters and the like, e.g., for YCS from 1900 to 2019, 120 years and 120 initial vales of YCS must be specified; this is hard to do by hand and it can be error prone as well as difficult to match values for each year. Here, the range short cut (*:)* can be used so the the year specification is *1900:2019*, and the multiplier short cut (**)* to give the initial values specification as *1*120*.

There is also shorthand notation for categories since each category can be quite complicated. . First use the `format` subcommand in the *@categories* block to define the factors that make up the sections of the category names. A "." (period) character delineates each factor and this structure allows a shorthand syntax to compose category names.

The `names` subcommand is used to list the category names. Sections within the shorthand syntax for *names* are required to match the order of factors in the *format* subcommand so CASAL2 can organise and search on them. In these sections, levels for each factor use the "list specifier" and range characters, e.g.,

```
@categories
format sex.stage.tag # 2 sexes, 2 stages, tag years 2001 to 2005 = 20 categories

names male.immature # Invalid: No tag information
names female # Invalid: no stage of tag information
names female.immature.notag.1 # Invalid: Additional format segment not defined

names male,female.immature,mature.notag,2001:2005 # Valid shortcut

# Without the shorthand syntax these categories would be written:

names male.immature.notag male.immature.2001 male.immature.2002 male.immature.2003 male.
    immature.2004 male.immature.2005 male.mature.notag male.mature.2001 male.mature.2002 male.
    mature.2003 male.mature.2004 male.mature.2005 female.immature.notag female.immature.2001
    female.immature.2002 female.immature.2003 female.immature.2004 female.immature.2005 female
    .mature.notag female.mature.2001 female.mature.2002 female.mature.2003 female.mature.2004
    female.mature.2005
```

The shorthand syntax available are: [TODO: edit list]

- * Specify all categories
- + Categories join, e.g., *categories* *+ joins all categories together into one unit; *categories male+female* specifies that the observation covers both sexes combined.
- *:* Specify a range of integers <int1>:<int2>, e.g., *2000:2005* expands to *2000, 2001, 2002, 2003, 2004, 2005*
- Lists using ",": <item1>,<item2>,<item3>, e.g., *male,female,unsexed* are the levels for the factor *sex*.
- Repeats a number or label: <number | label>*<integer>, e.g., *1*5 –> 1 1 1 1 1*
- *format=<X>=<x>=<int>* ??? no idea how it works *<factor>=level>=<year range>*, e.g., *tag=2001=1999:2003* the categories with level 2001 in the tag factor are accessible from year 1999 to 2003 inclusive, i.e, these categories are not not part of the partition in the years prior to 1999 (saves computation effort). format=??
- *[]* replace label to a command block with the block defined inline, e.g., *catchability [q = 1e-5]* rather than *catchability CHATq* where *CHATq* labels a command block somewhere in the input files

Example of specifying categories using the short cuts:

This syntax is the long way:

```
@categories
format sex.stage
names male.immature male.mature female.immature female.mature
```

A shorter way to specify the exact same partition structure using *lists*:

```
@categories
format sex.stage
names male,female.immature,mature
```

CASAL2 requires categories in processes and observations so that the correct model dynamics can be applied to the correct elements of the partition.

This block illustrates using categories required for the ageing process:

```
# 1. The long-hand way
@ageing my_ageing
categories male.immature male.mature female.immature female.mature

# 2. The first shorthand way
@ageing my_ageing
categories male,female.immature,mature

# 3. Wild Card (all categories)
@ageing my_ageing
categories *

# 4. The second shorthand way (\textit{obscure}
@ageing my_ageing
categories sex=male sex=female
```

To combine/aggregate categories together, use the "+" special character. For example, this feature can be used to specify that the total biomass of the population is made up of both males and females.

For example,

```
@observation CPUE
type biomass   # observation using an index of biomass
categories male+female
        ...   # other subcommands to link index to the fishery etc
```

This combination/aggregation functionality can be used to compare an observation to the total combined population:

```
@observation CPUE
type biomass
categories *+
 ...   # other subcommands to link index to the fishery etc
```

If the levels `male` and `female` are the only categories in a population (i.e., factor sex), then this is the same syntax as the command block above it.

Shorthand syntax can be useful when applying processes to a select group of categories from the partition.

For example, to apply a spawning migration to the mature categories in the partition giving the partition definition:

```
@categories
format area.maturity.tag
names north,south.immature,mature.notag,2001:2005
```

Then, to migrate a portion of the mature population from the southern area to the northern area:

```
@process spawn_migration
type transition_category   #process to move fish from one category to another
from format=south.mature.* #move all south mature fish, both notag and tagged fish
to format=north.mature.*   # into the relevant north categories
```

Advanced partition syntax

Specific data for a year in a category can be set up so that this category is not to be processed during specific years or in the initialisation phases. Years that each category is available can be specified and these years which will override the default setting of all years in the model. Any category with the default years overridden will no longer be accessible in the initialisation phases.

Examples:

```
@model
start_year 1998   #the model starts in 1996 and
final_year 2010   # goes through to 2010

@categories
format sex.stage.tag
names male,female.immature,mature.notag,2001:2005 # Valid
# Specify categories with the tag value "2001" are available in years
        #1999, 2000, 2001, 2002, 2003
# And also for categories with the tag value "2005" are available in years
        #2003, 2004, 2005, 2006, 2007
years tag=2001=1999:2003  tag=2005=2003:2007
```

Category	Age					names syntax fragment
	1	2	3	4	5	
male.immature	0	0	0	0	0	male,female.immature,mature
male.mature	0	0	0	0	0	
female.immature	0	0	0	0	0	
female.mature	0	0	0	0	0	
male.immature.2005	0	0	0	0	0	male,female.immature,mature.2005:2010
male.immature.2006	0	0	0	0	0	
male.immature.2007	0	0	0	0	0	
male.immature.2008	0	0	0	0	0	
male.immature.2009	0	0	0	0	0	
male.immature.2010	0	0	0	0	0	
male.mature.2005	0	0	0	0	0	
male.mature.2006	0	0	0	0	0	
male.mature.2007	0	0	0	0	0	
male.mature.2008	0	0	0	0	0	
male.mature.2009	0	0	0	0	0	
male.mature.2010	0	0	0	0	0	
female.immature.2005	0	0	0	0	0	
female.immature.2006	0	0	0	0	0	
female.immature.2007	0	0	0	0	0	
female.immature.2008	0	0	0	0	0	
female.immature.2009	0	0	0	0	0	
female.immature.2010	0	0	0	0	0	
female.mature.2005	0	0	0	0	0	
female.mature.2006	0	0	0	0	0	
female.mature.2007	0	0	0	0	0	
female.mature.2008	0	0	0	0	0	
female.mature.2009	0	0	0	0	0	
female.mature.2010	0	0	0	0	0	
unsexed.immature	0	0	0	0	0	unsexed.immature

Table 4.1: Partition using: *names male,female.immature,mature male,female.immature,mature.2005:2010 unsexed.immature*

Category shortcuts can be used in a sequence to create a complex partition. For example, using the following creates the partition shown in Table 4.1.

```
@categories
```

```
format sex.maturation.tag
names male,female.immature,mature male,female.immature,mature.2005-2010 unsexed.immature
#<is equal to>
names male.immature male.mature female.immature female.mature
     male.immature.2005 male.immature.2006 male.immature.2007 male.immature.2008 male.immature
         .2009 male.immature.2010
     male.mature.2005 male.mature.2006 male.mature.2007 male.mature.2008 male.mature.2009 male
         .mature.2010
     female.immature.2005 female.immature.2006 female.immature.2007 female.immature.2008
         female.immature.2009 female.immature.2010
     female.mature.2005 female.mature.2006 female.mature.2007 female.mature.2008 female.mature
         .2009 female.mature.2010
     unsexed.immature
```

Note that the algorithm goes from left to right over the factors in the *format* subcommand. So substituting *unsexed* for *unsexed.immature* is valid and gives a category called *unsexed* in the *sex* factor. But substituting *.immature* (or *.zzz*) for *unsexed.immature* results in one less category and generates an error (i.e., no level provided for factor *sex*).

4.3. Referencing vector and map parameters

TODO: has ref been done above?? shift? or new sction?

To build relationships between command blocks, CASAL2 uses a referencing system so that blocks and parameters within blocks can be accessed. In its simplest form, command blocks are referenced by their label. To access specified parameters within a command block, the syntax used is:

```
<syntactic element>     #<>  enclosing a description of the element

<block type>[<label of block>].<parameter name>         # most used version
<block type>[<label of block>].method_<parameter name>  #e.g., identify a  fishery

#Examples
process[recruitment].ycs         #ycs parameter in the process block called recruitment
process[Fishing].m               #natural mortality in the process called Fishing

process[Fishing].method_pot      #pot fishery in the process called Fishing
                                 # it is usual to define all fisheries in one
                                 # mortality process block so we need a way to
                                 # identify each one
```

Parameters can be scalars (one value), vectors (several values), or maps. A map consists of two vectors: one containing a key value (for searching or uniquely indexing), and another vector that contains values associated with the index, e.g., specifying YCS values for each year; the years are the key (or index). To reference one or more components of a vector or map use the *{}* syntax. This is sometime needed when specifying which element(s) in a vector or map are to be estimated.

An example of a map parameter is `ycs_values` in a recruitment process

```
@process   WestRecruitment
type       recruitment_beverton_holt  # Beverton-Holt function
ycs_values 1 1 1 1 1 1 1 1      # initial values of the YCS (a vector with 9 values)
ycs_years 1975:1983             #key into YCS
# An alternative method to specify a sequence of values
# use an asterisks to represent a vector of repeating integers
ycs_values 1*8
```

To specify that only the last four years of the YCS parameter `process[WestRecruitment].ycs_values` are to be estimated:

```
@estimate YCS    #YCS is a label to identify this block
parameter process[WestRecruitment].ycs_values{1980:1983}  #estimate 4 values only: 1980 1981 1982 & 198
```

To estimate a common value for a block of years in a map parameter use the *same* subcommand. We illustrate the idea within the process `@time_varying[label].type=constant`, where we want to fix q over a specified block of years, 1992 to 1995.

First specify the relationships in a `@time_varying` block:

```
@time_varying q_step1
type          constant                #specify a set value for a year
parameter     catchability[Fishq].q   # parameter ref for q in block Fishq
years  1992 1993 1994 1995  #or 1992:1995 = key into value
value  0.2 0.2 0.2 0.2   # or 0.2*4, initial values of q
```

Next, to estimate only one q value for the time block, pick one element of the map (say 1992), and then force all other years to have the same value:

```
@estimate q_block_1992
parameter time_varying[q_step1].value{1992}  #estimate this one
same      time_varying[q_step1].value{1993:1995}  #set these to the value for 1992
type      uniform                   #uniform prior on q
lower_bound 0.1
upper_bound  10
```

Keys are restricted in CASAL2 to years and categories. An example using categories as a key in a map:

```
@category
factor sex
names male female

@process recruit
categories male female   #key for m
m  0.17 0.17             # natural mortality values indexed by categories
...

@estimate M
type uniform                      #prior
parameter recruitment.[m]{male}  # estimate male M, "male" is a level for factor sex
same recruitment.[m]{female}     # set female M to the same value as male's
```

For vector parameters (i.e., no key values), the index is an integer starting with 1 for the first value. An example is the selectivity *all.values.bounded* which can be defined by:

```
@selectivity MatSel
type         all_values_bounded
L            2              #lower bound at age 2
H            4              # upper bound at age 4
```

```
v            0.1 0.2 0.7   # 3 values, one for each age 2, 3, and 4

@estimate  mature
type       uniform                   #prior
parameter selectivity[MatSel].v{2}  # estimate the 2nd value only, i.e., age 3
lower_bound 0.1                     # lower parameter range
upper_bound 1.0                     # upper parameter range
```

The integer *2* cannot be used to specify the *q* parameter for 1993 in the above example labeled *q_block_1992*. This will pass the syntax test, but it will fail at the validate stage in CASAL2.

In-line declaration, avoiding extra command blocks

In-line declarations can help shorten models by defining `@` blocks within the subcommand line instead of having a label that points to a command block define somewhere else in the input files.

For example, catchability for an CPUE index series can be defined inline:

```
@observation chatCPUE
type biomass                   # biomass index
catchability [q=6.52606e-005]  # define catchability here
categories male+female         # index cover both sexes together


@estimate chatCPUE_q
parameter catchability[chatTANbiomass.one].q #how to reference q
type uniform_log     # prior
lower_bound 1e-2
upper_bound 1
```

In the above code catchability is defined and estimated without explicitly creating a `@catchability` block.

4.4. Examples? not needed?

5. Specifying the population model

5.1. Introduction

This section shows how to specify a model for the population dynamics. It describes the model time and age scope, the population processes used (e.g., recruitment, ageing, migration, and mortality), the selectivities, and how to set values for their associated parameters, or starting values if they are going to be estimated.

The basic structure of the population is defined in terms of its partitions and the succession of processes that act on them throughout a year. CASAL2 assumes an annual cycle, i.e., rates like natural mortality are assumed to be for a year. To place certain processes or observations (e.g., a research survey) into the right part of the year, the year can be divided into one or more time steps, and each time step needs at least one process. Each time step can represent a specific period of the calendar year, or it can be an abstract sequence of events. Certain processes like natural mortality and growth can have a proportion of the effects of the process assigned to different time steps to crudely mimic seasonal effects, or fisheries that occur in short periods of the year, as well as place a

survey within the year relative to the proportion of annual natural mortality that has occurred (5.4, p. 53).

The *state* is the current status of the population at any given time and it can change one or more times during the year. The state object must contain sufficient information to determine how the population changes over time, given a model and a complete set of parameters. The partition is key to the state, but it has no "memory". Thus, other information must also be kept, such as the mature biomass from a previous year or time step to calculate the recruit numbers into first age class via the spawner-recruitment function. The latter are specified as *derived variables* and they are kept for the whole model run. However, the *derived variables* record only summary information from the partition at a specified time step and year.

Processes can change the partition and examples include recruitment, natural mortality, fishing mortality, ageing, migration, and maturation. These processes are repeated for each year of the model, although some processes can be specified to occur in a subset of years only.[REPEATED?]

The specification and ordering of processes in multiple time steps can be used to represent complex dynamics, with the intermingling of multiple species and stocks, migration patterns occurring over multiple areas, and/or multiple sources of anthropogenic impacts using a range of methods which cover different areas and times. However, the complexity of a stock structure definition is constrained by the available data. It is challenging to use a complex structure to model a population when there are no observations to support that structure. For information on how to define categories and use the shorthand syntax see Section 15.2.

Topics covered are:

- The model scope, such as the ages covered, the years over which the model runs, and the end year for projections;
- Linking processes, such as growth to each category;
- The number of time steps and the processes that are applied in each time step;
- The specification of and the parameters for the population processes: processes that add or remove individuals from a partition, or shift individuals between ages and categories in a partition;
- The initialisation process: the state of the partition at the start of the first year;
- Defining selectivities and linking them to observations;
- The parameters: their definitions, initial values, and other characteristics; and
- Derived quantities, e.g., mature biomass, to include in density-dependent processes such as the spawner-recruit relationship

5.2. Model scope and structure

The model needs scoping for ages and year covered. This is done in the @*model* command block.

Each CASAL2 model requires:

- The minimum and maximum population ages
- Whether the maximum age is a plus group
- The start and final year
- The names of all of the categories

The ages used starts at the minimum age through to the maximum age in steps of one. The model is run from the start year through to the final year. It can also be run past the final year to project the state of the population through the final projection year.

An example of how to specify a potential model with two categories is outlined below; the `@model` and `@categories` blocks are:

```
@model
start_year 1981
final_year 2000
projection_final_year 2010
base_weight_units     tonnes
min_age     1
max_age   20
age_plus_group        true
initialisation_phases Equilibrium_phase
time_steps            step1 step2 step3

@categories
format      sex
names       male female
age_lengths male_growth female_growth  #labels for growth blocks
```

This model runs for 20 years, starting in 1981, and project over 10 years for a population aged between one and twenty with age 20 being the plus-group, i.e., all ages 20 and over are accumulated there. Each year is divided into three time-steps. The categories are male and female (one factor, sex) and each category has an age-specific age-length relationship (growth).

Whist CASAL2 generally uses generic formulation, it does have some specific population concepts, in this case, growth which can vary for each category. Additionally, there is a length-weight concept which is specified in the age-length blocks, here blocks starting with *@age_size male_growth* and *@age_size female_growth* that are placed elsewhere in the input files (not shown).

CASAL2 allows categories of the partition to exist for a subset of years of a model. This feature enables more efficient computations when models contain categories that do not persist over all model years. A model may define one-off processes that transition individuals from one category into another in a subset of the model initialisation phases or years (e.g., tagging events). Excluding categories for certain years can be more efficient as CASAL2 will not initialise these categories or apply processes to categories in years or time steps in which they do not exist.

The structure of the partition is defined in a configuration block with the `@categories` block (Section 5.2).

Derived quantities are another important component of the state object. An example of a derived quantity is spawning stock biomass (SSB; the biomass of [female] spawning fish calculated at the mid point of the spawning season). CASAL2 calculates derived quantities using the command `@derived_quantity`, which may be required for some processes. In fisheries stock assessment models, a recruitment process which includes a stock-recruitment relationship requires the definition of a derived quantity that specifies the mid-season spawning stock biomass (5.4, p. 53).

[TODO: SHIFT SOMEWHERE] For this example four time steps are defined and labelled 1 through 4: `step1` for the non-spawning fishery period, `step2` for the migration to the spawning area, `step3` for the spawning fishery period, and `step4` for recruitment and migration back to the non-spawning area. The default order of processes within a time step has migrations occurring before fisheries (TODO: check this CASAL concept), so that the processes in steps 2 and 3 could have occurred in

one time step. Other details that describe the population structure are also linked to time steps, such as proportion of natural mortality occurring in each time step and in which time step the observations occur.

5.2.1. The implicit annual cycle

There is an implicit annual cycle that orders the sequence of processes within the year, but there is no command block as such. The implementation is by ordering processes within the time-steps. This sequence is repeated for every year. Time steps are used to break the year into separate components and allow observations to be associated with specific time periods and processes. Any number of processes can occur within each time step, in any order, although there are restrictions for mortality-based processes (see Section 5.3.3.1); processes can occur multiple times within each time step. Time steps are not implemented during the initialisation phases (effectively there is only one initialisation time step), and the annual cycle in the initialisation phases can be different from the annual cycle specified for the model years (5.2.2).

Figure 5.2 shows an example of the annual cycle using three time-steps. [TODO: REDO FIGURE]

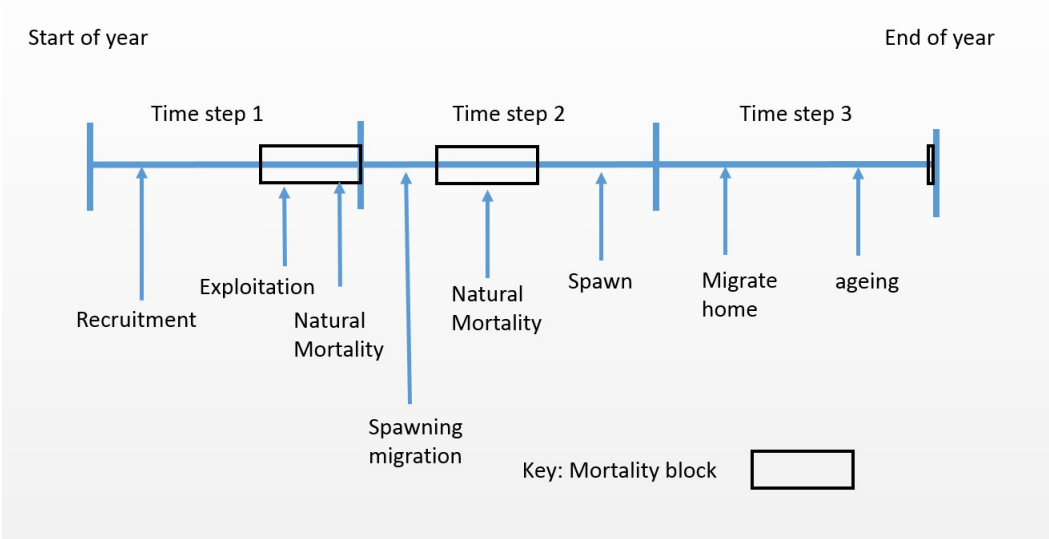

Figure 5.1: A example sequence for an annual cycle.

This would be specified using @time_step block:

```
@model
time\_steps step1 step2 step3
```

This gives the order and labels for each time step, i.e., 3. Processes are sequenced using order within the *@time_step* block:

```
@time_step step1
processes Recruitment Fishing

@time_step step2
processes Spawn_migration Fishing

@time_step step3
processes Home_migration Ageing
```

The *Recruitment, Fishing,Spawn_migration, Home_migration* and *Ageing* are all labels to command blocks where these processes are parameterised (see 5.3 for allowed processes). The order that the processes are executed is in the same order as specified. The process *Fishing* could be the process type *InstantaneousMortality* (5.3.3.4) which takes natural mortality as a parameter as well as specifying the catches in the time-steps, so it is possible to have all catch taken in time-step *step1* with some natural mortality, and no fishing in time-step *step2* where the rest of the natural mortality occurs.

Although *Spawn* represents a biological process, spawning, for the usual modelling it is the time that the spawning stock biomass is calculated since this is needed to calculate recruitment is there is a spawning biomass recruitment relationship. A related concept is maturity which can be in the partition, so there needs to be a transfer of immature fish into the mature category, i.e., a process, but it is only indirectly related to spawning. Hence, in modelling, spawning is not a process that affects the partition directly, but it the time to calculate the SSB which must be setup as a derived variable (from the partition). Hence, *Spawn* is located in Figure 5.2 and it is an important component in stock assessment.

To calculate the SSB a @*derived_quantity* command block is needed in which the "timing" of the SSB calculation in terms of which time-step and the proportion of natural mortality within it is specified (5.4).

5.2.2. The initialisation phases

Initialisation is the process of determining the model starting state at the start of the first year (*Start_year*. The initial state can be equilibrium/steady state or some other initial state for the model (e.g., exploited), prior to the start year of the model.

There are multiple options for partition initialisation in CASAL2, including

- Iterative: run the model for a specified number of years to get the converged state.
- Derived: Use the analytical solution (i.e., faster than iterative) for the initial state, but it does not work for some processes (e.g., density dependant migration)
- Cinitial: Allow the estimation of the initial partition's numbers-at-age
- state_category_by_age: specify the partition's numbers-at-age

Initialisation specifications starts with nominating the initialisation label in the @*model* command block followed by a @*initialisation_phase* command block specifying the type and other settings:

```
@model
...     # other subcommands
initialisation_phase int_label

@initialisation_phase int_label
type iterative  #choose one from the list above
...             # specify option values
```

If needed, the processes used and their order in the initialisation are those specified in the annual cycle, but these can by changed by either excluding some processes or including others by using the `exclude_processes` or `insert_processes` subcommands in the *initialisation_phase* command blocks,

```
@initialisation_phase int_label
type iterative
exclude_processes Fishing
insert_processes step1(recruitment)=initialFishing
            #format=<step>(<insert before label>)-<new block label>
...             # specify option values
```

where *Fishing* is the normal fishing process which defines natural mortality so when excluded, initialisation can use another value that incorporates some unrecorded fishing before the start of the assessment period by setting natural mortality to a higher value in the process *initialFishing*. The place to insert *initialFishing*is in the time-step labeled *step1* before the process *recruitment* which must be in that time-step (process label is enclosed in brackets). To insert at the end of the time-step use *()*, i.e. *step1()=initialFishing*.

Normally,the type *iteration* is used, but more complicated initialisation can by used by sequencing other phases one after another,

```
@model
...     # other subcommands
initialisation_phase int_label int_label2


@initialisation_phase int_label
type derived    #choose one from the list above
...             # specify option values

@initialisation_phase int_label2
type iterative    #choose one from the list above
...             # specify option values
```

which may be faster overall since less iterations can be used in the second phase. The order of applying each initialisation is that given in the *@model* command block.

The multi-phased initialisation allows for flexibility in the number and type of initialisation, for initialising a non-equilibrium starting state, or applying simple processes before applying more complex ones.

In each initialisation phase, the processes defined for that phase are applied and used as the starting point for the following phase or, if it is the last phase, the start year of the model.

The *first* initialisation phase is always initialised with each age and category set to zero. Care must be taken when using complex category inter-relationships or density-dependent processes that depend on a previously calculated state, as they may fail when used in the first phase of an initialisation.

Multi-phase iterations can also be used to determine if an initialisation has converged. A second initialisation phase can be added for 1 year, with the same processes applied as in the first phase. The state at the end of the first and second phase is then output. If these states are identical, then it is likely that the initialisation has converged to an equilibrium state.

5.2.2.1. Iterative Initialisation

The `iterative` initialisation is a general solution for initialising the model, but can be slow to converge, depending on the model.Its value is that it can work on complex structured models that

may be difficult or impossible to implement using analytic approximations.

The number of iterations in the iterative initialisation can increase the model output, and the number of iterations should be chosen to be large enough to allow the population state to fully converge. A period of about two times the maximum age is recommended to ensure convergence. CASAL2 can be configured to report convergence statistics that can assist in determining convergence properties.

In addition, the iterative initialisation phase can optionally be stopped early if user-defined convergence criteria is met. For a list of supplied years in the initialisation phase, the convergence criteria is met if the proportional absolute summed difference between the state in year $t-1$ and the state in year t ($\widehat{\lambda}$) is less than the user-defined value of λ, where

$$\widehat{\lambda} = \frac{\sum_{i,j} \left| \text{element}(t)_{i,j} - \text{element}(t-1)_{i,j} \right|}{\sum_{i,j} \text{element}(t)_{i,j}} \tag{5.1}$$

where $\text{element}(t)_{i,j}$ denotes the numbers at time step t in category j and (age?) class i.

Hence, for the initialisation define:

- The number of initialisation phases,
- The number of years in each phase, and
- The processes to apply in each phase, where the default processes are those applied in the annual cycle

An example with one initialisation phase:

```
@model
...
initialisation_phases Iterative_initialisation

@initialisation_phase Iterative_initialisation
type iterative
years 50                # do 50 iterations
lambda 0.0001
convergence_years 20 40 # test for convergence at 20 and 40 iterations
```

5.2.2.2. Derived Initialisation

The `derived` initialisation is an analytical solution that calculates the equilibrium age structure and the plus group using a geometric series solution. The benefit of this method is it can be solved in `max_age - min_age + 1` years or time-steps units, so it is computationally faster than the iterative initialisation phase. Under some process combinations (e.g., one-way migrations) this initialisation does not calculate the exact equilibrium partition. When using this initialisation, confirm that the partition has reached an equilibrium state by either comparing with an iterative initialisation, or by adding a second iterative initialisation phase with a limited number of iterations for comparison.

An example with one initialisation phase:

```
@model
...
initialisation_phases Equilibrium_initialisation
```

```
@initialisation_phase Equilibrium_initialisation
type derived
```

5.2.2.3. Cinitial Initialisation UNTESTED

The `cinitial` initialisation can only be applied after *derived* or *iterative* initialisation phases. This initialisation can be a method for estimating the non-equilibrium state of population if there is exploitation before data is collected. The estimated *cinitial* factors shift the initial population away from an equilibrium state prior to the start year.

After the first initialisation phase we have an equilibrium age-structure denoted by N_{equil}.

Ciniital specifies an age structure denoted by N_{cinit} (in numbers), but this can be combinations of categories, say both sexes by two areas.

$Multiplier = N_{cinit}/N_{equil}^{combined}$

where $N_{equil}^{combined}$ is summed over the same combined categories as *Cinitial*. Then

$N_{init} = N_{equil} * Multiplier$

N_{init} is the numbers-at-age by category for the start of the model run.

It would be helpful to include an observation of age composition data for the first year of the model in order to estimate the non-equilibrium population state.

An example with two initialisation phases:

```
@model
...
initialisation_phases Iterative Cinitial

@initialisation_phase Iterative
type iterative
years 10
lambda 0.0001
convergence_years 10 20

@initialisation_phase Cinitial
type cinitial
categories spawn.male+nonspawn.male spawn.female+nonspawn.female
table n
spawn.male+nonspawn.male     5e7 5e7 7e6 6e6 5e6 4e6 3e6 2e6 1e6 1e6 1e1 1e1 1e1 1e1
spawn.female+nonspawn.female 5e7 5e7 7e6 6e6 5e6 4e6 3e6 2e6 1e6 1e6 1e1 1e1 1e1 1e1
end_table
```

The Cinitial factors can also be estimated with the syntax

```
@estimate cinit_male
parameter initialisation_phase[Cinitial].spawn.male+nonspawn.male
same initialisation_phase[Cinitial].spawn.female+nonspawn.female
lower_bound  2e2  2e2  2e2  2e2  2e2  2e2  2e2  2e2  2e2  2e2  2e0  2e0  2e0  2e0
upper_bound  2e9  2e9  2e9  2e9  2e9  2e9  2e9  2e9  2e9  2e9  2e9  2e9  2e9  2e9
type uniform
```

5.2.2.4. State_category_by_age UNTESTED

The `state_category_by_age` initialisation uses a user-defined table as the initial partition numbers-at-age for the beginning of the start year. Models can be initialised by specifying the numbers-at-age for each category.

An example with one initialisation phase:

```
@model
...
initialisation_phases Fixed

@initialisation_phase Fixed
type state_category_by_age
categories male female
min_age 3
max_age 10
table n
male   1000 900 800 700 600 500 400 700
female 1000 900 800 700 600 500 400 700
end_table
```

When initialising models with this type, undefined behaviour may result if the model applies processes that require derived quantities to be calculated in the initialisation phase. (e.g., SSB so that recruitment can be calculated for the start year). In the latter case, you would have to use a subsequent initialisation phase *iterator* that has natural mortality set to zero (i.e., *insert_processes* subcommand to introduce zero natural mortality and *exclude_processes* to exclude the mortality process that defines natural mortality) for as many year needed to set up the SSBs.

5.3. Population processes

Population processes are processes that change the model state. These processes produce changes in the partition by adding or removing individuals, or by moving individuals between ages and/or categories.

Current population processes available include:

- recruitment,
- ageing,
- growth,
- maturation,
- mortality events (e.g., natural and fishing), and
- category transition processes, i.e., processes that move individuals between categories while preserving their overall age structure.

There are two types of processes: (1) processes that occur across multiple time steps in the annual cycle, e.g., `mortality_constant_rate` and `mortality_instantaneous`; and (2) processes that occur only within the time step in which they are specified.

5.3.1. Recruitment

Recruitment processes add new individuals to the partition. Recruitment depends on virgin biomass or alternatively recruitment in the virgin state and so these parameters are located in this process (as *b0* and *r0*). The other factors needed are SSB if there is a stock-recruitment relationship and the CV for the prior on YCS (the mean is mandated to be 1 over some specified year range). Thus, a SSB label may have to be included (pointing to a derived quantity).

In the recruitment processes, a number of individuals are added to a single age class (subcommand *age*) within the partition, with the number determined by the type of stock-recruitment process specified. If recruits are added to more than one category, then the proportion of recruits to be added to each category is specified by the `proportions` subcommand. For example, if recruiting to categories labelled `male` and `female`, then the proportions may be set to 0.5 and 0.5, so that half of the recruits are added to the male category and the other half to the female category.

Recruitment can differ between a spawning event or the creation of a cohort/year class. One view for fisheries is that recruitment usually refers to individuals "recruiting" to a fishery. This definition is used because there is usually not a lot of information on younger age classes between the time of spawning and being vulnerable to a survey or fishery for data collection. However, here, recruitment is to a fixed age class for one or more categories.

The offset between spawning and recruitment is parameterised either by the recruitment subcommand `age`, or `min_age`, which is the default value for the `age` subcommand in the recruitment process. The CASAL2 parameter `age` is the same as the CASAL parameter `y_enter`.[IS IT????] Notice that the minimum age is usually different from zero and so there is usually a one or more year's delay between spawning and recruitment into the partition. There is also a complication from when spawning occurs in the annual cycle and when recruitment occurs.

CASAL2 has two recruitment processes, constant recruitment and the Beverton-Holt stock-recruitment relationship (Beverton and Holt, 1957). The number of individuals following recruitment in year y is

$$N_{y,a,j} \leftarrow N_{y,a-1,j} + p_j(R_y) \tag{5.2}$$

where $N_{y,a,j}$ is the numbers in year y and category j at age a, p_j is the proportion added to category j, and R_y is the total number of recruits in year y.

5.3.1.1. Constant recruitment

In the constant recruitment process the total number of recruits added in each year y in age a is R_y, with $R_y = R_0$ for all years

$$R_{y,j} = p_j(R_0) \tag{5.3}$$

Constant recruitment is equivalent to a Beverton-Holt recruitment process with steepness (h) set to 1.

For example, to specify a constant recruitment process where individuals are added to the male and female immature categories at $age = 1$ in equal proportion (`proportions` = 0.5), and the number to add is $R_0 = 5 \times 10^5$, the syntax is

```
@process Recruitment
type constant_recruitment
categories male.immature female.immature
proportions 0.5 0.5
r0 500000
age 1
```

5.3.1.2. Beverton-Holt recruitment

In the Beverton-Holt recruitment process the total number of recruits added each year is R_y. R_y is the product of the average recruitment R_0, the annual year class strength multiplier YCS, and the stock-recruit relationship $SR(SSB_y)$

$$R_{y,a,j} = p_j(R_0 \times YCS_{ycs_year} \times SR(SSB_{ycs_year})) \tag{5.4}$$

where

$$ycs_year = y - \texttt{ssb_offset} \tag{5.5}$$

and a is age, p_j is the proportion of recruits to enter category j, and `ssb_offset` is the number of years lag between spawning and recruitment.

Recruitment refers to recruitment into the population and may differ from the spawning event. See below on more information about `ssb_offset`. In general this parameter should not be specified by the user.

$SR(SSB_y)$ is the Beverton-Holt stock-recruit relationship parametrised by the steepness h, and based on Mace and Doonan (1988) parametrisation

$$SR(SSB_y) = \frac{SSB_y}{B_0} / \left(1 - \frac{5h-1}{4h}\left(1 - \frac{SSB_y}{B_0}\right)\right) \tag{5.6}$$

The Beverton-Holt recruitment process requires a value for B_0 and SSB_y to calculate the number of recruits. A derived quantity (see Section 5.4) must be defined that provides the annual SSB_y for the recruitment process. B_0 is then defined as the value of the SSB at the end of one of the initialisation phases, which is defined by the parameter `b0_initialisation_phase`.

During initialisation the YCS multipliers are assumed to be equal to 1, and recruitment that happens in the initialisation phases that occur before and during the phase when B_0 is determined are assumed to have steepness $h = 1$ (i.e., in those initialisation phases, recruitment is equal to R_0).

Recruitment in the initialisation phases after the phase where B_0 was determined are calculated using the Beverton-Holt stock-recruit relationship. R_0 and B_0 have a direct relationship when there are no density-dependent processes in the annual cycle. Models can thus be initialised using B_0 or R_0.

An example of the specification of a Beverton-Holt recruitment process, where individuals are added to the category "immature" at $age = 1$, and the number added is $R_0 = 5 \times 10^5$; `SSB_derived_quantity` is a derived quantity that specifies the total spawning stock biomass that contributed to the year class, with B_0 the value of the derived quantity at the end of the initialisation phase labelled `phase1`; and YCS are standardised to have mean one in the period 1995 to 2004, and recruits enter into the model two years following spawning

```
@process Recruitment
type recruitment_beverton_holt
categories immature
proportions 1.0
r0 500000
b0_initialisation_phase phase1
steepness 0.75
age 1
ssb SSB_derived_quantity
```

The property `ssb_offset` should not be manually specified; CASAL2 determines `ssb_offset` by the order of ageing, recruitment, spawning, and the recruitment parameter `age`

- if the annual time step order is recruitment, ageing, spawning, then `ssb_offset` should equal `age` + 1, or
- if the annual time step order is spawning, ageing, recruitment, then `ssb_offset` should equal `age` - 1, or
- `ssb_offset` = `age`

There may be scenarios where the user will input these values, e.g., if there are multiple ageing processes in the annual cycle. CASAL2 does not have functionality to accommodate this situation, so in this case `ssb_offset` would be manually defined.

There are two variants of this process and they refer to how the stock recruitment residuals or YCS_{ycs_year} are parametrised. This parametrisation can either be in natural space as year class strength (YCS) multipliers, or in log space as recruitment deviations. Due to the difference in terminology, these variants are implemented in two separate processes, `type recruitment_beverton_holt` and `type recruitment_beverton_holt_with_deviations`, respectively.

YCS (YCS_y)

The YCS parameter (`ycs_years`) is defined in Equation (5.5). The parameter `ycs_values` is referenced by the `ycs_years` parameter and is important to note when defining `@estimate`, `@project`, and `@time_varying` blocks for the parameter `ycs_values`. An example is at the end of the section.

A common practice when estimating YCS is to standardise using the Haist parametrisation, which was described by V. Haist. CASAL2 will standardise YCS only if subccommand `standardise_ycs_years` is defined. The model parameter `ycs_values` is a vector $\mathbf{Y}$, covering the years from `start_year` - `ssb_offset` to `final_year` - `ssb_offset`, as defined by the parameter `ycs_years`. The resulting year class strengths are calculated by $YCS_i = Y_i/\bar{\mathbf{Y}}$, where the mean is calculated over the user-specified years `standardise_ycs_years`.

$$YCS_i = \begin{cases} Y_i/mean_{y \in S}(Y_y) & : y \in S \\ Y_i & : y \notin S \end{cases}$$

where S is the set of years from `standardise_ycs_years`. One effect of this parametrisation is that R_0 is then defined as the mean estimated recruitment over the set of years S, because the mean YCS multiplier over these years will always be one.

Typically `standardise_ycs_years` is defined to span the years over which YCS is reasonably well estimated. For years that are not well estimated, Y_y can be set to 1 for some or all years $y \in S$ (which is equivalent to forcing $R_y = R_0$ x $SR(SSB_y)$) by setting the lower and upper bounds of these Y values to 1. An exception to this might occur for the most recent YCS values, which the user may estimate but not include in the definition of R_0 (because the estimates may be based on too few data). One or more years may be excluded from the range of years for the averaging process of the Haist parametrisation.

The advantage of the Haist parametrisation is that a large penalty is not necessary to force the mean of the YCS parameter to be 1, although a small penalty should still be used to stop the mean of **Y** from drifting. These adjustments may improve MCMC performance. Projected YCS values are not affected by this feature. A disadvantage with this parametrisation in a Bayesian analysis is that the prior applies to Y, not YCS.

In the example givrn above, YCS are standardised to have mean one in the period 1995 to 2004, and recruits enter into the model two years following spawning

```
@process Recruiment
type recruiment_beverton_holt
...            #subcommand above
standardise_ycs_years 1995:2004
ycs_years   1994   1995   1996   1997   1998   1999   2000   2001   2002   2003   2004   2005   2006
ycs_values  0.65   0.87    1.6   1.13 1.0235  0.385  2.653   1.35      1      1      1      1      1
```

Recruitment deviations, ε_y (*type recruitment_beverton_holt_with_deviations*) UNTESTED

Recruitment deviations represent the stock-recruitment relationship residuals in log space, with the link between YCS_y and ε_y

$$YCS_y = exp(\varepsilon_y - b_y\sigma_R^2/2) \tag{5.7}$$

where $\varepsilon_y \sim N(0, \sigma_R^2)$, σ_R^2 is the variance of the stock-recruitment residuals, and b_y is a bias correction defined by Methot Jr and Taylor (2011)

$$b_y = \left\{ \begin{array}{ll} 0, & \text{for } y \leq y_1^b \\ b_{max}(1 - \frac{y - y_1^b}{y_2^b - y_1^b}), & \text{for } y_1^b < y < y_2^b \\ b_{max}, & \text{for } y_2^b \leq y \leq y_3^b \\ b_{max}(1 - \frac{y_3^b - y}{y_4^b - y_3^b}), & \text{for } y_3^b < y < y_4^b \\ 0, & \text{for } y_4^b \leq y \end{array} \right\} \tag{5.8}$$

The ε_y values are normally distributed in log space and thus lognormal when back-transformed to the resulting stock-recruitment relationship YCS_y. Recent work has found that this transformation does not technically lead to the *a priori* assumption that the resulting YCS_y are lognormal. See Appendix **??** for more discussion.

The ramp function described above for the bias correction has the additional subcommands controlling the ramp

- $y_1^b =$ `last_year_with_no_bias`

- $y_2^b =$ first_year_with_bias
- $y_3^b =$ last_year_with_bias
- $y_4^b =$ first_recent_year_with_no_bias
- $b_{max} =$ b_max

```
@process Recruitment
type recruitment_beverton_holt_with_deviations
categories immature
proportions 1.0
r0 500000
last_year_with_no_bias 1940
first_year_with_bias 1950
last_year_with_bias 2016
first_recent_year_with_no_bias 2018
b_max 0.85
b0_initialisation_phase phase1
steepness 0.75
age 1
ssb SSB_derived_quantity
deviation_years   1994   1995   1996   1997   1998   1999   2000   2001   2002   2003   2004   2005   2
deviation_values  0 -0.2 0.4 0 0 0 0 0 0 0 0 0 0
```

Recruitment when modelling two stocks (or species)

To specify a Beverton-Holt recruitment for each stock, the information required is:

1. *YCS*, starting from year (start_year - ssb_offset) and extending up to year (final_year - ssb_offset)
2. the value of age (which is y_enter in CASAL)
3. the steepness parameter h
4. in a multi category model, the proportion of recruits for each category
5. a label for the derived quantity

When an @initialisation_phase (Section 5.2.2) type = derived is specified and the recruitment is defined by b0, then all categories must be specified in the @recruitment block. Usually in a recruitment processes only the categories that receive recruits need to be defined. For example, a population has a spawning area that is different from the area where recruits enter the population. An area-specific model could then be specified which contains spawning categories and recruiting categories. The recruiting categories would be specified in the subcommand categories, as these would be the categories receiving recruits.

If @initialisation_phase, type=derived is used, then all categories that are a part of that recruitment process need to be specified as well

```
@process Recruitment_stock1
type recruitment_beverton_holt
categories stock1.immature.male stock1.immature.female stock1.spawn.male stock1.spawn.female
proportions 0.5 0.5 0.0 0.0
r0 500000
ssb SSB1
....
```

```
@process Recruitment_stock2
type recruitment_beverton_holt
categories stock2.immature.male stock2.immature.female stock2.spawn.male stock2.spawn.female
proportions 0.5 0.5 0.0 0.0
r0 200000
ssb SSB2
....
```

The `proportions = 0.0` for "spawn.male" and "spawn.female" are needed due to the way the derived initialisation phase works. The derived initialisation finds a solution for when `r0` = 1.0 based on an infinite geometric series for the plus group, and scales the initial partition by `r0`. Thus, if all categories are not specified, then those that are missed would not be initialised to true values and this could lead to inaccurate model outputs. This set-up extends to multiple-stock fisheries model configurations as well, where all of the categories that make up the stock need to be listed.

5.3.2. Ageing

The ageing process "ages" individuals, i.e., this process moves all individuals in the named categories j from one age class a to age class $a+1$, or accumulates them if the last age class is a plus group.

The ageing process is defined as,

$$\text{element}(a+1, j) \leftarrow \text{element}(a, j) \tag{5.9}$$

except in the case of the plus group (if defined),

$$\text{element}(a_{\max}, j) \leftarrow \text{element}(a_{\max}, j) + \text{element}(a_{\max-1}, j). \tag{5.10}$$

For example, to apply ageing to the categories `immature` and `mature`, the syntax is

```
@process Ageing
type ageing
categories immature mature
```

Note: the ageing process is *NOT* applied by CASAL2 by default. As with other processes, CASAL2 will not apply a process unless it is defined and specified as a process within the annual cycle. Hence, it is possible to specify a model where a category is not aged. CASAL2 *will NOT check or otherwise warn if there is a category defined where ageing is not applied.***You have been warned.**

5.3.3. Mortality

There are 8 types of mortality processes available in CASAL2:

- constant rate,
- event,
- biomass-event,
- instantaneous,

- instantaneous retained (discards),
- Hollings,
- initialisation, and
- a density-dependent relationship based on prey suitability.

These processes remove individuals from the partition, either as a rate, as a total number (abundance), as a biomass of individuals or, as a combination of these. CASAL2 does not (yet) implement the Baranov catch equation. However, instantaneous mortality is considered an approximation to the Baranov catch equation.

To apply both natural and biomass-event mortality, the mortality type `mortality_instantaneous` can be specified. Or, you can use `mortality_instantaneous_retained`, where discards are allowed. Mortality blocks are special because they allow both natural mortality and fishing mortality at the same time. Note that all mortality processes occur within the mortality block of a time step. See Section 5.3.3.1 for more information and definitions on mortality blocks.

5.3.3.1. Timing evaluation interval; timing the point when observations are fitted or derived quantities are evaluated

[TODO: review; put into observations?]

Observations (see Section 7) and derived quantities (see Section 5.4) need a concept called a *timing evaluation interval* so that the "time" within a year can be specified for their fit or evaluation. This interval is intimately tied into mortality processes.

There can be one or more mortality processes specified within a time-step, but these must be grouped sequentially, i.e., there cannot be a non-mortality process between any two mortality processes within any one time step. The sequence of mortality processes is called a *timing evaluation interval*. If no mortality processes occurs in a time step, then the *timing evaluation interval* is defined to occur at the end of the time step, i.e., it is a virtual, unspecified, process. Thus each time step has one *timing evaluation interval*.

CASAL2 will output an error if more than one *timing evaluation interval* occurs in a single time step.

The "time" for an observation or derived quantity is based on the proportion of mortality that has occurred within the *timing evaluation interval*. The starting and ending partition are saved so that a partition can be estimated by interpolation between the start and end partitions.

For example, the point of calculation can be set to a point when 75 % of the deaths from natural mortality plus catch has occurred The partition at this point is based on interpolating between the start and end of the interval as the partition is known at those points. Two methods are available: `weighted_sum` and `weighted_product`, and are defined as

- `weighted_sum`: after proportion p through the mortality block, the partition elements are given by $n_{p,j} = (1-p)n_j + p'_j$
- `weighted_product`: after proportion p through the mortality block, the partition elements are given by $n_{p,j} = n_j^{1-p} n_j'^{p}$

where $n_{p,j}$ is the derived quantity at proportion p of the mortality block for category j, n_j is the quantity at the beginning of the mortality block, and n'_j is the quantity at the end of the mortality block.

In the case of a virtual *timing evaluation interval*, the partition at the end of the time-step is used.

[TODO: REDO FIGURE TO REFLECT TEI rather than mortality blocks; have two mortality processes in step 2]

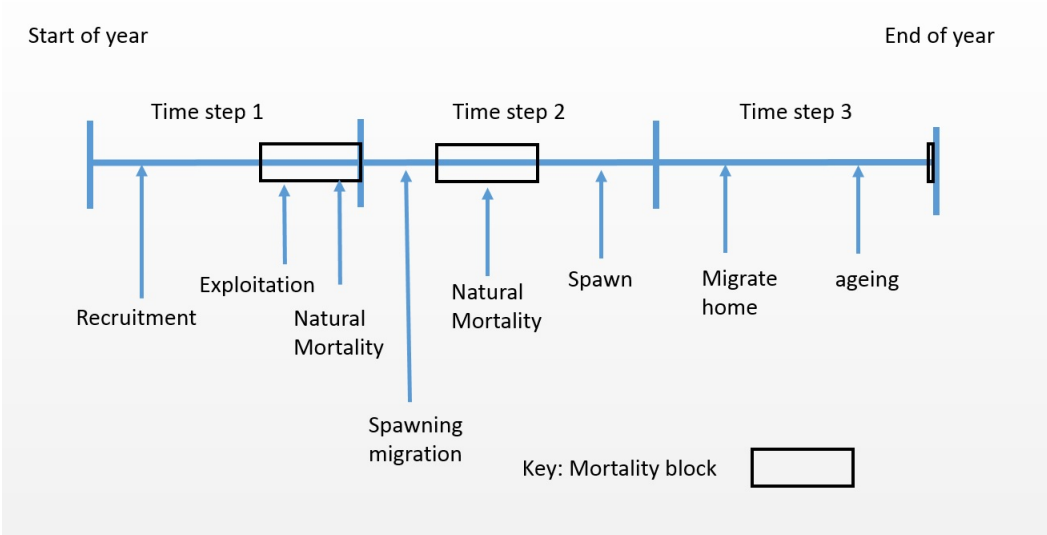

Figure 5.2: A example sequence for an annual cycle.

TODO: GO OVER M and specifing M-by-age HERE FOR ALL M-BASED PROCESSES

TODO: MAX U rate specification + Penalities

5.3.3.2. Constant mortality rate UNTESTED

To specify a constant annual mortality rate (e.g. $M = 0.2$) for categories "male" and "female"

```
@process NaturalMortality  #label is NaturalMortality
type          mortality_constant_rate
categories    male female
relative_m_by_age One One     #effectively age related mortality
m             0.2 0.2
```

The total number of individuals removed from a category

$$D_{j,t} = \sum_a N_{a,j,t}[1 - \exp(-S_{a,j}M_{a,j}p_t)] \tag{5.11}$$

where $D_{j,t}$ is the total number of deaths in category j in time step t, $N_{a,j,t}$ is the number of individuals in category j of age a in time step t, $S_{a,j}$ is the selectivity value for age a in category j, $M_{a,j}$ is the mortality rate for category j for age a, and p_t is the proportion of the mortality rate to apply in time step t.

The mortality rate process requires the specification of the mortality-by-age curve which is specified using a selectivity. To apply the same mortality rate over all age classes in a category, use a selectivity defined as $S_{a,j} = 1.0$ for all ages a in category j

```
@selectivity One
type constant
c 1
```

Age-specific mortality rates can also be applied. For example, the hypothesis that mortality is higher for younger and older individuals and lowest when individuals are at their optimal fitness could be defined by using a double exponential selectivity (see Section 5.10)

```
@selectivity age_specific_M
type double_exponential
x0 7.06524
x1 1
x2 17
y0 0.182154
y1 1.43768
y2 1.57169
alpha 1.0

@process      NaturalMortalityByAge
type          mortality_constant_rate
categories    male female
relative_m_by_age age_specific_M age_specific_M
m             1.0 1.0
```

INSERT FIG OF M-by-age

In this definition `m` is set to 1.0 and the rate is described through the selectivity. Otherwise, $M_{age} = S_{age} * m$. This concept can be constructed similarly for other mortality methods such as `instantaneous_mortality`.

5.3.3.3. Event and biomass-event mortality UNTESTED?

WHEN NOT DOING M and FISHING AT THE SAME TIME

The event mortality and biomass-event mortality processes are applied in a similar manner, except that they remove a specified abundance (number of individuals) or biomass, respectively. These mortality processes can be used to define mortality events where the numbers of removals are known, e.g., fishing, rather than applying mortality as a rate.

In these cases, the abundance or biomass removed is also constrained by a maximum exploitation rate. CASAL2 removes as many individuals or as much biomass as possible, while not exceeding the maximum exploitation rate.

Event mortality processes require a penalty to avoid estimating parameter values that will not allow the defined number of individuals to be removed. The model penalises those parameter estimates that result in an too low a number of individuals in the defined categories (after applying selectivities) to allow for removals at the maximum exploitation rate, with a similar penalty for biomass. See Section 6.8 for more information on how to specify penalties.

The event mortality applied to user-defined categories i, with the numbers removed at age j determined by a selectivity-at-age S_j:

First, calculate the vulnerable abundance for each category j in $1 \ldots J$ for ages $a = 1 \ldots A$ that are subject to event mortality

$$V_{a,j} = S_{a,j} N_{a,j} \tag{5.12}$$

and define the total vulnerable abundance V_{total} as

$$V_{total} = \sum_j \sum_a V_{a,j} \tag{5.13}$$

The exploitation rate to apply is

$$U = \begin{cases} C/V_{total}, & \text{if } C/V_{total} \leq U_{max} \\ U_{max}, & \text{otherwise} \end{cases} \tag{5.14}$$

The number removed $R_{a,j}$ from each age a in category j is,

$$R_{a,j} = UV_{a,j} \tag{5.15}$$

For example, to specify an **abundance-based** fishing mortality process with catches given for a set of specific years over categories "immature" and "mature", with selectivity "FishingSel", and assuming a maximum possible exploitation rate of 0.7, the syntax is

```
@process     Fishing
type          event_mortality
categories    immature mature
years         2000 2001 2002 2003
U_max         0.70
selectivities FishingSel FishingSel
penalty       event_mortality_penalty
```

and specified similarly for a **biomass-based** fishing mortality process

```
@process      Fishing
type          mortality_event_biomass
categories    immature mature
years         2000 2001 2002 2003
U_max         0.70
selectivities FishingSel FishingSel
penalty      event_mortality_penalty
```

5.3.3.4. Instantaneous mortality

The instantaneous mortality process combines both natural mortality and event biomass mortality into a single process. This allows the simultaneous application of both natural mortality and anthropogenic mortality to occur across multiple time steps. This process applies half the natural mortality in each time step, then the mortalities from all the concurrent removals instantaneously, then the remaining half of the natural mortality. In fisheries models this is the most commonly used mortality process [TODO: reference an approx cf Baranov equ].

This process allows for multiple removal events, e.g., a fisheries model with multiple fisheries and/or fleets. A removal method can occur in one time step only, although multiple removals can be defined to cover events during the year.

The equations for instantaneous mortality:

TODO redo equations, as notation is not consistent with that above, e.g., $S_{a,j}$

- An exploitation rate (actually a proportion) is calculated for each fishery, as the catch divided by the selected-and-retained biomass,

$$U_f = \frac{C_f}{\sum_a \bar{w}_a S_{f,a} n_a exp(-0.5tM_a)}$$

- The mortality pressure associated with method f is defined as the maximum proportion of fish taken from any element of the partition in the area affected by the method f

$$U_{f,obs} = max_a(\sum_k S_{k,a} U_k)$$

  where the maximum is over all partition elements affected by fishery f, and the summation is over all methods k which affect the jth partition element in the same time step as fishery f.

  In most cases the mortality pressure will be equal to the exploitation rate (i.e., $U_{f,obs} = U_f$), but can be different if: (a) there is another removal method operating in the same time step as removal method f and affecting some of the same partition elements, and/or (b) the selectivity $S_{f,a}$ does not have a maximum value of 1.

  There is a maximum mortality pressure limit of $U_{f,max}$ for each method of removal f. So, no more than proportion $U_{f,max}$ can be taken from any element of the partition affected by removal method f in that time step. Clearly, $0 \leq U_{max} \leq 1$. It is an error if two removal methods, which affect the same partition elements in the same time step, do not have the same U_max.

  For each f, if $U_{f,obs} > U_{f,max}$, then U_f is multiplied by $U_{f,max}/U_{f,obs}$ and the mortality pressures are recalculated. In this case the catch actually taken from the population in the model will differ from the specified catch, C_f.

- The partition is updated using

$$n'_a = n_a exp(-tM_a)\left[1 - \sum_f S_{f,a} U_f\right]$$

For example, to apply natural mortality of 0.20 across three time steps on both male and female categories, with two methods of removals (fisheries `FishingWest` and `FishingEast`) and their respective catches (kg) known for years 1975:1977 (the catches are given in the `catches` table and information on selectivities, penalties, and maximum exploitation rates are given in the `method` table), the syntax is

TODO where is `time_step_ratio` and its usage defined?

```
@process instant_mort
type mortality_instantaneous
m 0.20
time_step_ratio 0.42 0.25 0.33
relative_m_by_age One
categories male female
units kgs

table catches
year FishingWest FishingEast
1975 80000 111000
1976 152000 336000
1977 74000 1214000
end table
```

```
table method
method       category  selectivity u_max   time_step penalty
FishingWest   stock     westFSel    0.7     step1     CatchPenalty
FishingEast   stock     eastFSel    0.7     step1     CatchPenalty
end_table
```

and for referencing catch parameters for use in projecting, time-varying, and estimating, the syntax is

```
parameter process[mortality_instantaneous].method_"method_label"{2018}
```

where "`method_label`" is the label from the `catch` or `method` table and continuing the example,

```
parameter process[instant_mort].method_FishingWest{2018}
```

To calculate weight by empirical weight-at-age matrices as described in Section 5.7, the method table would include an additional column to reference weight-at-age objects:

```
@age_weight jan_weight_at_age
type data
table data
year 1  2  3  4
1980 3.4 5.6 7.23  8.123
end_table

table method
method       category  selectivity  u_max  time_step  penalty       age_weight
FishingWest  stock     westFSel     0.7    step1      CatchPenalty  jan_weight_at_age
FishingEast  stock     eastFSel     0.7    step1      CatchPenalty  jan_weight_at_age
end_table
```

5.3.3.5. Instantaneous mortality with retained catch and discards

The instantaneous mortality retained process builds on the instantaneous mortality process (5.3.3.4) which has simultaneous applications of fishing and natural mortality, but with all catch-at-sea being landed, i.e., no discarding. The process `mortality_instantaneous_retained` allows for retained catch, discards, and also a mortality to be applied to discards, i.e., some are allowed to survive. The method for taking catch from the partition and the constraints used are the same as in `mortality_instantaneous`.

This process was implemented to address issues with the pot fishery for blue cod which has a minimum legal size and so some catch is discarded at sea and some of these discards are expected to survive (based on some experimental work). There are length data taken at sea, so the total catch selectivity can be estimated, and length and age data taken from the landed catch (retained), so the retention selectivity can also be estimated.

In this mortality process, discard mortality is specified by defining a selectivity to represent mortality by age or length (e.g., constant or asymptotic descending logistic). This discard selectivity is not be estimated since there is no observation class associated with it. If discard mortality is not provided, it is assumed that all discards die. Landed catch, and both the retained and total catch selectivites must be specified.

Extending the example shown in instantaneous mortality process (5.3.3.4) to use retained weight instead of catch, the commands are:

```
@process FishingRetainedCatch
type mortality_instantaneous_retained
# natural mortality
m 0.20
# the ratio of natural mortality in each of the three time steps
time_step_ratio 0.42 0.25 0.33
relative_m_by_age One
#for natural mortality by age
categories male female
units kgs

table catches
# two fisheries, West and East
year FishingWest FishingEast
# the catches are now landed catch
1975 80000 111000
1976 152000 336000
1977 74000 1214000
end table

table method
# all discards die
method      category selectivity retained_selectivity u_max time_step penalty
FishingWest stock    westFSel    westRetainedSel      0.7   step1     CatchPenalty
FishingEast stock    eastFSel    eastRetainedSel      0.7   step1     CatchPenalty
end_table
```

If discard mortality is less than 1.0, use:

```
table method
# 50% discard mortality
method      category selectivity retained_selectivity discard_mortality u_max time_step penalty
FishingWest stock    westFSel    westRetainedSel      DisMort           0.7   step1     CatchPenalt
FishingEast stock    eastFSel    eastRetainedSel      DisMort           0.7   step1     CatchPenalt
end_table

@selectivity DisMort
Type constant
# 50% mortality of discards
c 0.5
```

See the instantaneous mortality process (5.3.3.4) for referencing catch parameters and calculating weight using empirical weight-at-age matrices.

The report outputs total catch, actual landed catch, and discards, without and with discard mortality:

```
@report Mortality
type process
process Instantaneous_Mortality_Retained
```

TODO redo notation, as it is not consistent with that above, e.g., $S_{a,j}$ and R_y

In the following, fisheries are indexed by f, and a indexes both age and category combinations.

The total catch is found by applying a selectivity, $S_{f,a}$, in the same way as in the instantaneous mortality process. Retention, $R_{f,a}$, is defined by specifying a selectivity, which can be a function of length or age. The retained catch is the product of these two values, $R_{f,a} * S_{f,a}$. If sex is in the partition, then there are potentially two retention curves, one for each sex.

In general, there is a retention curve for each category in the partition. This property does not apply to surveys. Discard mortality is also specified as a selectivity, $D_{f,a}$. The fraction of dead fish from fishing activity is $S_{f,a} * [R_{f,a} + (1.0 - R_{f,a}) * D_{f,a}]$. If $D_{f,a}$ is 1.0, then all selected fish are dead, and if it is 0.0, then only the retained fish are dead.

The equations for the `mortality_instantaneous_retained` process:

- Total catch (catch-on-board), C_f, is calculated by (retained catch) * VF / VR, where VF is vulnerable retained biomass, j indexes categories and t is the proportion of M in the time step, and VF is the full vulnerable biomass, $VF = \sum_{a,j} \overline{w}_a S_{a,j} n_{a,j} \exp(-0.5tM_{a,j})$.
- An exploitation rate (actually a proportion) is calculated for each fishery, as the total catch (retained + discards) divided by the selected biomass (VF above) using selectivity $S_{f,a}$,

$$U_f = \frac{C_f}{\sum_a \bar{w}_a S_{f,a} n_a \exp(-0.5tM_a)}$$

- The mortality pressure associated with method f is defined as the maximum proportion of fish taken from any element of the partition in the area affected by the method f,

$$U_{f,obs} = max_a(\sum_k S_{k,a} U_k)$$

  where the maximum is over all partition elements affected by fishery f, and the summation is over all methods k which affect the jth partition element in the same time step as fishery f.

  In most cases the mortality pressure will be equal to the exploitation rate (i.e., $U_{f,obs} = U_f$), but can be different if: (a) there is another removal method operating in the same time step as removal method f and affecting some of the same partition elements, and/or (b) the selectivity $S_{f,a}$ does not have a maximum value of 1.

  There is a maximum mortality pressure limit of $U_{f,max}$ for each method of removal f. So, no more than proportion $U_{f,max}$ can be taken from any element of the partition affected by removal method f in that time step. Clearly, $0 \leq U_{max} \leq 1$. It is an error if two removal methods, which affect the same partition elements in the same time step, do not have the same U_max.

  For each f, if $U_{f,obs} > U_{f,max}$, then U_f is multiplied by $U_{f,max}/U_{f,obs}$ and the mortality pressures are recalculated. In this case the catch actually taken from the population in the model will differ from the specified catch, C_f.
- Discard numbers-at-age (including their share of natural mortality) is $S_{a,j}(1 - R_{a,j}) n_{a,j} \exp(-0.5tM_{a,j})$, and those that die at the end of the time step (updating the partition) are $D_{a,j} S_{a,j}(1 - R_{a,j}) n_{a,j} \exp(-tM_{a,j})$, where $D_{f,a}$ is the fraction that die on return to the sea.
- The partition is updated by removing landed catch, natural mortality, and discard mortality

$$n'_a = n_a \exp(-tM_a) \left[1 - \sum_f S_{f,a} U_f (R_{f,a} + D_{f,a}(1 - R_{f,a}))\right]$$

5.3.3.6. Holling mortality rate UNTESTED

The density-dependent Hollings mortality process applies the Holling Type II or Type III functions (Holling, 1959), and is generalised by the Michaelis-Menten equation (Michaelis and Menten, 1913).

This mortality process removes a number or biomass from a set of categories according to the total (selected) abundance (or biomass) and some "predator" abundance (or biomass), and is constrained by a maximum exploitation rate.

The mortality applied to user-defined categories k, with the numbers removed at age l, determined by a selectivity-at-age S_l is applied as follows:

First, calculate the total predator abundance (or biomass) over all predator categories k in $1 \ldots K$ and ages $l = 1 \ldots L$ that are applying the mortality

$$P_{k,l} = S_l^{predator} N_{k,l}^{predator} \tag{5.16}$$

And define the total predator abundance (or biomass) P_{total} as

$$P_{total} = \sum_K \sum_L P_{k,l} \tag{5.17}$$

Then calculate the total vulnerable abundance (or biomass) over all prey categories k in $1 \ldots K$ and ages $l = 1 \ldots L$ that are subject to the mortality

$$V_{k,l} = S_l^{prey} N_{k,l}^{prey} \tag{5.18}$$

Then define the total vulnerable abundance (or biomass) V_{total} as

$$V_{total} = \sum_K \sum_L V_{k,l} \tag{5.19}$$

The number to remove is then determined by

$$R_{total} = P_{total} \frac{a V_{total}^{x-1}}{b + V_{total}^{x-1}} \tag{5.20}$$

where $x = 2$ for the Holling type II function, $x = 3$ for the Holling type III function, or a different value of $x \geq 1$ for the generalised Michaelis-Menten function; $a > 0$ and $b > 0$ are the Holling function parameters.

The exploitation rate to apply is

$$U = \begin{cases} R_{total}/V_{total}, & \text{if } R_{total}/V_{total} \leq U_{max} \\ U_{max}, & \text{otherwise} \end{cases} \tag{5.21}$$

And the number removed R from each age l in category k is

$$R_{k,l} = UV_{k,l} \tag{5.22}$$

The density-dependent Holling mortality process is applied either as a function of biomass or abundance, depending on the value of the `is_abundance` switch.

For example, a biomass Holling type II mortality process on prey `prey` by predator `predator` has the syntax

```
@process HollingMortality
type Holling_mortality_rate
is_abundance F
a 0.08
b 10000
x 2
categories prey
selectivities One
predator_categories predator
predator_selectivities One
u_max 0.8
```

5.3.3.7. Initialisation-event mortality UNTESTED

Initialisation event mortality is a process that can occur only in the initialisation phase. It applies abundance or biomass mortality events specifically in initialisation phases. This option can be useful if the population is not in equilibrium before model start.

This process applies a single catch value for all iterations within the initialisation phase, and mortality will not be applied outside of the initialisation phase. This process should not be embedded in the annual cycle.

This process should be used in conjunction with the `insert_processes` command in the `@initialisation_phase` block.

Example syntax where the `initialisation_mortality_event` has been specified in the initialisation phase `Predation_state` but not in the annual cycle:

```
initialisation_phases Equilibrium_state Predation_state
time_steps Oct_Nov Dec_Mar

@initialisation_phase Equilibrium_state
type derived

@initialisation_phase Predation_state
type iterative
insert_processes Oct_Nov()=predation_Initialisation

@process predation_Initialisation
type initialisation_mortality_event
categories male.HOKI female.HOKI
catch 90000
selectivities Hakesl Hakesl
```

```
time_step Oct_Nov
processes Mg1 Instantaneous_Mortality

@time_step Dec_Mar
processes Recruitment Instantaneous_Mortality
```

5.3.3.8. Prey-suitability mortality UNTESTED

The density-dependent prey-suitability mortality process applies predation mortality from a predator group to its prey groups simultaneously. It removes an abundance (or biomass) from each prey group according to the total (selected) abundance (or biomass) of each prey group, the total (selected) abundance (or biomass) of the other prey groups, some "predator" abundance (or biomass), and the preference (electivity) of the predator for each prey group, constrained by a maximum exploitation rate. The predator-prey suitability functions were based on the multispecies Virtual Population Analysis (MSVPA) functions (Jurado-Molina et al., 2005).

The mortality applied to the user-defined prey group g of category k, with the numbers removed at age l determined by a selectivity-at-age S_l is applied as follows:

First, calculate the total predator abundance (or biomass) over all predator categories k in $1 \ldots K$ and ages $l = 1 \ldots L$ that are applying the mortality

$$P_{k,l} = S_l^{predator} N_{k,l}^{predator} \tag{5.23}$$

And define the total predator abundance (or biomass) P_{total} as

$$P_{total} = \sum_K \sum_L P_{k,l} \tag{5.24}$$

Then, given the total vulnerable abundance (or biomass) of prey group g over all categories k in $1 \ldots K$ and ages $l = 1 \ldots L$ that are subject to the mortality

$$V_{g,k,l} = S_l^{prey} N_{k,l}^{prey} \tag{5.25}$$

And define the total vulnerable abundance (or biomass) of each prey group V_{total}^g as

$$V_{total}^g = \sum_K \sum_L V_{g,k,l} \tag{5.26}$$

And the total availability A_{total}^g for each prey group as

$$A_{total}^g = \frac{V_{total}^g}{\sum_G V_{total}^g} \tag{5.27}$$

The vulnerable abundance (or biomass) and availability every prey group g in $1 \ldots G$ is calculated simultaneously. Then the abundance (or biomass) to remove from each prey group g is a function

of its electivity E_g, the availability of all other prey groups i in $1 \ldots G$, the electivity of the predator for each prey group E_i, and the total consumption rate of the predator CR and its abundance (or biomass) P_{total}

$$R^g_{total} = P_{total} CR \frac{A^g_{total} E_g}{\sum_G A^i_{total} E_i} \tag{5.28}$$

The exploitation rate to apply to each prey group g is then

$$U_g = \begin{cases} R^g_{total}/V^g_{total}, & \text{if } R^g_{total}/V^g_{total} \leq U_{max} \\ U_{max}, & \text{otherwise} \end{cases} \tag{5.29}$$

And the number removed R^g in each prey group g from each age l in category k is

$$R_{g,k,l} = U_g V_{g,k,l} \tag{5.30}$$

Prey suitability choice occurs only between the prey groups specified by the process. The total predator consumption rate represents the consumption of the predator on those prey groups alone. The electivities must sum to 1. Further, the consumption rate can be modified by a layer to be cell specific.

The density-dependent prey-suitability process is applied as either a biomass or an abundance depending on the value of the `is_abundance` switch.

Individual categories can be aggregated into prey groups using the "+" symbol. To indicate that two (or more) categories are to be aggregated, separate them with a "+" symbol.

For example, to specify two prey groups of two species made up of the males and females in each prey group

```
prey_categories maleSpeciesA + femaleSpeciesA maleSpeciesB + femaleSpeciesB
```

This syntax indicates that there are two prey groups, `maleSpeciesA + femaleSpeciesA` and `maleSpeciesB + femaleSpeciesB`, with each group having its own electivity.

For example, a biomass prey-suitability mortality process with an overall consumption rate of 0.8 of prey `species A` and `species B` (modelled as males and females) by the predator `predatorSpecies` with electivities between `species A` and `species B` of 0.18 and 0.82 has syntax

```
@process PreySuitabilityMortality
type prey-suitability_predation
is_abundance F
consumption_rate 0.8
categories maleSpeciesA + femaleSpeciesA maleSpeciesB + femaleSpeciesB
electivities 0.18 0.82
selectivities One One One One
predator_categories predatorSpecies
predator_selectivities One
u_max 0.8
```

5.3.4. Transition By Category

The transition by category process moves individuals between categories. This process is used to specify transitions such as maturation (individuals move from an immature to mature state) and migration (individuals move from one area to another).

5.3.4.1. Annual transition by category

A special process type is the annual transition by category process, which allows a transition to occur in a specific subset of years only, where each year can have a different rate.

In both cases, there is a one-to-one relationship between the "from" category and the "to" category, i.e., for every source category there is one target category only

$$N_{a,j} = N_{a,i} \times P_i \times S_{a,i} \tag{5.31}$$

where $N_{a,j}$ is the number of individuals that have moved to category j from category i in age a, $N_{a,i}$ is the number of individuals in category i, P_i is the proportion parameter for category i, and $S_{a,i}$ is the selectivity at age a for category i.

To merge categories repeat the "to" category multiple times.

For example, to specify a simple spawning migration of mature males from a western area to an eastern (spawning) area, the syntax is

```
@process Spawning_migration
type transition_category
from West.males
to East.males
selectivities MatureSel
proportions 1
```

where `MatureSel` is a selectivity that describes the proportion of age or length classes that are mature and thus move to the eastern area.

5.3.5. Tag Release events UNTESTED

Tagging processes can be age- or length-based processes, whereby numbers of individuals are moved from an untagged category to a tagged category defined in the `@categories` block. Tag release processes can also account for initial tag-induced mortality on individuals.

Age-based tag release events move a known number of individuals tagged for each age to a tagged category, along with applying additional mortality. Individuals are removed from the non-tagged categories and added to tagged categories. Often the ages of tagged individuals are not known, so length-based tag release events are more commonly used.

Length-based tag release processes are more complicated, as the age-length matrix is calculated and the exploitation for each length bin to then move the correct numbers-at-age based on the known lengths of release. CASAL2 also allows for initial tag loss.

The tag-release process:

For each length bin l of the input vector of numbers-at-length $\tilde{N}_l$

$$N_{l,j} = \sum_{a=1} N_{a,l,j} * S_a$$

where $N_{a,l,j}$ is the numbers at age a and length l for category j, and S_a is the selectivity at age a.

Calculate the total numbers-at-length T_l across all source categories at length l, taking into account the selectivities

$$T_l = \sum_{j=1} N_{l,j}$$

Calculate the transition rate for length bin u_l

$$u_l = \tilde{N}_l / T_l$$

Check that the threshold u_{max} is not exceeded, which is analogous to the u_{max} in a mortality processes

$$u_l = \begin{cases} u_{max}, & \text{if } u_l > u_{max} \textbf{ flag a penalty} \\ u_l, & \text{otherwise} \end{cases}$$

Calculate the numbers-at-age in this category that will be moved by multiplying across the age-length matrix and storing the result by age, for each age accumulated across all length bins. Then move the necessary

$$N_{a,j} += N_{a,l} * u_l$$

The syntax for an example of tag release by length process

```
@process 2005Tags_shelf
type tag_by_length
years 2005
from male.untagged+female.untagged
to male.2005  female.2005
selectivities ShelfselMale ShelfselFemale
penalty tagging_penalty
initial_mortality 0.1
table proportions
year 30 40 50 60 70 80 90 100 110 120 130 140 150 160 170 180 190 200 210 220
2005  0 0 0.0580 0.1546 0.3380 0.1981 0.1643 0.0531 0.0242 0.0097 0 0 0 0 0 0 0 0 0 0
end_table
n 207
U_max 0.999
```

This process moves 207 individuals from a combination of male.untagged and female.untagged categories, based on the combination of growth rates and selectivity, into tagged male and tagged female categories.

5.3.6. Tag Loss UNTESTED

Tag Loss is the process which accounts for tags being lost from a tagged individual due to, for example, tag failure or tags getting knocked off. This process is applied as an instantaneous migration rate that can happen over multiple time steps in the annual cycle. This method assumes that when tags are lost the individuals are transferred from the `from` category to the `to` category.

The tag loss rate is applied depending on whether the individuals were tagged with a single tag only (`tag_number_per_animal = 1`), double tagged (`tag_number_per_animal = 2`), or tagged with n tags (`tag_number_per_animal = n`).

The syntax for the tag loss is

```
@process Tag_loss
type tag_loss
categories tagged_fish
tag_loss_rate 0.02
time_step_ratio 0.25 0.75
selectivities One
tag_loss_type single
year 1985
```

5.4. Derived quantities

Some processes require a population value derived from the population state as an argument. These values are `derived quantities`. Derived quantities are values calculated in a specified time step in every year, and thus have a single value for each year of the model. The time withing the time-step is and the end unless otherwise specified (using *proportion_mortality*-like subcommand.

Derived quantities can be calculated as either abundance or biomass. Abundance-derived quantities are the sum over the specified categories (after applying a selectivity). Biomass-derived quantities are calculated similarly.

Derived quantities are also calculated during the initialisation phases. Therefore, the time step during each initialisation phase must be specified. If the initialisation time steps are not specified, the derived quantity will be calculated during the initialisation phases. [TODO: review]

A common use of an derived quantities is as input into a stock-recruit relationship which requires an equilibrium biomass (B_0) and annual spawning stock biomass values (SSB_y) to calculate recruitment into the first age class. SSB_y is an derived quantity based on the mature biomass, usually at spawning time.

Derived quantities can be associated with a *time evaluation interval*; see section 5.3.3.1 for more detail on mortality blocks. In this case, the point of calculation can be set to any point within the mortality block, e.g., when 75 % of the deaths from natural mortality plus catch has occurred, which is based on interpolating between the start and end of the block as the partition is known at those points. Two methods are available: `weighted_sum` and `weighted_product`, and are defined as

- `weighted_sum`: after proportion p through the mortality block, the partition elements are given by $n_{p,j} = (1-p)n_j + p'_j$
- `weighted_product`: after proportion p through the mortality block, the partition elements are given by $n_{p,j} = n_j^{1-p} n_j'^{p}$

where $n_{p,j}$ is the derived quantity at proportion p of the mortality block for category j, n_j is the

quantity at the beginning of the mortality block, and n'_j is the quantity at the end of the mortality block.

For example, to define a biomass-derived quantity spawning stock biomass, *SSB*, calculated at the end of the first time step (labelled `step_one`), over all "mature" male and female categories and halfway through the mortality block using the `weighted_sum` method, the syntax is

```
@derived_quantity SSB
type          biomass
time_step     step_one
categories    mature.male mature.female
selectivities One
time_step_proportion        0.5
time_step_proportion_method weighted_sum
```

5.5. Age-length relationship

The age-length relationship defines the functional form of the length-at-age (and the weight-at-length; see Section 5.6) of individuals at age/category within the model.

There are four length-age relationship options. The first is the naive "no relationship", where each individual has length 1 regardless of age. The others are: von Bertalanffy relationship, the Schnute relationship, and "data" (mean length-at-age for each model year).

The length-at-age relationship is used to calculate the length frequency given age, and with the length-weight relationship, the weight-at-age of individuals within an age/category. When defining length-at-age, the length-weight relationship must also be defined (see Section 5.6).

Changes in length-at-age during the year, i.e., growth between birthdays, are represented by incrementing age as specified by the `time_step_proportions` parameter.

5.5.0.1. The "no relationship" relationship, `none`

The length of each individual is 1 for all ages, and the `none` length-weight relationship must also be used.

5.5.0.2. The von Bertalanffy relationship, `von_bertalanffy`

$$\bar{s}(age) = L_\infty \left(1 - \exp\left(-k\left(age - t_0\right)\right)\right) \tag{5.32}$$

5.5.0.3. The Schnute relationship, `schnute`

$$\bar{s}(age) = \begin{cases} \left[y_1^b + (y_2^b - y_1^b)\dfrac{1-\exp(-a(age-\tau_1))}{1-\exp(-a(\tau_2-\tau_1))}\right]^{1/b}, & \text{if } a \neq 0 \text{ and } b \neq 0 \\ y_1 \exp\left[\ln(y_2/y_1)\dfrac{1-\exp(-a(age-\tau_1))}{1-\exp(-a(\tau_2-\tau_1))}\right], & \text{if } a \neq 0 \text{ and } b = 0 \\ \left[y_1^b + (y_2^b - y_1^b)\dfrac{age-\tau_1}{\tau_2-\tau_1}\right]^{1/b}, & \text{if } a = 0 \text{ and } b \neq 0 \\ y_1 \exp\left[\ln(y_2/y_1)\dfrac{age-\tau_1}{\tau_2-\tau_1}\right], & \text{if } a = 0 \text{ and } b = 0 \end{cases} \tag{5.33}$$

The von Bertalanffy relationship has parameters L_∞, k, and t_0. The Schnute relationship (Schnute, 1981) has parameters y_1 and y_2, which are the mean lengths at reference ages τ_1 and τ_2, and a and b; when $b = 1$, this relationship reduces to the von Bertalanffy relationship with $k = a$.

5.5.0.4. Data: `data`

There is an option to input empirical length at age by year, which is an alternative to using an age-length growth model such as the von Bertalanffy and Schnute model. CASAL2 will interpolate values for missing years across time steps. The calculations of length-at-age throughout the model years occur in the same time step.

```
@age_length   male_AL
type          data
time_step_proportions 0.0 0.0   #use age at start of time-step
length_weight wgt_male          # needed to convert numbers-at-age into catch
distribution  normal            # distribution of lengths around the mean length
cv_first      0.1               # cv of the distribution at the first age
cv_last       0.1               # cv of the distribution at the maximum age
time_step_measurements_were_made step2
internal_gaps mean
external_gaps mean
table data                      # first line has column labels for year and age
year   2     3    4     5     6    7     8      9    10    11    12    13    14    15
1980 30.13 34.9 38.43 40.61 42.45 43.02 43.94 43.63 43.36 43.7 43.84 43.51 43.45 43.45
1981 30.33 34.78 38.03 40.15 42.22 42.89 44.21 44.07 43.99 44.32 44.64 44.28 44.04 43.93
end_table
```

When the values for *cv_last* and *cv_first* are different, the cv used for intermediate ages is, by default, interpolate by that age's mean length. There is a legacy switch for testing the conversion of models from CASAL into CASAL2, i.e., use the subcommand, *by_length false* which allows the interpolation to be by age, the default setting for CASAL. CASAL also fixes to by-age interpretation when doing calculating the mean weight (see 5.6.0.2).

5.6. Length-weight relationship

There are two length-weight relationships options. The first is the naive "no relationship" relationship, where the weight of an individual is always 1, regardless of length. The second

relationship is the "basic" relationship, which is the standard length-weight relationship, $W = aL^b$.

5.6.0.1. The "no relationship" relationship, `none`

$$\text{mean weight} = 1 \tag{5.34}$$

5.6.0.2. The standard length-weight relationship, `basic`

The mean weight $\hat{w}_a$ of an individual of age a is

$$\hat{w}_a = a\hat{l}_a^b \tag{5.35}$$

where $\hat{l}_a$ is the mean length at age a. If a distribution of length-at-age is specified, then the mean weight is calculated over the distribution of lengths

$$\hat{w}_a = (a\hat{l}_a^b)(1+cv^2)^{\frac{b(b-1)}{2}} \tag{5.36}$$

where the cv is the coefficient of variation (CV) of the length-at-age relationship. This adjustment is exact for lognormal distributions, and an approximation for normal distributions if the CV is not large (Bull et al., 2012).

For comparing CASAL with CASAL2 results, there is a small difference between the two programs. CASAL adjusted the CVs `by_length` only when CVs are used in distribution calculations (length-based selectivities, length-based processes, and [length-based?] observations), and is not done in the above correction.

Note: the scale of a can be specified incorrectly. If the catch is in tonnes and the growth curve is in centimetres, then a should convert a length in centimetres to a weight in tonnes. There are reports available that can be used to help check that the units specified are plausible (see Section 8).

```
@length_weight length_weight
type basic
units tonnes
a 0.00000123
b 3.132
```

5.7. Age-weight relationship UNTESTED

Empirical weight-at-age data can be input. This option is different from the method above as it uses empirical data for weight-at-age, rather than calculating it with the growth functions (age -> length -> weight). This bypasses the growth machinery which is expected to be present and so using weight-at-age data needs to be declared in blocks that use this method, i.e., biomass observation blocks, fishery mortality blocks, and biomass derived quantities e.g., SSB). The subcommand to use is "`age_weight_label` ageWeight.block.label" within the block, but in mortality fisheries blocks, `age_weight_label` is a column in the *table method* part with the corresponding *ageWeight.block.label* in the body of the table. More than one *@age_weight* blocks can be used, and both weight-at-age data and the usual growth version can be used in the same model (but clearly not in the same block).

This option specifies the weight-at-age values for categories at a point in time.

An example

```
@age_weight age_weight NOT AUTO-PICKED UP IN MANUAL?????
type Data
units tonnes
table data   #year then ages; 1st row is the column labels
year 1 2 3 4 5 6 7 8 9 10
1986 0.134 0.686 1.639 2.719 3.649 4.901 6.329 6.591 7.238 7.491
1987 0.132 0.724 1.534 2.829 4.092 4.853 5.705 6.143 7.179 8.089
1988 0.122 0.641 1.533 2.641 3.796 5.054 5.652 6.356 6.95 8.857
1989 0.137 0.722 1.606 2.416 3.629 5.027 5.561 6.35 6.933 7.217
1990 0.138 0.773 1.645 2.74 3.711 4.506 5.684 6.929 7.424 7.479
end_table
```

If weight is defined by the empirical weight-at-age data, then the age-length block in the `@categories` block can be omitted.

```
@categories
format stock
names Stock
```

5.8. Weightless model UNTESTED

To model abundance (i.e., to model the population in numbers and not convert to biomass), the `@length_weight` argument is turned off by specifying the keyword `none` in the `@age_length` block

```
@age_length age_size
type schnute
...
length_weight none
```

In this case any "biomass" generated by CASAL2 will actually be abundance, and care should be taken with interpretation of the output.

5.9. Maturity, in models without maturing in the partition

When maturity is not a factor in the partition, processes may still depend on maturity. You must then make the assumption that the proportion of mature fish in each element of the partition remains constant over time. A selectivity is used to define the proportion of mature fish in each age class and these can depend on categories (a length selectivity could be used).

This selectivity is specified in the block that requires mature fish, the most common one is for SSB as a derived quantity.

5.10. Selectivities

Selectivity is a term used in CASAL2 to mean an ogive that is a function of age. They can be used to specify the selection curve for a fishery or observation (Section 6) or to modify the effects of processes on each age class, e.g., migration rates by age (Section 5). The curves can use length rather

than age, in which case they operate on the length distribution for each age (use the subcommand "*by_length true*", *false* is the default). Do not expect too much from length selectivities because in the next time-step or year, the length distribution for each age reverts to being as defined in the *age_length* blocks, e.g., normal, rather than being partially truncated because, e.g., larger fish in an age class have been preferentially caught.

There are a number of different parametric forms as options, including logistic and double normal curves. Selectivities are defined in command block `@selectivity <label>`, where the unique label of the selectivity is used by observations and processes to identify which selectivity to apply.

For example, a logistic selectivity can be defined by:

```
@selectivity trawlSel    #label for the trawl fishery selectivity
type      logistic       # type of curve
a50       4.4            # age at 50% selection
ato95     1.5            # interval (yr) from a50 to the age at 95% selection
                         #  age at 95% selectivity is 5.9 yr; at 5%, 2.9 yr

#at_length true          #if used, then a50 & ato95 refer to length
```

For some selectivities, the function values for some choices of parameters can result in numeric overflow or underflow errors (i.e., the number calculated from parameter values is either too large or too small to be well represented). CASAL2 implements range checks on some parameters to test for these errors before calculating function values.

For example, the logistic selectivity is implemented such that if $(a_{50} - x)/a_{to95} > 5$ then the value of the selectivity at $x = 0$, i.e., for $a_{50} = 5$, $a_{to95} = 0.1$, then the value of the selectivity at $x = 1$, without range checking would be 7.1×10^{-52}. With range checking, that value is 0 (as $(a_{50} - x)/a_{to95} = 40 > 5$).

The selectivity options are:

- Constant
- Knife-edge
- All values
- All values bounded
- Increasing
- Logistic
- Inverse logistic (descending logistic?)
- Logistic producing
- Double normal
- Double exponential

See Figure 5.3 for plots of example selectivities (p. 62).

5.10.1. constant

$$f(x) = C \tag{5.37}$$

The constant selectivity has the estimable parameter C.

Input fragment:

```
type constant
c    0.5
```

5.10.2. `knife_edge` UNTESTED

$$f(x) = \begin{cases} 0, & \text{if } x < E \\ \alpha, & \text{if } x \geq E \end{cases} \tag{5.38}$$

The knife-edge ogive has the estimable parameter E and a non-estimable scaling parameter α, where the default value of $\alpha = 1$.

Input fragment:

```
type  knife_edge
e     8
alpha 0.5
```

5.10.3. `all_values`

$$f(x) = V_x \tag{5.39}$$

The all-values selectivity has estimable parameters V_{low}, $V_{low+1} \ldots V_{high}$. The selectivity value for each age class must be set.

5.10.4. `all_values_bounded`

$$f(x) = \begin{cases} 0, & \text{if } x < L \\ V_x, & \text{if } L \leq x \leq H \\ V_H, & \text{if } x > H \end{cases} \tag{5.40}$$

The all-values-bounded selectivity has non-estimable parameters L and H. The estimable parameters are V_L, $V_{L+1} \ldots V_H$. Selectivity values for each age class from $L \ldots H$ must be set.

Selectivities `all_values` and `all_values_bounded` can be included in additional priors using the syntax

```
@selectivity maturity
type all_values
v 0.001 0.1 0.2 0.3 0.4 0.3 0.2 0.1

## encourage ages 3-8 to be smooth.
@additional_prior smooth_maturity
type vector_smooth
parameter selectivity[maturity].values{3:8}
```

5.10.5. `increasing` UNTESTED?

$$f(x) = \begin{cases} 0, & \text{if } x < L \\ f(x-1) + \pi_x(\alpha - f(x-1)), & \text{if } L \leq x \leq H \\ f(\alpha), & \text{if } x \geq H \end{cases} \tag{5.41}$$

The increasing ogive has non-estimable parameters L and H. The estimable parameters are π_L, π_{L+1} $\ldots \pi_H$; if these are estimated, they should always be constrained to be between 0 and 1. α is a scaling parameter, with default value of $\alpha = 1$. The increasing ogive is similar to the all-values-bounded ogive, and is constrained to be non-decreasing.

Input fragment:

```
type  increasing
l     3
h     7
v     0.2 0.3 0.4 0.5 0.6
```

5.10.6. `logistic`

$$f(x) = \alpha/[1 + 19^{(a_{50}-x)/a_{to95}}] \tag{5.42}$$

The logistic selectivity has estimable parameters a_{50} and a_{to95}. α is a scaling parameter (input files, *alpha*, with default value of $\alpha = 1$. The logistic selectivity takes values 0.5α at $x = a_{50}$ and 0.95α at $x = a_{50} + a_{to95}$.

5.10.7. `inverse_logistic` UNTESTED??

$$f(x) = \alpha - \alpha/[1 + 19^{(a_{50}-x)/a_{to95}}] \tag{5.43}$$

The inverse logistic selectivity has estimable parameters a_{50} and a_{to95}. α is a scaling parameter (*alpha*, with default value of $\alpha = 1$. The logistic selectivity takes values 0.5α at $x = a_{50}$ and 0.95α at $x = a_{50} - a_{to95}$.

Input fragment:

```
type  inverse_logistic
a50   4
ato95 1
alpha 0.5   #default is 1.0
```

5.10.8. `logistic_producing`

$$f(x) = \begin{cases} 0, & \text{if } x < L \\ \lambda(L), & \text{if } x = L \\ (\lambda(x) - \lambda(x-1)) \,/\, (1 - \lambda(x-1)), & \text{if } L < x < H \\ 1, & \text{if } x \geq H \end{cases} \tag{5.44}$$

The logistic-producing selectivity has non-estimable parameters L and H. The estimable parameters are a_{50} and a_{to95}. α is a scaling parameter, with default value of $\alpha = 1$.

For category transitions, $f(x)$ represents the proportion moving, not the proportion that have moved. This selectivity was designed for use in an age-based model to model maturity. In such a model, a logistic-producing maturation selectivity will, in the absence of other influences, make the proportions mature follow a logistic curve with parameters a_{50} and a_{to95}.

Input fragment:

```
type  logistic_producing
l      2
h      8
a50    4
ato95  1
#alpha 1.0
```

5.10.9. double_normal

$$f(x) = \begin{cases} \alpha 2^{-[(x-\mu)/\sigma_L]^2}, & \text{if } x \leq \mu \\ \alpha 2^{-[(x-\mu)/\sigma_R]^2}, & \text{if } x \geq \mu \end{cases} \tag{5.45}$$

The double-normal selectivity has estimable parameters a_1, s_L, and s_R. α is a scaling parameter, with default value of $\alpha = 1$. It has values α at $x = a_1$, and 0.5α at $x = a_1 - s_L$ and $x = a_1 + s_R$.

Input fragment:

```
type  double_normal
mu       6     #age at switch over from left to right normal curves
               #  = mean for both normal curves
sigma_1  1     # standard deviation for left normal
sigma_2  10    # standard deviation for right normal
#alpha 1.0
```

5.10.10. double_exponential

$$f(x) = \begin{cases} \alpha y_0 (y_1/y_0)^{(x-x_0)/(x_1-x_0)}, & \text{if } x \leq x_0 \\ \alpha y_0 (y_2/y_0)^{(x-x_0)/(x_2-x_0)}, & \text{if } x > x_0 \end{cases} \tag{5.46}$$

The double-exponential selectivity has non-estimable parameters x_1 and x_2. The estimable parameters are x_0, y_0, y_1, and y_2. α is a scaling parameter, with default value of $\alpha = 1$. This selectivity curve can be "U-shaped". Bounds for x_0 must be such that $x_1 < x_0 < x_2$. With $\alpha = 1$, the selectivity passes through the points (x_1, y), (x_0, y_0), and (x_2, y_2). If both y_1 and y_2 are greater than y_0 the selectivity is "U-shaped" with minimum at (x_0, y_0).

Input fragment:

```
type  double_exponential
x0    15   # age at middle point
y0    0.1  # selection at x0; here a minimum --> U shape
x1    1    # left point
y1    0.5  #  selection at x1
x2    30   # right point
y2    0.8  # selection at x2
#alpha 1.0
```

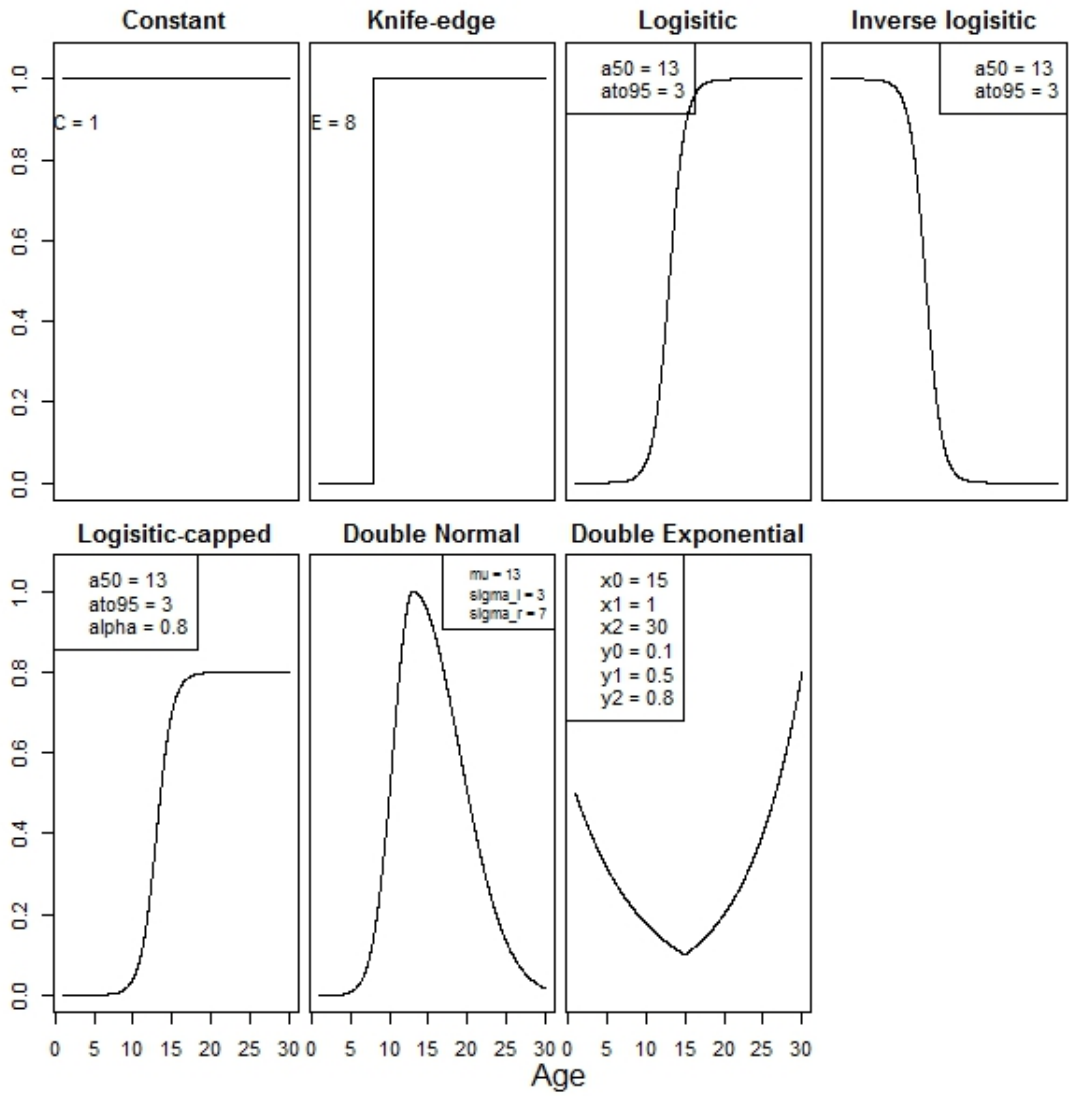

Figure 5.3: Examples of the selectivities

5.11. Time-varying Parameters UNTESTED

Any parameter can be varied annually for blocks of years or in specific years within the model run (not in initialisation phases??). For years that are not specified, the parameter will default to the input, or if in an iterative state such as estimation mode, the value being trialled at that iteration.

Method types for a time-varying parameter are:

- `constant`,
- `random_walk`,
- `exogenous`,
- `linear`,
- `annual_shift`, and
- `random_draw`.

This option allows for a parameter to be fixed in a year, or be the result of a deterministic or stochastic equation. **Note:** the stochastic time-varying functionality was added for simulation purposes. **It has not been tested in an estimation context.**

To implement hierarchical models, the prior parameter values need to be estimated using hyperpriors. To implement a hierarchical model using the time-varying functionality, use MCMC estimation as a way to calculate the integral which is required to obtain unbiased estimates. In an MCMC context, a Gibbs sampler is assumed. That is, every draw is from a conditional distribution and so every draw is a candidate value. [TODO: review]

When allowing removals with annually varying catchabilities, selectivities, and/or other model components, simulated observations more closely model real data and associated conclusions become more useful. Implementing time-varying parameters also allows for mean or location parameters of selectivities to change between years based on an explanatory variable. An example of this is in the New Zealand Hoki fishery where the μ and a_{50} parameters are allowed to shift depending on when the fishing season occurs. Descriptive analysis showed that when fishing was earlier relative to other years smaller fish were caught and vice versa. This can be shown in the CASAL2/Examples/2stock directory, implemented at line: `382` in the `population.csl2` file. [Craig to edit]

5.11.1. `constant`

This option allows a parameter to have an different value during specified years to the rest of the model run,and this value can be estimated. To allow survey catchability to be different in the year block 1975 to 1988 from teh rest of the series we write:

```
@time_varying q_time_var
type          constant
parameter     catchability[survey_q].q
years         1975:1988
values        0.001      #same for all years
```

To estimate catchability for 1975 and 1976, use the following:

```
@estimate q_time_var
type uniform   #prior
parameter time_varying[q_time_var].values{1975:1976}
lower_bound 1e-6 1e-6
upper_bound 2     2
```

To make the catchability be same over the year block we need to estimate it for one year (say 1975) and use the *same* subcommand to make the others take the same value

```
@estimate q_time_var
type uniform
parameter time_varying[q_time_var].values{1975}
same      time_varying[q_time_var].values{1976:1988}
lower_bound 1e-6
upper_bound 2
```

Caution: do not estimate both the actual parameter and its time-varying counterpart, as the time-varying value will overwrite the actual parameter making the actual value unidentifiable. [Craig to edit]

5.11.2. random_walk

A random deviate drawn from a standard normal distribution is added to the previous year's value. This option has an estimable parameter σ_p for each time-varying parameter p. For reproducible modelling when using stochastic functionality, set the random seed (see Section 3.3).

```
@time_varying q_time_var
type          random_walk
parameter     catchability[survey_q].q
distribution  normal
mean          0
sigma         3
```

If the `parameter` specified in the `@time_varying` block is associated with an `@estimate` block, then the parameter is constrained to stay within the lower and upper bounds of the `@estimate` block.

WARNING: if the parameter does not have an associated `@estimate` block then there is no safeguard against the application of a random deviate resulting in parameter values which cause the model to fail, i.e., generates NA or INF values. To avoid this, specify an `@estimate` block even though the parameter is not actually being estimated; see the example syntax below.

A constraint whilst using this functionality is that a parameter cannot be less than 0.0. If it is then CASAL2 sets it equal to 0.01.

```
@estimate survey_q_est
type      uniform
parameter catchability[survey_q].q
lower_bound 1e-6
upper_bound 10
```

This configuration will insure the random walk time-varying process will set the any new candidate values within the lower and upper bound of the `@estimate` block.

Syntax abuse: now overloaded many parameters with just one universal one?

5.11.3. annual_shift

A parameter generated in year y (θ'_y) depends on the value specified by the user (θ_y) along with three coefficients a, b, and c

$$\bar{\theta}_y = \frac{\sum_y^Y \theta_y}{Y} \tag{5.47}$$

$$\theta'_y = a\bar{\theta}_y + b\bar{\theta}_y^2 + c\bar{\theta}_y^3 \tag{5.48}$$

5.11.4. exogenous

Parameters are shifted based on an exogenous variable. An example of this is an exploitation selectivity parameters that may vary between years based on known changes in exploitation

behaviour such as season, start time, and average depth of exploitation.

$$\delta_y = a(E_y - \bar{E}) \tag{5.49}$$

$$\theta'_y = \theta_y + \delta_y \tag{5.50}$$

where δ_y is the shift or deviation in parameter θ_y in year y to generate the new parameter value in year y (θ'_y). a is an estimable shift parameter, E is the exogenous variable, and E_y is the value of this variable in year y. For more information readers can see Francis et al. (2003).

5.12. Equation Parser UNTESTED

CASAL2 has an equation parser, which is currently implemented in Projections (section 5.13.1), Derived quantities (section 5.4), and Reports (section 8).

Examples of syntax for implementing the equation parser are below. For more information on the parser, see `https://github.com/nickgammon/parser/blob/master/parser.cpp`

```
equation process[Recruitment].r0 * 2 #double the recruitment
```

mathematical functions such as `sqrt()`, `log()`, `exp()`, `cos()`, `sin()`, and `tan()` can be used

```
equation sqrt(process[Recruitment].r0)
```

exponents can be used with `pow()`

```
equation pow(2, 3)
```

the absolute value of an equation using `abs()`

```
equation abs(sqrt(process[Recruitment].r0) * 1.33)
```

`if-else` statements can be used

```
equation if(process[Recruitment].r0 > 23, 44, 55)
## if R0 is greater than 23 return 44 else return 55
```

`if-else` statements can also be linked, more complex syntax

```
equation if(process[Recruitment].r0 > 23, 44,
          if(process[Recruitment].r0 > 10, 55, 66))
## if R0 is greater than 23 return 44 else if R0 less than 23 but greater than 10 return 55,
else R0 must be less than 10 return 66
```

Only single values can be referenced, so an equation cannot be applied to a vector, e.g., `process[Recruit].ycs_values{1974:1980}` cannot be referenced. More information on which parameters can be included in an equation parser is available (Section 13). Any subcommand that has a `type estimable` could be referenced with the equation parser.

Note: the equation parser will not catch all user configuration errors, such as checking whether a parameter that exists in the system has been populated when it is required.

For example, the wrong year could be misspecified in the case of removals in year y which is based on the state of the population in year $y-1$

```
parameter process[removals].catch
year 2015
equation derived_quantity[percent_b0].values{2020}
```

This example is a valid equation but it will have nonsensical results, since a value for 2020 is to be calculated using values for 2015. Although the equation parser adds flexibility, it is easy to misspecify equations.

5.13. Specifying projections UNTESTED

Given a set of estimated parameter values from a *-e* or a MCMC run, the model can be projected into the future. Projection years are after the model run years, and are defined in the *@model* command block using the `final_projection_year` subcommand, i.e., projection years are `final_year + 1` through to `final_projection_year`.

Parameter values for the projected years can be specified in a stochastic way or fixed at some value (the default is the estimated value if the parameter is not time-varying) and these are specified in the *@projection* block,

```
@project Future_ycs  #label
type       lognormal_empirical  #which method to use
parameter  process[Recruitment].ycs_values
years      2012:2016
multiplier 1
....            #other parameters
```

The subcommands `years` and `parameter` are common to all projection methods. Subcommand `years` specifies the years to apply the new values to for the parameter in `parameter`. Note that the years can be before the *finial_year*, e.g., it is normal to vary the last few YCS in a projection run because they are usually poorly estimated or they have been set to 1. The argument `multiplier` is a constant which is multiplied with the projected value after it has been generated. The `type` subcommand gives the method to use to generate new parameter values.

CASAL2 allows any estimable parameter to be specified in a `@project` block and then varied from the estimated value in a projection. The available projection types for these parameters include:

- constant
- lognormal
- empirical-lognormal
- empirical re-sampling

- user-defined

CASAL2 has no default projection properties for parameters that are specified by year, e.g., year class strength parameters, time-varying parameters, and as a special case, future catches). For these, projections must have a `@project` command block. For example, CASAL2 will produce errors if run in projection mode without a `@project` block for the `ycs_values` parameter being specified.

Note for the year class parameters: the definition of year applies to the `ycs_years`, not the model years. As defined in Section 5.3.1.2, `ycs_years` are offset between the time of spawning and when individuals are added to the partition.

Future catches are also specified in a *@project* block, one for each fishery (see 5.13.1.6 for examples). Here, a fishery is reference in the *parameter* subcommand with the *method_* fragment to identify it,

process[<block label>].method_<fisheries label>,

For a process called *Fishing* that has three fisheries defined, it would be *process[Fishing].method_pot* to specify the fishery labeled *pot*.

The CASAL2 command to run the model in projection mode is `Casal2 -f 1`. []TODO: review; NOT IN MANUAL OR ELSEWHERE] This functionality allows for the exploration of many scenarios with a single set of parameters. The number of projections should be greater than 1 only if applying a projection type that is stochastic.

The `--tabular` flag should be used when running projections after a Bayesian analysis. This option will output a tabular report (see Section 8.21) which can then be analysed in **R**.

An example of the command line evocation is

casal2 -f 1 -i mcmc.txt –tabular > projection.out.txt

where *mcmc.txt* is output from a MCMC run, one parameter set per row, which will give one projection per row, and *projection.out.txt* will contain one row for each MCMC run in each of the reports specified in (usually) the *Report.csv2* file (quantities as specified in the *report.csl2* file).

For a projection run in CASAL2, the model is initialised and run through the model years from `start_year` to `final_year`. During this run mode CASAL2 stores all parameter values so that projection classes can allow parameters before `final_year` to be projected. The model then is re-run from `start_year` to `projection_final_year`, where any parameter can either be fixed or drawn from a stochastic distribution or process.

5.13.1. Projection methods

This section lists all the projections classes available, their functionality, and an example of the syntax.

5.13.1.1. The constant projection type, `constant`

A parameter can either be fixed during all projection years or specified individually for each projection year. This is a deterministic assumption, where the parameter is assumed to be known without error during projection years.

```
@project Future_ycs
type      constant
parameter process[Recruitment].ycs_values
```

```
years     2012:2016
values    1 2 1 2 0.5  #"values 3" means all years use 3
multiplier 1
```

5.13.1.2. Sampling from a range of years, type `empirical_resampling`

Parameters that have time components associated with them can be re-sampled uniformly with replacement over a range of years and used as values for the projected years. The year range to sample from is between `start_year` and `final_year`:

```
@project   Future_ycs
type       empirical_sampling
parameter  process[Recruitment].ycs_values
years     2012:2016
start_year 1988     # re-sample from estimated values
final_year 2008     # from 1988 to 2008 inclusive
multiplier 1
```

5.13.1.3. Sampling from a lognormal distribution, type `lognormal` UNTESTED

The parameters are drawn from a Gaussian distribution in log space and exponentiated to result in the lognormal distribution

$$X_p = exp(\varepsilon_p - \sigma^2/2) \tag{5.51}$$

where $\varepsilon_p \overset{iid}{\sim} N(\mu, \sigma)$ and X_p is the projected value for parameter X, and μ and σ are the mean and standard deviation on the log scale.

An example of applying this process to draw future year class parameters from a lognormal distribution with mean 1 and standard deviation 0.8

```
@project Future_ycs
type     lognormal
parameter process[Recruitment].ycs_values
years     2012:2016
mean      0     #mean 1 on un-transformed scale
sigma     0.8   # log scale
multiplier 1
```

5.13.1.4. Sampling from a lognormal distribution where the variance is estimated from values over a specified year range, type `lognormal_empirical` UNTESTED

This method applies a lognormal draw as in the `LogNormal` method above and specifies a year range from which the standard deviation of the distribution is calculated. Then equation (5.51) is used to generate future values with a specified μ and empirically calculated σ,

```
@project Future_ycs
type      lognormal_empirical
parameter process[Recruitment].ycs_values
years     2012:2016
```

```
mean      0
start_year 1988   # range of years to take the
final_year 2008   # values for \math{\sigma}
multiplier 1
```

5.13.1.5. Sample from a user-defined function, `user_defined` UNTESTED

This method uses the equation parser to calculate the values to use in the projection. This was set up to define and apply harvest control rules (i.e., apply a management action such as changing the TACC based on the current or previous state).

In fisheries models, this option can be used to calculate the projected catch based on an exploitation rate multiplied by the vulnerable biomass, where the exploitation rate is based on a rule (Figure 5.4).

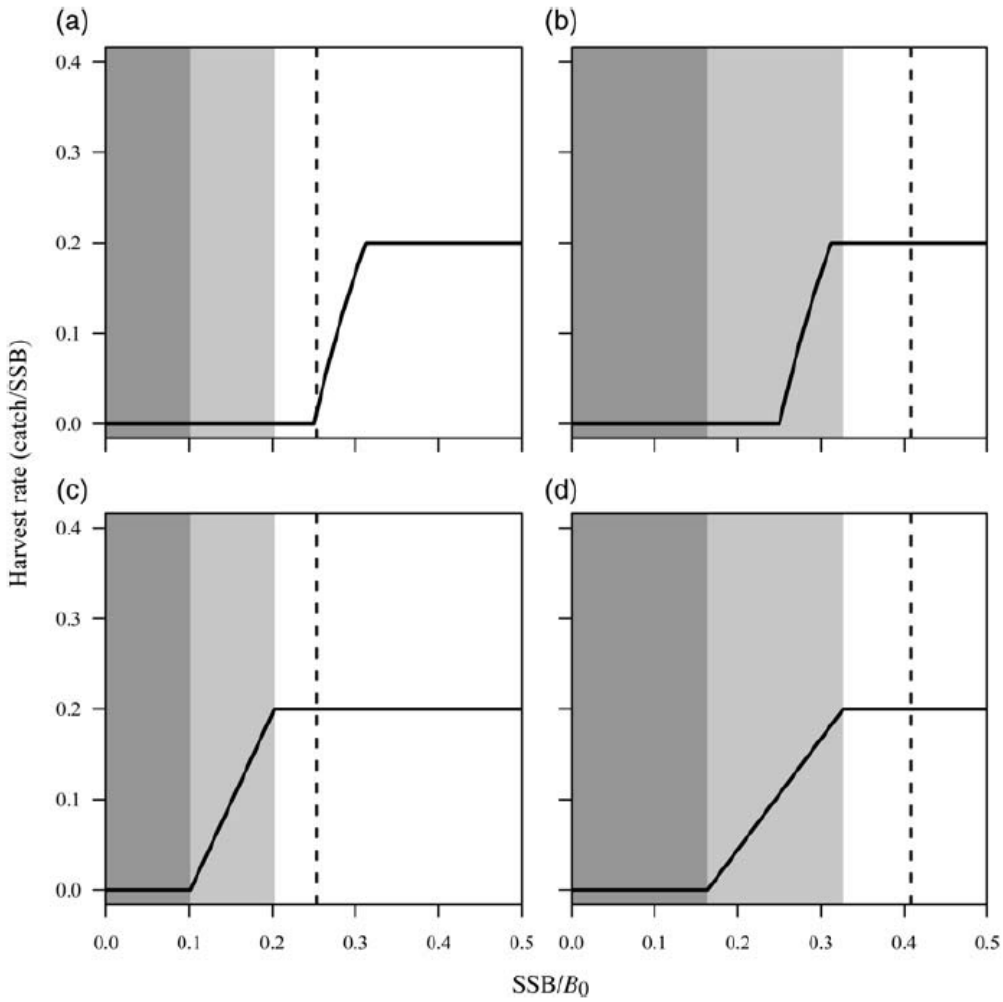

Figure 5.4: Examples of control rules based on current stock status.

```
@project HCR_2015
type       user_defined
parameter process[Instantaneous_Mortality].method_Sub_Ant_F
years 2015
equation if(derived_quantity[SSB].values{2014} / process[Recruitment].b0 <= 0.1, 0.0,
if(derived_quantity[SSB].values{2014} / process[Recruitment].b0 > 0.1 &&
derived_quantity[SSB].values{2014} / process[Recruitment].b0 < 0.2,
derived_quantity[SSB].values{2014} * derived_quantity[SSB].values{2014}
/ process[Recruitment].b0,
derived_quantity[SSB].values{2014} * 0.2))
```

Care should be taken when writing user-defined equations. The above equation is: if $\%B_{2014} \leq 0.1$ then set next year's catch to 0.0, else if $\%B_{2014} > 0.1$ & $\%B_{2014} \leq 0.2$ then set next year's catch equal to $\%B_{2014} \times SSB_{2014}$, else set next year's catch to $0.2SSB_{2014}$.

5.13.1.6. Specifying catch for projections

Catches are unique in that they are known inputs in a table format. For example, to project catches that are in a table

```
# fishing process
@process Fishing
type mortality_instantaneous_retained
m 0.17*6  #0.17    #testing at old values
time_step_ratio 1
relative_m_by_age One*6   #for age based M
categories *
table catches
year FishingLine FishingPot Recreation
1900 0 0 0
1901 13.2 0 22.9
1902 26.4 0 23.5
1903 39.6 0 24
end_table

# projection block
@project future_catch
type      constant
parameter process[Fishing].method_fishingpot
years     2020:2029
values    4000
```

This uses the syntax `block_type[block_label].method_fishinglabel`. **Note:** the fishing label which is defined in the table needs to be lower case form in the `@projection` block. Notice the use of *method_* syntax to identify the right fishery.

6. The estimation section

6.1. Role of the estimation section

The role of the estimation section is to define the tasks carried out by CASAL2:

1. Define the objective function (see Section 6.2)
2. Define the parameters to be estimated (see Section 6.3)
3. Calculate a point estimate, i.e., the maximum posterior density estimate (MPD) (see Section 6.4)
4. Calculate a posterior profile on selected parameters, i.e., for each of a series of values of a parameter, minimise the objective function, allowing the other estimated parameters to vary (see Section 6.5)
5. Generate MCMC samples from the posterior distribution (see Section 6.6)
6. Calculate the approximate covariance matrix of the parameters as the inverse of the minimizer's approximation to the Hessian, and the corresponding correlation matrix (see Section 6.4)

The estimation section defines the objective function, parameters of the model, and the method of estimation (point estimates, Bayesian posteriors, profiles, etc.). The objective function is based on a goodness-of-fit measure of the model to observations, the assumed priors, and the penalties. See the observation section for a description of the observations, likelihoods, priors, and penalties.

6.2. The objective function

In Bayesian estimation, the objective function is a negative log-posterior,

$$Objective(p) = -\sum_i \log\left[L\left(\mathbf{p}|O_i\right)\right] - \log\left[\pi\left(\mathbf{p}\right)\right] \tag{6.1}$$

where π is the joint prior density of the parameters p.

The contribution to the objective function from the likelihood components is described in Section 7.2. In addition to likelihoods, priors (see Section 6.7) and penalties (see Section 6.8) are components of the objective function. Note that if the priors are specified as uniform, then the prior contribution is zero and the optimisation is now a penalised likelihood and not Bayesian.

Penalties can be used to ensure that the estimated parameter values and derived quantities meet certain restrictions. For example, exploitation rate constraints on mortality events (i.e., fisheries) that are not violated (otherwise there is nothing to prevent the model from having abundances so low that the recorded catches could not have been taken); penalties on category transitions (to ensure there are enough individuals to move); penalties such that estimated values are similar or smooth, etc.

Equation 6.1 can be reduced to a penalised likelihood equation if all priors are assumed to be uniform. This is because uniform priors have no contribution to the objective function so Equation 6.1 reduces to the likelihood components plus penalties.

6.3. Specifying the parameters to be estimated

The parameters to be estimated (estimables) are defined using `@estimate` commands (see Section 10).

For example, a `@estimate` command block

```
@estimate male.m
parameter process[NaturalMortality].m{male}
lower_bound 0.1
upper_bound 0.4
type uniform
```

See Section 3.4.5 for information on how to specify the parameter name. At least one parameter is required to be estimated if doing an estimation `-e`, profile `-p`, or MCMC `-m` run. Initial values for the parameters to be estimated are required, and these values are used as the starting values for the minimiser. However, these values may be overwritten if a set of alternative starting values is provided (i.e., using `casal2 -i`, see Section 3.3).

All parameters are estimated within the specified bounds. For each parameter estimated, the lower and upper bounds and the prior (`type`) (Section 6.7) must be specified. The bounds and the prior should be chosen carefully as they affect the values over which the minimisers search. Some minimisers convert the lower and upper bounds into a minimisation space (for example -1,1 space for the numerical differences algorithm). If estimating only some elements of a vector, either define each element of the vector to be estimated (see 3.4.5) or fix the others by setting the the lower and upper bounds to the same value as the initial value.

6.4. Point estimation

Point estimation is invoked with `casal2 -e`, which attempts to find a minimum of the objective function. CASAL2 has multiple minimisation algorithms. There are three automatic differentiation (AD) minimisers: ADOL-C, CppAD, and BetaDiff (the minimiser used in CASAL). There are also three non-automatic differentiation minimisers: numerical differences, differential evolution minimiser (`de_solver`), and the dlib minimiser (see Section 1.9). AD minimisers are recommended for more complex models as they are on average much faster and tend to find a more robust minimum when exploring a complex objective surface.

An important input parameter for most minimisers is the `tolerance` parameter. This is one of the stopping rules that minimisers use to define when they have found a 'solution' (although a solution may be a local minimum and not the global minimum). To evaluate the robustness of a solution, use a set of different starting values, with the `-i parameter_file.txt` option.

Start with the default `tolerance` parameter value of 0.002 and decrease it while developing a model. For a given model, the parameter estimates when minimising with tolerance value 0.0000002 may be quite different from those with the default tolerance value. This is not robust model behaviour and more investigative and diagnostic work may be required to determine what parameters are causing this behaviour. `Note:` this issue will also affect the covariance matrix. When a different tolerance value and/or minimiser is used, a different approximate Hessian matrix which is inverted to solve for the covariance matrix may be produced. The effect that this behaviour has on MCMC samples is not well-determined.

6.4.1. The numerical differences minimiser

The minimiser has three kinds of (non-error) exit status, depending on the minimiser:

- Successful convergence (suggests a local minimum has been found, at least).
- Convergence failure (a local minimum has not been found, although the results may be 'close enough').
- Convergence unclear (the minimiser halted but was unable to determine if convergence occurred. The result may be a local minimum, although this can be checked by restarting the minimiser at the final values of the estimated parameters).

The maximum number of quasi-Newton iterations and objective function evaluations allowed can be specified. If either limit is exceeded, the minimiser exits with a convergence failure. Set the maximum number of evaluations and iterations to values larger than the defaults of 300 and 1000, unless convergence is reached with fewer. An alternative starting point of the minimiser can be specified using `casal2 -i`.

The minimisers are local optimisation algorithms trying to solve a global optimisation problem. What this means is that, even if a 'successful convergence' is reached, the solution may be only a local minimum, and not a global one. To diagnose this problem, start multiple runs from different starting points and comparing the results, or do profiles of one or more key parameters and seeing if any of the profiled estimates finds a better optimum than than the original point estimate.

The approximate covariance matrix of the estimated parameters can be calculated as the inverse of the minimiser's approximation to the Hessian, and the corresponding correlation matrix is also calculated.

Note that

- the Hessian approximation develops over many minimiser steps, so if the minimiser has only run for a small number of iterations the covariance matrix can be a very poor approximation; and
- the inverse Hessian is not a good approximation to the covariance matrix of the estimated parameters, and may not be useful to construct, for example, confidence intervals.

Also note that if an estimated parameter has equal lower and upper bounds, it will have entries of '0' in the covariance matrix and `NaN` or `-1.#IND` (depending on the operating system) in the correlation matrix.

```
@minimiser numerical_diff
type numerical_differences
tolerance 1e-6
iterations 2500
evaluations 4000
```

6.4.2. The differential evolution minimiser

The differential evolution minimiser is a simple population-based, stochastic function minimizer, but is claimed to be quite powerful in solving minimisation problems. It is a method of mathematical optimization of multidimensional functions and belongs to the class of evolution strategy optimizers (TODO reference?).

Initially, the procedure randomly generates and evaluates a number of solution vectors (the population size), each with p parameters. Then, for each generation (iteration), the algorithm creates a candidate solution for each existing solution by random mutation and uniform crossover. The random mutation generates a new solution by multiplying the difference between two randomly selected solution vectors by some scale factor, then adding the result to a third vector. Then an element-wise crossover takes place with probability P_{cr}, to generate a potential candidate solution. If this is better than the initial solution vector, it replaces it, otherwise the original solution is retained. The algorithm terminates after either a predefined number of generations (`max_generations`) or when the maximum difference between the scaled individual parameters from the candidate solutions from all populations is less than some predefined amount `tolerance`.

The differential evolution minimiser can be good at finding global minimums in surfaces that may have local minima. However, the speed of the minimiser, and the ability to find a good minima depend on the number of initial 'populations'. Some authors recommend that the number of populations be set at about $10 * p$, where p is the number of free parameters. However, depending on the model, this value can be set to a lower value and still find a robust solution.

There is no proof of convergence for the differential evolution solver, but several papers have found it to be an efficient method of solving multidimensional problems. Some results suggest that it can often find a better minima and may be faster or longer (depending on the actual model specification) at finding a solution when compared with the numerical differences minimiser. Comparisons with automatic differentiation minimisers or other more sophisticated algorithms have not been made.

```
@minimiser DE_solver
type de_solver
tolerance 1e-6
iterations 2500
evaluations 4000
```

6.4.3. The BetaDiff minimiser

An automatic differentiation minimiser for non-linear models, This is the minimiser from the original CASAL package, based on ADOL-C.

```
@minimiser beta_diff
type beta_diff
tolerance 1e-6
iterations 2500
evaluations 4000
```

6.4.4. The ADOL-C minimiser

An automatic differentiation minimiser for non-linear models. See `https://projects.coin-or.org/ADOL-C` for more information.

```
@minimiser ADOLC
type adolc
step_size 1e-6
iterations 2500
evaluations 4000
tolerance 1e-6
```

6.4.5. The CppAD minimiser

An automatic differentiation minimiser for non-linear models using the MUMPS solver. See `https://coin-or.github.io/CppAD/doc/cppad.htm` and `https://www.coin-or.org/CppAD/Doc/ipopt_solve.htm` for more information.

```
@minimiser CPPAD
type cppad
```

CppAD has an implicit tolerance value of 1e-9.

Some results suggest that this solver may be one of the quickest solvers for models that have a reasonably well-defined solution, i.e., the data are informative with respect to determining the estimated parameters.

6.4.6. The Dlib minimiser

Non-automatic differentiation minimiser. See `http://dlib.net/` for more information.

```
@minimiser Dlib
type dlib
tolerance 1e-6
iterations 2500
evaluations 4000
```

6.5. Posterior profiles

If profiles are run using the command `casal2 -p`, CASAL2 will first calculate a point estimate. For each scalar parameter or, in the case of vectors or selectivities, the element of the parameter to be profiled, CASAL2 will fix its value at a sequence of n evenly spaced numbers ($step$) between the specified lower and upper bounds l and u, and calculate a point estimate at each value.

By default $step = 10$, and $(l,u) =$ (lower bound on parameter plus $(range/(2n))$, upper bound on parameter less $(range/(2n))$. Each minimisation starts at the final parameter values from the previous resulting value of the parameter being profiled. CASAL2 will report the objective function for each parameter value. The initial point estimate should be compared with the profile results, to check at least that none of the other points along the profile have a better objective function value than the initial 'minimum'.

The parameters to be profiled are specified, and optionally the number of steps, and lower bound and upper bound, for each parameter. In the case of vector parameters, the element(s) of the vector to be profiled are specified.

The initial starting point for the estimation can also be specified using `casal2 -i file`, which may improve the minimiser performance for the profiles.

If the profile results are not reasonable, it may be a result of not using enough iterations in the minimiser or a poor choice of minimiser control variables (e.g., the minimiser tolerance). It may also be useful to try other minimisers and compare the results.

6.6. Bayesian estimation

CASAL2 can use Markov chain Monte Carlo (MCMC) functionality to generate a sample from the posterior distribution of the estimated parameters with command `casal2 -m` and output the sampled values to a file, optionally keeping only every *n*th set of values.

As CASAL2 has no post-processing capabilities. CASAL2 cannot produce MCMC convergence diagnostics. To calculate these diagnostics, use a package such as BOA, plot/summarize the posterior distributions of the output quantities, and/or use a general-purpose statistical package such as **R**.

Bayesian methodology and MCMC are both large and complex topics. See Gelman et al. (1995) and Gilks et al. (1994) for details of both Bayesian analysis and MCMC methods. In addition, see Punt & Hilborn (2001) for an introduction to quantitative fish stock assessment using Bayesian methods.

This section briefly describes the MCMC algorithms used in CASAL2. See Section 10.3 for a description of the sequence of CASAL2 commands used in a full Bayesian analysis.

CASAL2 uses a straightforward implementation of the Metropolis-Hastings algorithm (Gelman et al., 1995, Gilks et al., 1994). The Metropolis-Hastings algorithm attempts to draw a sample from a Bayesian posterior distribution, and calculates the posterior density π, scaled by an unknown constant. The algorithm generates a 'chain' or sequence of values. Typically the beginning of the chain is discarded (the burn-in period) and every *N*th element of the remainder is taken as the posterior sample.

The chain is produced by taking an initial point x_0 and repeatedly applying the following rule, where x_i is the current point:

1. Draw a candidate step s from a proposal distribution J, which should be symmetric i.e., $J(-s) = J(s)$
2. Calculate $r = min(\pi(x_i + s)/\pi(x_i), 1)$
3. Let $x_{i+1} = x_i + s$ with probability r, or x_i with probability $1 - r$

An initial point estimate is produced before the chain starts, which is done so as to calculate the approximate covariance matrix of the estimated parameters (as the inverse Hessian), and may also be used as the starting point of the chain.

The starting point of the point estimate minimiser can be specified using the command `casal2 -i`. Don't start it too close to the actual estimate (either by using `casal2 -i`, or by changing the initial parameter values in input configuration file) as it takes a few iterations to determine a reasonable approximation to the Hessian.

There are currently two options for the starting point of the MCMC:

- Start from the point estimate; or
- Restart a chain given a covariance matrix and a previous starting point.

The chain moves in natural space, i.e., no transformations are applied to the estimated parameters. The default proposal distribution is a multivariate Student's t distribution centred on the current point, with covariance matrix equal to a matrix based on the approximate covariance produced by the minimiser, multiplied by a stepsize factor.

The following steps define how the initial covariance matrix of the proposal distribution is calculated:

1. The covariance matrix is taken as the inverse of the approximate Hessian from the quasi-Newton minimiser.
2. The covariance matrix is modified so as to decrease all correlations greater than `@mcmc.max_correlation` down to `@mcmc.max_correlation`, and similarly to increase all correlations less than -`@mcmc.max_correlation` up to -`@mcmc.max_correlation` (the `@mcmc.max_correlation` parameter defaults to 0.8). This should help to avoid getting 'stuck' in a lower-dimensional subspace.
3. The covariance matrix is then modified either by

   - `@mcmc.adjustment_method=covariance`: that if the variance of the ith parameter is non-zero and less than `@mcmc.min_difference` multiplied by the difference between the parameters' lower and upper bound, then the variance is changed, without changing the associated correlations, to $k =$min_diff$(upper_bound_i - lower_bound_i)$. This is done by setting

     $$\mathrm{Cov}\,(i,j)' = \mathrm{sqrt}\,(k)\,\mathrm{Cov}\,(i,j)\big/\mathrm{sd}\,(i)$$

     for $i \neq j$, and $\mathrm{var}\,(i)' = k$
   - `@mcmc.adjustment_method=correlation`: that if the variance of the ith parameter is non-zero and less than `@mcmc.min_difference` multiplied by the difference between the parameters' lower and upper bounds, then its variance is changed to $k = min_diff(upper_bound_i - lower_bound_i)$. This differs from (i) above in that the effect of this option is that it also modifies the resulting correlations between the ith parameter and all other parameters.

   This allows each estimated parameter to move in the MCMC even if its variance is very small according to the inverse Hessian. In both cases, the `@mcmc.min_difference` parameter defaults to 0.0001.
4. The `@mcmc.stepsize` (a scalar factor applied to the covariance matrix to improve the acceptance probability) is set by the user. The default is $2.4d^{-0.5}$ where d is the number of estimated parameters, as recommended by Gelman et al. (Gelman et al., 1995). However, a smaller value may often be better.

The proposal distribution can also change adaptively during the chain, using two different mechanisms. Both are offered as means of improving the convergence properties of the chain. It is important to note that any adaptive behaviour must finish before the end of the burn-in period, i.e., the proposal distribution must be finalised before the kept portion of the chain starts.

The adaptive mechanisms are:

- The stepsize changes adaptively at one or more sample numbers (See next paragraph for details on the stepsize adaptation methods)
- The entire covariance matrix changes adaptively at one or more sample numbers. At each adaptation, the covariance matrix is replaced with an empirical covariance matrix, derived from the MCMC chain. The idea is that an empirical covariance is a better approximation of the proposal distribution than the inverse of the Hessian matrix, and can improve convergence and mixing of the chain.

The two options to adapt the step size are `double_half` or `ratio`, which is chosen with the input parameter `adapt_stepsize_method`. The `double_half` method is used in CASAL (see Gelman et al. (Gelman et al., 1995) for justification).

The algorithm for `double_half` is, at each adaptation, the stepsize is doubled if the acceptance rate since the last adaptation is more than 0.5, or halved if the acceptance rate is less than 0.2. The `ratio` is taken from SPM. It adapts the current step size by the acceptance rate since the last adaptation multiplied by 4.1667 to reach an acceptance rate of ≈ 0.24. See Sherlock and Roberts (2009) for justification on that acceptance rate.

The stepsize parameter is now on a completely different scale, and must be rescaled. It is set to a user-specified value (which may or may not be the same as the initial stepsize). Set the stepsize adaptations to occur after this, so that the stepsize can be readjusted to an appropriate value which gives good acceptance probabilities with the new matrix.

All modified versions of the covariance matrix are printed to the standard output, but only the initial covariance matrix (inverse Hessian) is saved to the objectives file. The number of covariance modifications by each iteration is recorded as a column on the objectives file.

The variance-covariance matrix of this sub-sample of chain is calculated. As above, correlations greater than `@mcmc.max_correlation` are reduced to `@mcmc.max_correlation`, correlations less than `-@mcmc.max_correlation` are increased to `-@mcmc.max_correlation`, and very small non-zero variances are increased (`@mcmc.covariance_adjustment` and `@mcmc.min_difference`). The result is the new variance-covariance matrix of the proposal distribution.

The procedure used to choose the sample of points is that, to start, all points on the chain so far are taken. (TODO reword this paragraph) All points in an initial user-specified period are discarded. The assumption is that the chain will have started moving during this period. If this is incorrect and the chain has still not moved by the end of this period, it is a fatal error and CASAL2 stops. The remaining set of points must contain at least some user-specified number of transitions. If this is incorrect and the chain has not had at least this number of transitions, then it is also a fatal error. If this test is passed, the set of points is systematically sub-sampled down to 1000 points (and it must be at least this long to start with).

The probability of acceptance for each jump is 0 if the jump would move a parameter value outside of its bounds, 1 if it improves the posterior, or $(newposterior/oldposterior)$ otherwise. How often the position of the chain is recorded is specified with the `keep` parameter. For example, with `keep 10`, only every 10th sample is recorded.

The option to specify that some of the estimated parameters are fixed during the MCMC is available. If the chain starts at the point estimate or at a random location, these fixed parameters are set to their values at the point estimate.

If the start of the chain is specified with the command `casal2 -i`, these fixed parameters are set to the values in the file.

Restarting an MCMC chain: in the case where an MCMC chain was halted or interrupted, the MCMC chain can be restarted from where it finished with

```
casal2 -m --resume --objective-file Objective_file_name --sample-file Sample_file_name
```

where `Objective_file_name` is the file name for the objective function report and `Sample_file_name` is the file name for the sample report from a MCMC chain.

The posterior sample can be used for (projections (Section 5.13)) or simulations (Section 7.6) with the values supplied with the command `casal2 -i` *`file`*.

A multivariate Student's t distribution is used as an alternative to the multivariate normal proposal distribution. If you request multivariate Student's t proposals, change the degrees of freedom from the default of 4. As the degrees of freedom decreases, the t distribution becomes more heavy tailed.

This may lead to better convergence properties. Note the default is the multivariate Student's t.

Given a posterior (sub)sample, CASAL2 can calculate a list of output quantities for each sample point (see Section 8 specifically tabular report). These quantities can be output to a file (with the command `casal2 -r -tabular`) and read into an external software package where the posterior distributions can be plotted and/or summarised.

The posterior sample can also be used for projections (Section 5.13). The advantage of this is that the parameter uncertainty, as expressed in the posterior distribution, can be included into the risk estimates.

6.7. Priors

In a Bayesian analysis, a prior is required for every parameter that is being estimated. There are no default priors.

When some of these priors are parameterised in terms of mean, c.v., and standard deviation, these refer to the parameters of the distribution before the bounds are applied. The moments of the prior after the bounds are applied may differ.

CASAL2 has the following priors (expressed in terms of their contribution to the objective function):

1. Uniform

$$-\log(\pi(p)) = 0 \tag{6.2}$$

2. Uniform-log (i.e., $\log(p) \sim$ uniform)

$$-\log(\pi(p)) = \log(p) \tag{6.3}$$

3. Normal with mean μ and c.v. c

$$-\log(\pi(p)) = 0.5\left(\frac{p-\mu}{c\mu}\right)^2 \tag{6.4}$$

4. Normal with mean μ and standard deviation σ

$$-\log(\pi(p)) = 0.5\left(\frac{p-\mu}{\sigma}\right)^2 \tag{6.5}$$

5. Lognormal with mean μ and c.v. c

$$-\log(\pi(p)) = \log(p) + 0.5\left(\frac{\log(p/\mu)}{s} + \frac{s}{2}\right)^2 \tag{6.6}$$

where s is the standard deviation of $\log(p)$ and $s = \sqrt{\log(1+c^2)}$.

6. Beta with mean μ and standard deviation σ, and range parameters A and B

$$-\log(\pi(p)) = (1-m)\log(p-A) + (1-n)\log(B-p) \tag{6.7}$$

where $\nu = \frac{\mu-A}{B-A}$, and $\tau = \frac{(\mu-A)(B-\mu)}{\sigma^2} - 1$ and then $\mu = \tau\nu$ and $n = \tau(1-\nu)$. Note that the beta prior is undefined when $\tau \leq 0$.

Vectors of parameters can be independently (but not necessarily identically) distributed according to any of the above forms, in which case the joint negative-log-prior for the vector is the sum of the negative-log-priors of the components. Values of each parameter need to be specified for each element of the vector. Example of syntax to define the estimation of a parameter and the prior assumed:

```
## uniform-log example estimate
@estimate B0
type uniform_log # this command "type" defines the prior type.
parameter process[Recruitment].b0 # "Recruitment" is the label of your process
upper_bound 20000
lower_bound 1000

## Lognormal YCS estimation
@estimate year_class_strengths_1990_1995
type lognormal
parameter process[Recruitment].ycs_values{1990:1995}
# ycs_year  1990 1991 1992 1993 1994 1995
mu    1    1    1    1    1    1
cv  0.9  0.9  0.9  0.9  0.9  0.9
lower_bound 0.01 0.01 0.01 0.01 0.01 0.01
upper_bound 9 9 9 9 9 9
```

6.8. Penalties

Penalties are associated with processes and can be used to enforce parameter value or derived quantity restrictions or model outputs that are invalid by adding a penalty to the objective function. For example, estimated parameter values can be restricted so that a known mortality event removes enough individuals from the population within an event mortality process. CASAL2 requires penalty functions for processes that remove or shift a *number* of individuals between categories or from the partition. Many of the penalties that were available in CASAL have been moved to be additional priors in CASAL2()see Section 6.9).

For most penalties, a multiplier is required to be specified, and the objective function is increased by this multiplier multiplied by the penalty value. In some cases the multiplier may need to be quite large to prohibit some model behaviour.

Penalties are implemented for the processes

- `@process[label].type=event_mortality`,
- `@process[label].type=mortality_instantaneous`,
- `@process[label].type=tag_by_length`,
- `@process[label].type=tag_by_length`, and

- `@process[label].type=category_transition`

For these processes, two types of penalties can be defined: on the natural scale (the default) and on the log scale. Both of these types add a penalty value of the squared difference between the observed value (e.g., the actual number of individuals to be removed in an event mortality process or the actual number of individuals to shift in a category transition process), and the number that were moved (if less than or equal), multiplied by the penalty multiplier.

The natural scale penalty calculates the squared difference on a natural scale, and the log scale penalty calculates the squared difference of the logged values.

For example:

```
@process Mortality
type mortality_instantaneous
penalty CatchMustBeTaken

# define the penalty in an @penalty block
@penalty CatchMustBeTaken
type process
log_scale True
multiplier 10000
```

Penalties are added to the objective function in the following ways;

$$Penalty = (X_1 - X_2)^2 \tag{6.8}$$

or if `log_scale true`

$$Penalty = (log(X_1) - log(X_2))^2 \tag{6.9}$$

where, for example, X_1 is observed catch biomass and X_2 is the estimated catch biomass. Penalties are usually applied in situations when numbers or weight are known. Another example is for tagging, where the number of individuals that were tagged in a given year is known, so a penalty can be used to restrict the model to estimate reasonable values for the numbers of tagged individuals in that year.

6.9. Additional Priors

Additional priors can be thought of as the inverse of penalties (TODO please rephrase). For CASAL models, most of the legacy `@penalty` blocks have now been implemented as `@additional_prior` blocks. They restrict parameters in user-defined spaces (TODO please rephrase).

The types of additional priors available in CASAL2 are `vector_smoothing`, `vector_averaging`, `uniform_log`, `lognormal`, `element_difference`, and `Beta`:

- `vector_averaging`

  This prior can be applied to a vector parameter. Sum of squares of rth differences, optionally on a log scale. This encourages the vector to be like a polynomial of degree $(r-1)$. A range of the vector to be "smoothed" can be specified (and if not, the smoother is applied to the entire vector). However, this restriction must be specified by an index of the vector and must be between 1 and the length of the vector, inclusive.

- `vector_smoothing`

  This prior can be applied to a vector parameter. Square of (mean(vector)-k), or of (mean(log(vector))-l), or of (log(mean(vector)/m)). Restricts the vector to average arithmetically to k or m, or geometrically to exp(l). Typically used for YCS with k=1 or m=1 or l=0, to restrict the YCS to centre on 1. Optionally, indices can be chosen or excluded outside a given set of bounds.

- `lognormal` with mean μ and c.v. c

$$-\log(\pi(p)) = \log(p) + 0.5\left(\frac{\log(p/\mu)}{s} + \frac{s}{2}\right)^2 \tag{6.10}$$

- `uniform_log`

$$-\log(\pi(p)) = \log(p) \tag{6.11}$$

- `element_difference`

$$-\log(\pi(p_1, p_2)) = \sum_{i=1}^{n} (p_{1,i} - p_{2,i})^2 \tag{6.12}$$

- `Beta`

  Beta with mean μ and standard deviation σ, and range parameters A and B, for parameter value = p

$$-\log(\pi(p)) = (1-m)\log(p-A) + (1-n)\log(B-p) \tag{6.13}$$

  where $\nu = \frac{\mu - A}{B - A}$, and $\tau = \frac{(\mu - A)(B - \mu)}{\sigma^2} - 1$ and then $m = \tau\nu$ and $n = \tau(1-\nu)$. The beta prior is undefined when $\tau \leq 0$.

Methods available for the type `vector_average` are `l`, `k`, `m`. For a target vector parameter $\mathbf{X}$ and target mean k, the contribution to the objective score is

- `method k`

  $-\log(\pi(p)) = (\bar{X} - k)^2$

- `method l`

  $-\log(\pi(p)) = \left(\overline{ln(X)} - k\right)^2$

- `method m`

  $-\log(\pi(p)) = (ln(\bar{X}) - k)^2$

where $\overline{ln(X)}$ is the mean of the logged values.

There are a range of parameters and derived values that additional priors can be applied to. Here are a list of non-estimated (all parameters that can be estimated can have an additional prior attached to them) parameters that additional priors can be applied to.

- `selectivity[Selectivity_label].values{i:j}`.

  This subcommand applies a selectivity (???) to the actual selectivity value by age (for ages i through j). This option is available only for certain types of selectivities (`all_values`, `all_values_bounded`, `double_exponential`). See the Hoki stock assessment for an example of applying additional priors on selectivities.

- `catchability[Catchability_label].q`

  This subcommand is for catchabilities that are of type `nuisance` only. Since `nuisance` qs are not free parameters, additional priors can be applied to replicate CASAL models with `@estimate` blocks in nuisance qs. If a CASAL model applied a uniform prior, then this has a null effect and this functionality can be ignored when converting to a CASAL2 model.

This list may be useful for users who are trying to apply the equivalent CASAL penalties in a CASAL2 model.

6.10. Parameter Transformations

CASAL2 has multiple methods to transform a parameter, with some methods developed for legacy purposes. Transformations are implemented to try and achieve 'better' model optimisation. Complex population models can have highly correlated parameters so transforming them to be orthogonal or to be in a different space is a way of addressing correlations, and to allow the minimiser to find a 'global' minimum quicker. To read more about transformations and get a better understanding of why they are used, see Gilks et al. (1995), specifically chapter 6.

There are two main transformation methods available in CASAL2, `transform_with_jacobian` and `prior_applies_to_transform`. When using `transform_with_jacobian`, priors are defined on parameters in natural/model space (the usual priors) but when the parameter is passed to the minimiser it gets transformed and a Jacobian is added to the objective function to account for this transformation. The second method is when bounds and prior parameters on the parameters can be specified in transformed space. Note that both `prior_applies_to_transform` and `transform_with_jacobian` cannot be set to `true`.

There are two ways that transformations can be applied. The first is within the `@estimate` block, which is for univariate (simple) transformations only. For more complex transformations a `@estimate_transformation` block must be specified to describe the transformation.

For example:

```
## simple transformation
@estimate log_R0
type lognormal
transformation log
parameter process[Recruitment].r0
transform_with_jacobian true
mu 442413
cv 0.2
lower_bound 3000
upper_bound 24154953


## more complex transformation
@estimate R0
type lognormal
```

```
parameter process[Recruitment].r0
mu 442413
cv 0.2
lower_bound 3000
upper_bound 24154953

@estimate_transformation Log_R0
type log
estimate_label log_R0
transform_with_jacobian true
```

Transform with Jacobian

The support of a random variable X with density $p_X(x)$ is that subset of values for which it has non-zero density,

$$supp(X) = \{x | p_X(x) > 0\} \tag{6.14}$$

If f is a transformation function defined on the support of X, then $Y = f(X)$ is a new random variable (transformed variable).

This section shows the available transformations in CASAL2 and the probability density function of Y.

Suppose X is one dimensional and f: $supp(X) \rightarrow \mathbf{R}$ is a one-to-one, monotonic function with a differentiable inverse f^{-1}. Then the density of Y is

$$p_Y(y) = p_X(f^{-1}(y)) \left| \frac{\partial}{\partial y} f^{-1}(y) \right| \tag{6.15}$$

where $\left| \frac{\partial}{\partial y} f^{-1}(y) \right|$ is the Jacobian term. The Jacobian measures how the scale of the transformed variable changes with respect to the underlying variable. This can be expanded to the multivariate case where the Jacobian becomes a matrix of partial derivatives.

In equation 6.15 the term $p_X(f^{-1}(y)) = p_X(X)$ and in a Bayesian context is the prior of the untransformed variable/parameter. **Note:** if this functionality is in use be careful interpreting the covariance matrix as this will be related to the transformed variable not the variable space, e.g., if natural mortality (M) is estimated as $Y = M/2$, then the covariance matrix will be described for Y.

```
@estimate log_R0
type lognormal
transformation log
parameter process[Recruitment].r0
transform_with_jacobian true
mu 442413
cv 0.2
lower_bound 3000
upper_bound 24154953
```

Transform without Jacobian but prior defined in transformed space

This transformation is where the priors are defined in transformed space. This class of transformations contains functionality that was implemented in the CASAL.

If $f()$ is a transformation function defined on the support of X, then $Y = f(X)$ is a new random variable (transformed variable). In this class *a priori* information is specified with regard to $p_Y(y)$ and X can be thought of as a derived quantity.

For example:

```
@estimate log_R0
type lognormal
parameter process[Recruitment].r0
prior_applies_to_transform true
mu 13
cv 0.5
lower_bound 8
upper_bound 17
```

Transformation types

- `log` : natural logarithm transformation
  `is_simple = true`
  `jacobian defined = true`
  $Y = ln(X)$
  $\left|\frac{\partial}{\partial y} f^{-1}(y)\right| = X^{-1}$

- `inverse`
  `is_simple = true`
  `jacobian defined = true`
  $Y = X^{-1}$
  $\left|\frac{\partial}{\partial y} f^{-1}(y)\right| = -X^{-2}$

- `sqrt` : square root transformation
  `is_simple = true`
  `jacobian defined = true`
  $Y = \sqrt{X}$
  $\left|\frac{\partial}{\partial y} f^{-1}(y)\right| = -X^{-1.5}$

- `average_difference` : two parameters θ_1 and θ_2 are transformed to Y_1 and Y_2, where Y_1 is the average of the original parameters and Y_2 is the difference between the mean and each parameter.
  `is_simple = false`
  `jacobian defined = false`
  $Y_1 = \frac{\theta_1+\theta_2}{2}$
  $Y_2 = (Y_1 - \theta_2)2$

Restore transformations
$\theta_1 = Y_1 + 0.5Y_2$
$\theta_2 = \theta_1 - 0.5Y_2$
$\left|\frac{\partial}{\partial y} f^{-1}(y)\right|$ Hasn't been assessed (i.e it could exist) (TODO ?????)

- `log_sum` : two parameters θ_1 and θ_2 are transformed to Y_1 and Y_2, where Y_1 is the natural logarithm of the sum of θ_1 and θ_2. Y_2 describes the proportion of the sum with respect to θ_1
  `is_simple = false`
  `jacobian defined = false`
  $Y_1 = ln(\theta_1 + \theta_2)$
  $Y_2 = \theta_1/(\theta_1 + \theta_2)$
  Restore transformations
  $\theta_1 = exp(Y_1)Y_2$
  $\theta_2 = exp(Y_1)(1 - Y_2)$
  $\left|\frac{\partial}{\partial y} f^{-1}(y)\right|$ Hasn't been assessed (i.e it could exist) (TODO ?????)

- `orthogonal` : two parameters θ_1 and θ_2 are transformed to Y_1 and Y_2, where Y_1 is the multiplication of θ_1 and θ_2. Y_2 is the division of θ_1 and θ_2
  `is_simple = false`
  `jacobian defined = true`
  $Y_1 = \theta_1\theta_2$
  $Y_2 = \theta_1/\theta_2$
  Restore transformations
  $\theta_1 = \sqrt{Y_1Y_2}$
  $\theta_2 = \sqrt{Y_1/Y_2}$
  $\left|\frac{\partial}{\partial y} f^{-1}(y)\right| = 2Y_2$

- `SumToOne` : given two parameters θ_1 and θ_2 that have the constraint $\sum_{i=1}^{2} \theta_i$, estimate θ_1 only given $\theta_2 = 1 - \theta_1$
  `is_simple = false`
  `jacobian defined = false`

7. The observation section

7.1. Observations

The objective function calculates the goodness-of-fit of the model to the observation data. Observations are typically supplied at an instance in time, over a group of aggregated categories. Most observations are sampled over time, i.e., data which were recorded for one or more years, in the same format each year. Examples of time series data types include relative abundance indices, commercial catch length frequencies, and survey numbers-at-age.

Definitions for each type of observation are described below, including how the observed values should be formatted, how CASAL2 calculates the expected values, and the likelihoods that are available for each type of observation.

There are two main types of observations available in CASAL2. The first type is observations that are associated with a **mortality block**. The second type is observations that are associated with a specific process. These observations use the `type` subcommand.

If an observation begins with `process` it is an observation that is associated with a process. If an observation does **not** begin with `process` it is associated with the mortality block of the defined time step. For example, the observation type `process_abundance` is a process-based observation, whereas `process_abundance abundance` is an observation that is associated with a mortality block.

There are two types of process observations. **Specific process observations** are observations that are associated with a specific process (e.g., `process_proportions_migrating`), and **general process observations** are observations that can be associated with any process (e.g., `process_proportions_at_age`). The observation types are described in different sections.

7.1.1. Mortality block associated observations

All observations within this class are calculated similarly. That is, the expected values are calculated at the beginning of the mortality block and at the end of the mortality block. CASAL2 then uses a linear interpolation to approximate the expected values part way through a mortality block using the subcommand `time_step_proportion`. This feature could be useful if a survey occurs part way through an exploitation phase, which may be part way through a fishing season when modelling a fish population. Each observation in this class will evaluate different expectations of the partition (explained in the following descriptions).

The observation `types` available with this class of observations are:

- `abundance`
- `biomass`
- `proportions_at_age`
- `proportions_at_length`
- `proportions_by_category`
- `tag_recapture_by_length`
- `tag_recapture_by_age`

Abundance or biomass observations

Abundance (or biomass) observations are observations of either a relative or absolute number (or biomass) of individuals from a set of categories after applying a selectivity. The observation classes are the same, except that a biomass observation will use the biomass as the observed (and expected) value (calculated from mean weight of individuals within each age and category) while an abundance observation is the number of individuals.

Each observation is for a given year and time-step, for some selected age classes of the population (for a range of ages multiplied by a selectivity), for aggregated categories. Furthermore, the label of the catchability coefficient q is required; q can either be estimated of fixed. For absolute abundance or absolute biomass observations, define a catchability where $q = 1$.

The observations can be supplied for any set of categories. For example, for a model with the two categories *male* and *female*, an observation of the total abundance/biomass (male + female) or male-only abundance/biomass could be provided. The subcommand `categories` defines the categories used to aggregate the abundance/biomass. In addition, each category must have an associated selectivity, defined by `selectivities`.

For example,

```
categories male
selectivities male-selectivity
```

defines an observation for males after applying the selectivity male-selectivity. CASAL2 then requires that an observation is supplied. The expected values for the observations will be the expected abundance (or biomass) of males, after applying the selectivities, at the year and time-step specified.

CASAL2 calculates the expected values by summing over the defined ages (via the age range and selectivity) and categories at both the beginning and end of a mortality block. CASAL2 will approximate the expectation part way through the mortality block using the `time_step_proportion`. The default `time_step_proportion` value is 0.5. CASAL2 does linear interpolation between the start and end abundance (or biomass) from the mortality block.

For an abundance observation the expected value is

$$E_{i,1} = \sum_{c=1} \sum_{a=1}^{A} S_{a,c} N_{a,c,i,1} \tag{7.1}$$

$$E_{i,2} = \sum_{c=1} \sum_{a=1}^{A} S_{a} N_{a,c,i,2} \tag{7.2}$$

Where $E_{i,1}$ is the expectation at the beginning of time step and $E_{i,2}$ is the expectation at the end of the time-step. S_a is the selectivity for age a and category c. If there is no mortality related to this observation then E_i which is used in the likelihood contribution is $E_{i,1}$. If this was a biomass observation, then $N_{a,c,i,1}$ in Equations (7.1) and (7.2) is replaced with $N_{a,c,i,1}\bar{w}_{a,c}$, where $\bar{w}_{a,c}$ is the mean weight of category c at age a. If the user wishes to apply 100% mortality then $E_i = E_{i,2}$.

For applying quantities of mortality between these values (M_i), the linear interpolation is

$$E_i = |E_{i,1} - E_{i,2}| M_i \tag{7.3}$$

For each year of observations, the observation table `table obs` has a row with year in the first column, the observation per category in the middle column(s), and the error value in the final column:

```
@observation MyAbundance
type abundance
years 1999
...
categories male
table obs
1999 1000 0.10
end_table
...
```

For an observation aggregated over multiple categories:

```
@observation MyAbundance
type abundance
years 1990 1991
...
categories male+female
table obs
1990 1000 0.10
1991 1200 0.12
end_table
...
```

For observations for multiple categories:

```
@observation MyAbundance
type abundance
years 1990 1991
...
categories male female
table obs
1990 550 450 0.10
1991 700 500 0.12
end_table
...
```

To define a biomass observation instead of an abundance observation, use

```
@observation MyBiomass
type biomass
...
```

Proportions-at-age

Proportions-at-age observations are observations of the relative number of individuals at age, via some selectivity.

The observation is supplied for a given year and time-step, for some selected age classes of the population (i.e., for a range of ages multiplied by a selectivity), for categories aggregated over a

set of spatial cells. Note that the categories defined in the observations must have an associated selectivity, defined by `selectivities`.

The age range must be ages defined in the partition (i.e., between `@model.min_age` and `@model.max_age` inclusive); the upper end of the age range can optionally be a plus group, which must be either the same as or less than the plus group defined for the partition.

Proportions-at-age observations can be supplied as

- a set of proportions for a single category,
- a set of proportions for multiple categories, or
- a set of proportions across aggregated categories.

The method of evaluating expectations are the same for all three types of proportions. The definitions of these proportions and the expected dimensions of observation and error inputs that CASAL2 expects for each respective proportion type are described below with examples.

Like all types of observations that are associated with the mortality block, CASAL2 will evaluate the numbers at age before and after the mortality block for the specified time step of the observation, and applying the user-defined selectivity. CASAL2 then generates the expectations from the partition part way through the mortality block using the subcommand `time_step_proportion`. This approximation is a linear interpolation of the numbers-at-age over the mortality block.

The ageing error is then applied, if the user has specified it. Finally, CASAL2 converts the numbers-at-age to proportions-at-age by dividing all numbers in an age bin by the total numbers. The likelihood for the proportions-at-age observation is then calculated.

Defining an observation for a single category is used to model a set of proportions of a single category by age class. For example, to specify that the observations are of the proportions of male within each age class, then the subcommand `categories` for the `@observation[label].type=proportion_by_age` command is

```
categories male
```

CASAL2 then requires that there will be a single vector of proportions supplied, with one proportion for each age class within the defined age range, and that these proportions sum to one.

For example, if the age range was 3 to 10, then 8 proportions should be supplied, one proportion for each of the the ages 3, 4, 5, 6, 7, 8, 9, and 10. The expected values will be the expected proportions of males within each of these age classes (after omitting males aged less than 3 and older than 10), after applying a selectivity at the year and time-step specified. The supplied vector of proportions (i.e., in this example, the 8 proportions) must sum to one, which is evaluated with a default tolerance of 0.001.

```
@observation MyProportions
type proportions_at_age
...
categories male
min_age 3
max_age 9
years 1990
table obs
1990 0.01 0.09 0.20 0.20 0.35 0.10 0.05
end_table
...
```

Defining an observation for multiple categories extends the single category observation definition. It is used to model a set of proportions over several categories by age class. For example, to specify that the observations are of the proportions of male or females within each age class, then the subcommand `categories` for the `@observation[label].type=proportion_by_age` command is

```
categories male female
```

CASAL2 then requires that there will be a single vector of proportions supplied, with one proportion for each category and age class combination, and that these proportions sum to one across all ages and categories.

For example, if there were two categories and the age range was 3 to 10, then 16 proportions should be supplied (one proportion for each of the ages 3, 4, 5, 6, 7, 8, 9, and 10, for each category male and female). The expected values will be the expected proportions of males and females within each of these age classes (after omitting those aged less than 3 and older than 10), after applying a selectivity at the year and time-step specified. The supplied vector of proportions (i.e., in this example, the 16 proportions) must sum to one, which is evaluated with a default tolerance of 0.001.

For example,

```
@observation MyProportions
type proportions_at_age
...
categories male female
min_age 1
max_age 5
years 1990 1991
table obs
1990 0.01 0.05 0.10 0.20 0.20 0.01 0.05 0.15 0.20 0.03
1991 0.02 0.06 0.10 0.21 0.18 0.02 0.03 0.17 0.20 0.01
end_table
...
```

Defining an observation across aggregated categories allows categories to be aggregated before the proportions are calculated. It is used to model a set of proportions from several categories that have been combined by age class. To indicate that two (or more) categories are to be aggregated, separate them with a '+' symbol. For example, to specify that the observations are of the proportions of male and females combined within each age class, then the subcommand `categories` for the `@observation[label].type=proportion_by_age` command is

```
categories male + female
```

CASAL2 then requires that there will be a single vector of proportions supplied, with one proportion for each age class, and that these proportions sum to one.

For example, if there were two categories and the age range was 3 to 10, then 8 proportions should be supplied (one proportion for each of the the ages 3, 4, 5, 6, 7, 8, 9, and 10, for the sum of males and females within each age class). The expected values will be the expected proportions of males + females within each of these age classes (after omitting those aged less than 3 and older than 10), after applying a selectivity at the year and time-step specified. The supplied vector of proportions (i.e., in this example, the 8 proportions) must sum to one, which is evaluated with a default tolerance of 0.001.

For example,

```
@observation MyProportions
type proportions_at_age
...
years 1990 1991
categories male+female
min_age 1
max_age 5
table obs
1990 0.02 0.13 0.25 0.30 0.30
1991 0.02 0.06 0.18 0.35 0.39
end_table
...
```

The latter form can then be extended to include multiple categories, or multiple aggregated categories. For example, to describe proportions for the three groups: immature males, mature males, and all females (immature and mature females added together) for ages 1 through 4, a total of 12 proportions are required

```
@observation MyProportions
type proportions_at_age
...
categories male_immature male_mature female_immature+female_mature
min_age 1
max_age 4
years 1990
table obs
year 1990 0.05 0.15 0.15 0.05 0.02 0.03 0.08 0.04 0.05 0.15 0.15 0.08
end_table
...
```

Proportions-at-length

Functionality for defining combinations of categories and aggregated categories directly translates from proportions-at-age to proportions-at-length. The difference is the observation is over length bins instead of age classes. CASAL2 calculates the expected numbers-at-length by converting the numbers-at-age to numbers-at-length by using the age-length relationship and distribution specified for the category specified in the `@age_length` block.

Instead of supplying a minimum and maximum age, the user must supply a vector of length bins. If no length bins are specified, then the observation-specific length bins use the model length bins as the default. If observation-specific length bins are specified, they must be a sequential subset of the model length bins, with no missing or added values. For example, if the model length bins are `0 5 10 15 20 25 ... 100`, then the observation-specific length bins can be `20 25 30 35 40 45 50` but not `20 30 40 50`.

If there is no plus group, i.e., `length_plus=false`, then CASAL2 requires a vector of proportions for each year of length $n-1$, where n is the number of lengths supplied. If `length_plus=true` then CASAL2 expects a vector of proportions for each year of length n. The last proportion represents the numbers from the last length bin to the maximum length the age-length relationship allows.

```
@observation Observed_Length_frequency_Chat_east
```

```
type process_removals_by_length
years 1991 1992
likelihood multinomial
time_step Summer
fishery EastChathamRise
process instant_mort
categories male
length_plus false
length_bins 0 20 40 60 80 110
table obs
1991    0.2   0.25    0.15     0.2     0.2
1992    0.12  0.25    0.28    0.25    0.1
end_table
table error_values
1991 25
1992 37
end_table
```

Proportions-by-category observations

TODO 'between categories' or 'within categories'?

Proportions-by-category observations are observations of either the relative number of individuals between categories within age classes, or relative biomass between categories within age classes.

The observation is supplied for a given year and time-step, for selected age classes of the population (i.e., for a range of ages multiplied by a selectivity).

The age range must be ages defined in the partition (i.e., between `@model.min_age` and `@model.max_age` inclusive); the upper end of the age range can optionally be a plus group, which may or may not be the same as the plus group defined for the partition.

Proportions-by-category observations can be supplied for any set of categories as a proportion of themselves and any set of additional categories. For example, for a model with the two categories *male* and *female*, observations of the proportions of males in the population at each age class might be provided. The subcommand `categories` defines the categories for the numerator in the calculation of the proportion, and the subcommand `categories2` supplies the additional categories to be used in the denominator of the calculation. In addition, each category must have an associated selectivity, defined by `selectivities` for the numerator categories and `selectivities2` for the additional categories used in the denominator.

For example,

```
categories male
categories2 female
selectivities male-selectivity
selectivities2 female-selectivity
```

defines the proportion of males in each age class as a proportion of males + females. CASAL2 then requires that there will be a vector of proportions supplied, with one proportion for each age class within the defined age range, i.e., if the age range was 3 to 10, then 8 proportions should be supplied (one proportion for each of the the ages 3, 4, 5, 6, 7, 8, 9, and 10). The expected values will be the expected ratios of male to male + female within each of these age classes, after applying the selectivities at the year and time-step specified.

The observations must be supplied using all or some of the values defined by a categorical layer. CASAL2 calculates the expected values by summing over the ages (via the age range and selectivity) and categories for those spatial cells where the categorical layer has the same value as defined for each vector of observations.

For example,

```
@observation MyProportions
type proportions_by_category
years 1990 1991
...
categories male
categories2 female
min_age 1
max_age 5
table obs
1990 0.01 0.05 0.10 0.20 0.20
1991 0.02 0.06 0.10 0.21 0.18
end_table
...
```

Tag Recapture by length

Tag data is primarily used to estimate the population abundance of fish. In some models, this estimation can only be made outside the model and the result is used as an estimate of abundance in the model. But in CASAL2 the tagging data can, alternatively, be fitted within the model.

Before adding a tag-recapture time series, a tag-release process (Section 5.3.5) needs to be defined. Tagging events list the labels of the tags which are modelled, and define the events where fish are tagged (i.e., CASAL2 moves fish into the section of the partition corresponding to a specific tag).

The observations are divided into two parts: (i) the number of fish that were scanned, and (ii) the number of tags that were recaptured. Each number can be specified by categories, or for combinations of categories. The precise content of the scanned and recaptured observations depends on the sampling method.

The available options are:

- age: both the scanned and recaptured are vectors containing numbers-at-age. Only available in an age-based model. The selectivity ogive is redundant and cannot be supplied.
- size: both the scanned and recaptured are vectors containing numbers-at-size. Can be used in either an age- or size-based model. The selectivity ogive is redundant and cannot be supplied.

When defining the tag-recapture time series, the following are also required:

- the time step,
- the years (unlike a tag-release process, the tag-recapture observations can occur over several years),
- the probability that each scanned tagged fish is detected as tagged (may be less than 1 if the observers are not infallible). The expected number of tags detected is calculated by multiplying this number by the number of tagged fish in the sample,
- the tagged category or categories (Make up the recaptures),
- the categories scanned (All the fish sampled for tags),

- A selectivity used in the recapture process,
- the size classes if the observations are size-based in an age-based model.

An example of a tag recapture observation:

```
## For the following partition
@categories
format sex.area.tag
names   male.Area1.2011,notag female.Area1.2011,notag

@observation Tag_2011_Area1_recap_2012 ## individuals tagged in 2011 and recaptured in 2012
## in Area1
type tag_recapture_by_length
categories format=*.Area1.*+ ## scanned categories in Area1
tagged_categories *.Area1.2011+  ## male and femaled tagged categories
detection 0.85 ## detection probability
likelihood binomial ## likelihood choice
selectivities One ## label of selectivity for tagged
tagged_selectivities One ## label of selectivity for scanned
years 2012  ## years to apply observation
time_step step2  ## time_step to apply observation
time_step_proportion 0.5 ## proportion of mortality applied before observation is calculated

table scanned
2012 281271 41360 30239 12234
end_table

table recaptured
2012 15 20 12 2
end_table

delta 1e-11 ## robustification value
dispersion 6.3 ## dispersion factor
```

The observed ($O_{y,l}$) and expected ($E_{y,l}$) values in year y and length l of this observation are:

$$O_{y,l} = \frac{R_{y,l}}{S_{y,l}} \tag{7.4}$$

where $R_{y,l}$ is the number of recaptures in year y at length l and $S_{y,l}$ are the scanned values.

$$E_{y,l} = d\frac{\tilde{N}_{y,l,t} + (\tilde{N}_{y,l,t+1} - \tilde{N}_{y,l,t}) \times p}{N_{y,l,t} + (N_{y,l,t+1} - N_{y,l,t}) \times p} \tag{7.5}$$

where $\tilde{N}_{y,l,t}$ is an element in the tagged categories at the beginning of time step t and $\tilde{N}_{y,l,t+1}$ is an element in the tagged categories at the end of time step t, $N_{y,l,t}$ is the sum of the categories that were vulnerable to sampling when the observation occurred, p is the proportion of the time step that the observation was taken, and d is the detection probability.

For observations with multiple tagged categories and multiple categories that were vulnerable to

sampling:

$$\tilde{N}_{y,l,t} = \sum_{j=1}^{J} N_{y,l,t,j} \tag{7.6}$$

where $j = \{1,2,3,...,J\}$ are all the tagged categories, the same method is applied to the vulnerable categories to calculate $N_{y,l,t}$. The tagged categories should be defined in the vulnerable categories. In an extreme case where every individual in the population is tagged, this result would be divided by zero. So, to constrain the expectation to be between 0 and 1, the numerator must be in the denominator.

The tag-recapture likelihood (binomial) is specified below. It is a modified version of the more general binomial. Note that this likelihood does not have any user-set precision parameters such as N or $c.v.$, although there are user-specified robustification and dispersion parameters available. The factorials are calculated using the log-gamma function, to allow for non-integer arguments where necessary (and to avoid overflow errors).

7.1.2. General process observations

A list of `types` that are associated with this set of observations:

- `process_abundance`
- `process_biomass`
- `process_proportions_at_age`
- `process_proportions_at_length`
- `process_proportions_by_category`

These observations have the same expectations as the mortality block versions described in Section 5.3.3.1. With the exception that instead of wrapping a mortality block they can wrap any process type available in CASAL2.

7.1.3. Specific process observations

A list of `types` that are associated with this set of observations are:

- `process_removals_by_age`
- `process_removals_by_age_retained`
- `process_removals_by_age_retained_total`
- `process_removals_by_length`
- `process_removals_by_length_retained`
- `process_removals_by_length_retained_total`
- `process_proportions_migrating`

Process removals by age

Removals-at-age observations are observations of the relative number of individuals at age, part way through a process of type `mortality_instantaneous`. This observation is exclusively associated with the process of type `mortality_instantaneous`, and will produce an error if it is associated with any other process type.

The observation is supplied for a given year and time-step, for selected age classes of the population (i.e., for a range of ages multiplied by a selectivity that is associated with the process).

The age range must be ages defined in the partition (i.e., between `@model.min_age` and `@model.max_age` inclusive); the upper end of the age range can optionally be a plus group, which must be either the same or less than the plus group defined for the partition.

The expectations from this observation are generated whilst the process is being executed. The expectation of numbers at age a for category c from exploitation method m ($E[N_{a,c,m}]$) are

$$E[N_{a,c,m}] = N_{a,c} U_{a,m} S_{a,c,m} 0.5 M_{a,c} \tag{7.7}$$

where $N_{a,c}$ are the numbers-at-age in category c before the process is executed, $U_{a,m}$ is the exploitation rate for age a from method m, $S_{a,c,m}$ is the selectivity, and M is the natural mortality.

The observation class accesses the variable $E[N_{a,c,m}]$ and applies ageing error if the user has specified it. Then the observations are aggregated by method and category depending on how the user specifies the observation, before converting numbers-at-age to proportions-at-age and then calculating the likelihood.

Likelihoods that are available for this observation class are the multinomial, Dirichlet, and the lognormal. See Section 7.2 for information on the respected likelihood.

Process removals by age retained

Observations of retained and total catches by age can be included, using the labels `process_removals_by_age_retained` and `process_removals_by_age_retained_total`, respectively. Examples of two such observations are given below, with the associated process `Instantaneous_Mortality_Retained` having the form of the example in Section 5.3.3.5.

For retained catch:

```
@observation potFishAFtotal   #test syntax get catch AF out
type process_removals_by_age_retained_total
mortality_instantaneous_process Instantaneous_Mortality_Retained
method_of_removal FishingPot
years 2005
time_step 1
categories male
### ageing_error Normal_ageing
min_age 3
max_age 15
plus_group True
```

```
table obs
2005 0.0002814574 0.0095351205 0.1661896098 0.2701718827 0.2214454177 0.1661869474 0.110793028
end_table
table error_values
2005 651
end_table
likelihood multinomial
delta 1e-11
```

For total catch:

```
@observation potFishAFretained   #test syntax --> fits to discards not catch
type process_removals_by_age_retained
mortality_instantaneous_process Instantaneous_Mortality_Retained
method_of_removal FishingPot
years 2005
time_step 1
categories male
# ageing_error Normal_ageing
min_age 3
max_age 15
plus_group True
table obs
2005 1.650990e-10 7.566419e-07 1.771126e-03 1.962050e-01 3.192775e-01 2.413644e-01 1.609208e-0
end_table
table error_values
2005 651
end_table
likelihood multinomial
delta 1e-11
```

Process removals by length

Removals by length observations are observations of the relative number of individuals at length, part way through a process of type `mortality_instantaneous`. This observation is exclusively associated with the process of type `mortality_instantaneous`, and will produce an error if associated with any other process type.

The observation is supplied for a given year and time-step, for some selected age classes of the population (i.e., for a range of ages multiplied by a selectivity that is associated with the process).

The expectations from this observation are generated whilst the process is being executed. The expectation of numbers at age a for category c from exploitation method m ($E[N_{a,c,m}]$) are

$$E[N_{a,c,m}] = N_{a,c} U_{a,m} S_{a,c,m} 0.5 M_{a,c} \tag{7.8}$$

where $N_{a,c}$ are the numbers at age in category c before the process is executed, $U_{a,m}$ is the exploitation rate for age a from method m, $S_{a,c,m}$ is the selectivity, and M is the natural mortality.

The observation class accesses the variable $E[N_{a,c,m}]$ from the process and applies the age-length relationship specified in the model. This converts numbers-at-age to numbers-at-age and -length, which are then converted to numbers-at-length. The observations are

aggregated by method and category depending on how the user specifies the observation, before converting numbers-at-age to proportions and calculating the likelihood.

Similar to the proportions-at-length observation type, the user must supply a vector of length bins. The observation-specific length bins must be a sequential subset of the model length bins, with no missing or added values. For example, if the model length bins are `0 5 10 15 20 25 ... 100`, then the observation-specific length bins can be `20 25 30 35 40 45 50` but not `20 30 40 50`.

```
@observation observation_fishery_LF
type process_removals_by_length
...
years  1993 1994 1995
method_of_removal FishingEast
mortality_instantaneous_process instant_mort
length_plus false
length_bins 0 20 40 60 80 110
delta 1e-5
table obs
1993    0.0   0.05    0.05    0.10    0.80
1994    0.05  0.1     0.05    0.05    0.75
1995    0.3   0.4     0.2     0.05    0.05
end_table

table error_values
1993 31
1994 34
1995 22
end_table
```

Likelihoods that are available for this observation are the multinomial, Dirichlet and the lognormal. See Section 7.2 for information on the likelihoods.

Process removals by length retained

Observations of retained and total catches by length can be included, using the labels `process_removals_by_length_retained` and `process_removals_by_length_retained_total` respectively. Examples of two such observations are given below, with the associated process `Instantaneous_Mortality_Retained` having the form of the example in Section 5.3.3.5.

Similar to the proportions-at-length observation type, the user must supply a vector of length bins. The observation-specific length bins must be a sequential subset of the model length bins, with no missing or added values. For example, if the model length bins are `0 5 10 15 20 25 ... 100`, then the observation-specific length bins can be `20 25 30 35 40 45 50` but not `20 30 40 50`.

For retained catch:

```
@observation potFishLFtotal   #test syntax get catch LF out
type process_removals_by_length_retained_total
mortality_instantaneous_process Instantaneous_Mortality_Retained
method_of_removal FishingPot
years 2005
time_step 1
```

```
categories male
length_bins 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 # for LF in catch
length_plus False
table obs
2005 0.05344612 0.06432242 0.07357780 0.08050385 0.08473451 0.08619620 0.08502982 0.08152921 0
end_table
table error_values
2005 651
end_table
likelihood multinomial
delta 1e-11
```

For total catch:

```
@observation potFishLFretained   #test syntax get retained LF out
type process_removals_by_length_retained
mortality_instantaneous_process Instantaneous_Mortality_Retained
method_of_removal FishingPot
years 2005
time_step 1
categories male
length_bins 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 # for LF in catch
length_plus False
table obs
2005 0.02462879 0.03536036 0.04759163 0.06025858 0.07205340 0.08169356 0.08817806 0.09095124 0
end_table
table error_values
2005 651
end_table
likelihood multinomial
delta 1e-11
```

Proportions migrating

This observation is of the proportion migrating from one area to another. This observation is exclusively associated with the process type `transition_category`, and will produce an error when associated with any other process type. This observation is used to inform migration rates in migration processes. This observation class is used in the hoki stock assessment see Francis et al. (2003) for more information on how these observations are collected and a situation that uses it.

This observation calculates an expectation E_a of proportions for each age class a that have migrated, by

$$E_a = \frac{N_a - N'_a}{N_a} \tag{7.9}$$

where N_a are the numbers of individuals in age a before the migration process occurs, and N'_a are the number of individuals after the migration process occurs.

The likelihoods that are allowed for this observation are the lognormal, multinomial, and Dirichlet.

A section of the hoki stock assessment model:

```
@observation pspawn_1993
type process_proportions_migrating
years 1993
time_step step4
process Wspmg ## migration process that the observation is associated with
age_plus true
min_age 4
max_age 9
likelihood lognormal
categories male.west+female.west ## Categories to evaluate the prportion for
ageing_error Normal_offset ## label for an @ageing_error block
table obs
#age    4    5    6    7    8    9
1993 0.64 0.58 0.65 0.66 0.71 0.60
end_table

table error_values
## if lognormal these are c.v.'s
1993 0.25
end_table
```

7.2. Likelihoods

7.2.1. Likelihoods for proportions-at-age observations

CASAL2 implements three likelihoods for proportions-at-age observations, the multinomial likelihood, the Dirichlet, and the lognormal likelihood.

The multinomial likelihood

For the observed proportions at age O_i for age classes i, with sample size N, and the expected proportions at the same age classes E_i, the negative log-likelihood is:

$$-\log(L) = -\log(N!) + \sum_i \log((NO_i)!) - NO_i \log(Z(E_i, \delta)) \tag{7.10}$$

where $\sum_i O_i = 1$ and $\sum_i E_i = 1$. $Z(\theta, \delta)$ is a robustifying function to prevent division by zero errors, with parameter $\delta > 0$.

$$Z(\theta, \delta) = \begin{cases} \theta, & \text{where } \theta \geq r \\ \delta/(2 - \theta/\delta), & \text{otherwise} \end{cases} \tag{7.11}$$

The default value of δ is 1×10^{-11}.

The Dirichlet likelihood

For the observed proportions at age O_i for age classes i, with sample size N, and the expected proportions at the same age classes E_i, the negative log-likelihood is:

$$-\log(L) = -\log(\Gamma\sum_i(\alpha_i)) + \sum_i \log(\Gamma(\alpha_i)) - \sum_i(\alpha_i - 1)\log(Z(O_i,\delta)) \tag{7.12}$$

where $\alpha_i = Z(NE_i,\delta)$, $\sum_i O_i = 1$, and $\sum_i E_i = 1$. $Z(\theta,\delta)$ is a robustifying function to prevent division by zero errors, with parameter $\delta > 0$.

$$Z(\theta,\delta) = \begin{cases} \theta, & \text{where } \theta \geq r \\ \delta/(2-\theta/\delta), & \text{otherwise} \end{cases} \tag{7.13}$$

The default value of δ is 1×10^{-11}.

The lognormal likelihood

For the observed proportions at age O_i for age classes i, with c.v. c_i, and the expected proportions at the same age classes E_i, the negative log-likelihood is defined as;

$$-\log(L) = \sum_i \left(\log(\sigma_i) + 0.5\left(\frac{\log(O_i/Z(E_i,\delta))}{\sigma_i} + 0.5\sigma_i\right)^2 \right) \tag{7.14}$$

where

$$\sigma_i = \sqrt{\log(1+c_i^2)} \tag{7.15}$$

and the c_i's are the c.v.s for each age class i, and $Z(\theta,\delta)$ is a robustifying function to prevent division by zero errors, with parameter $\delta > 0$.

$$Z(\theta,\delta) = \begin{cases} \theta, & \text{where } \theta \geq r \\ \delta/(2-\theta/\delta), & \text{otherwise} \end{cases} \tag{7.16}$$

The default value of δ is 1×10^{-11}.

7.2.2. Likelihoods for abundance and biomass observations

Abundance and biomass observations are expected as an annual time series in CASAL2, where they select the same categories over that time series. The parameters and inputs needed to use this observation class are: a observation O_i, c.v. c_i, catchability coefficient q, where i indexed the year. CASAL2 calculates an expectation E_i and scales it by q before comparing it to O_i. This means that the value chosen for q will determine whether the observation is relative ($q \neq 1$) or absolute $q = 1$. Before we describe each of the likelihoods we will discuss the methods available to handle qs:

- The qs can be treated as 'nuisance' parameters. For each set of values of the free parameters, the model uses the values of the qs which minimise the objective function. These optimal qs are calculated algebraically (see Section 7.4). If one of the qs falls outside the bounds specified by the user, it is set equal to the closest bound. This approach reduces the size of the parameter vector and hence should improve the performance of the estimation method. However, it is not correct when calculating a sample from the posterior in a Bayesian analysis (except asymptotically, see Walters and Ludwig (1994)) and we offer the following alternative;
- The qs can be treated as ordinary free parameters.

For both options, it is necessary to evaluate the contribution of O_i to the negative log likelihood for a given value of q. Each observation O_i varies about qE_i, which expresses the variability of O_i in terms of its c.v. c_i (or in one case, its standard deviation si). Here are the likelihoods, which are expressed on the objective-function scale of -log(L):

The lognormal likelihood

The negative log likelihood for the lognormal is

$$-\log(L) = \sum_i \left(\log(\sigma_i) + 0.5 \left(\frac{\log(O_i/qZ(E_i,\delta))}{\sigma_i} + 0.5\sigma_i \right)^2 \right) \tag{7.17}$$

where

$$\sigma_i = \sqrt{\log\left(1 + c_i^2\right)} \tag{7.18}$$

and $Z(\theta,\delta)$ is a robustifying function to prevent division by zero errors, with parameter $\delta > 0$.

$$Z(\theta,\delta) = \begin{cases} \theta, & \text{where } \theta \geq r \\ \delta/(2-\theta/\delta), & \text{otherwise} \end{cases} \tag{7.19}$$

The default value of δ is 1×10^{-11}.

This formulation reflects the distributional assumptions that O_i has the lognormal distribution, that the mean of O_i is qE_i and the c.v. of O_i is c_i.

The normal likelihood

For observations O_i, c.v. c_i, and expected values qE_i, the negative log-likelihood is defined as;

$$-\log(L) = \sum_i \left(\log(c_i E_i) + 0.5 \left(\frac{O_i - E_i}{Z(c_i E_i, \delta)} \right)^2 \right) \tag{7.20}$$

and $Z(\theta,\delta)$ is a robustifying function to prevent division by zero errors, with parameter $\delta > 0$.

$$Z(\theta,\delta) = \begin{cases} \theta, & \text{where } \theta \geq r \\ \delta/(2-\theta/\delta), & \text{otherwise} \end{cases} \tag{7.21}$$

The default value of δ is 1×10^{-11}.

This reflects the distributional assumptions that O_i has the normal distribution, that the mean of O_i is qE_i and the c.v. of O_i is c_i.

7.2.3. Likelihoods for tag recapture by age and length observations

The binomial likelihood

This likelihood is for situations where the size frequencies or age frequencies of the recaptured tagged fish and of the scanned fish are known. Available in both age or size based models.

The likelihood is defined as a binomial, but based on sizes, rather than ages

$$\begin{aligned} -\log(L)' = -\sum_i [\, & \log(n_i!) - \log((n_i - m_i)!) - \log((m_i)!) + m_i \log\left(Z\left(\frac{M_i}{N_i}, \delta\right)\right) \\ & + (n_i - m_i)\log\left(Z\left(1 - \frac{M_i}{N_i}, \delta\right)\right) \,] \end{aligned} \tag{7.22}$$

where

n_i = number of fish at size or age i that were scanned

m_i = number of fish at size or age i that were recaptured

N_i = number of fish at size or age i in the available population (tagged and untagged)

M_i = number of fish at size or age i in the available population that have the tag after a detection probability p_d has been applied, $M_i = M_i' p_d$, where M_i' is the expected available population that have the tag.

$Z(x, \delta)$ is a robustifying function with parameter $r > 0$ (to prevent division by zero errors).

$$Z(x, \delta) = \begin{cases} x & \text{where } x \geq \delta \\ \frac{\delta}{(2 - x/\delta)} & \text{otherwise} \end{cases}$$

If a dispersion parameter (τ) is described in the observation then the final negative log likelihood $-log(L)$ contribution is

$$-log(L) = -log(L)'/\tau$$

7.2.4. Likelihoods for proportions-by-category observations

CASAL2 implements two likelihoods for proportions-by-category observations, the binomial likelihood, and the normal approximation to the binomial (binomial-approx).

The binomial likelihood

For observed proportions O_i for age class i, where E_i are the expected proportions for age class i, and N_i is the effective sample size for age class i, then the negative log-likelihood is

$$\begin{aligned} -\log(L) = -\sum_i [\, & \log(N_i!) - \log((N_i(1-O_i))!) - \log((N_i O_i)!) + N_i O_i \log(Z(E_i,\delta)) \\ & + N_i(1-O_i)\log(Z(1-E_i,\delta))\,] \end{aligned} \tag{7.23}$$

where $Z(\theta,\delta)$ is a robustifying function to prevent division by zero errors, with parameter $\delta > 0$.

$$Z(\theta,\delta) = \begin{cases} \theta, & \text{where } \theta \geq r \\ \delta/(2-\theta/\delta), & \text{otherwise} \end{cases} \tag{7.24}$$

The default value of δ is 1×10^{-11}.

The normal approximation to the binomial likelihood

For observed proportions O_i for age class i, where E_i are the expected proportions for age class i, and N_i is the effective sample size for age class i, then the negative log-likelihood is defined as;

$$-\log(L) = \sum_i \log\left(\sqrt{Z(E_i,\delta)Z(1-E_i,\delta)/N_i}\right) + \frac{1}{2}\left(\frac{O_i - E_i}{\sqrt{Z(E_i,\delta)Z(1-E_i,\delta)/N_i}}\right)^2 \tag{7.25}$$

where $Z(\theta,\delta)$ is a robustifying function to prevent division by zero errors, with parameter $\delta > 0$.

$$Z(\theta,\delta) = \begin{cases} \theta, & \text{where } \theta \geq r \\ \delta/(2-\theta/\delta), & \text{otherwise} \end{cases} \tag{7.26}$$

The default value of δ is 1×10^{-11}.

7.3. Process error

Additional 'process error' can be defined for any set of observations. Additional process error has the effect of increasing the observation error in the data, and hence of decreasing the relative weight given to the data in the fitting process.

For observations where the likelihood is parameterised by the c.v., the process error can be specified for a given set of observations as a c.v., in which case all the c.v.s c_i are changed to

$$c_i' = \sqrt{c_i^2 + c_{process_error}^2} \tag{7.27}$$

Note that $c_{process_error} \geq 0$, and that $c_{process_error} = 0$ is equivalent to no process error.

Similarly, if the likelihood is parameterised by the effective sample size N,

$$N_i' = \frac{1}{1/N_i + 1/N_{process_error}} \tag{7.28}$$

Note that this requires that $N_{process_error} > 0$, but the special case of $N_{process_error} = 0$ is valid, and $N_{process_error} = 0$ represents no process error (i.e., defined to be equivalent to $N_{process_error} = \infty$).

For both the c.v. and N process errors, the process error has more effect on small errors than on large ones. Note that a large value for the N process error means a small process error.

7.4. Calculating nuisance q parameters

This section describes the theory used to calculate nuisance (analytical) catchability coefficients qs (see Section 7.2.2). From the user's point of view, the essence is that you can use nuisance qs in the following situations:

- With maximum likelihood estimation
- With Bayesian estimation, providing that the additional prior on q is one of the following:
  - None (default)
  - Uniform-log
  - Lognormal with observations distributed lognormal, robustified lognormal

The scenarios in which the nuisance catchability q can be used in a Bayesian analysis (Table 7.1):

Table 7.1: Equations used to calculate nuisance qs. (*=no analytic solution found.)

Distribution of observations	Maximum Likelihood	None	Uniform-log	Normal	lognormal
Normal	(7.29)	(7.29)	(7.31)	*	*
Lognormal	(7.32)	(7.32)	(7.36)	*	(7.37)

Note that qs are calculated for robustified lognormal likelihoods as if they were ordinary lognormal likelihoods.

Let $\sigma_i = \sqrt{log(1+c_i^2)}$ throughout, and let n be the number of observations in the time series. The case of multiple time series sharing the same q, and the modifications required for the assumption of curvature, are addressed at the end of this subsection.

First, consider maximum likelihood estimation. When the (Oi) are assumed to be normally distributed

$$-log(L) = \sum_i log(c_i q E_i) + 0.5 \sum_i \left(\frac{O_i - qE_i}{c_i q E_i} \right)^2 \tag{7.29}$$

The value of q which minimises the objective function is found by solving for q under the following condition, $\partial/\partial q(-log(L)) = 0$

$$\frac{\partial}{\partial q}(-log(L)) = \frac{n}{q} + \frac{1}{q^2} \sum_i \frac{O_i}{c_i^2 E_i} - \frac{1}{q^3} \sum_i \left(\frac{O_i}{c_i E_i} \right)^2 \tag{7.30}$$

hence

$$\hat{q} = \frac{-S_1 + \sqrt{S_1^2 + 4nS_2}}{2n} \tag{7.31}$$

where $S_1 = \sum_i (O_i/c_i^2 E_i)$ and $S_2 = \sum_i (O_i/c_i E_i)^2$

When the (O_i) are assumed to be lognormally distributed,

$$-log(L) = \sum_i log(\sigma_i) + 0.5 \sum_i \left(\frac{log(O_i) - log(qE_i) + 0.5\sigma_i^2}{\sigma_i} \right)^2 \tag{7.32}$$

$$\frac{\partial}{\partial q}(-log(L)) = \frac{-1}{q} \sum_i \left(\frac{log(O_i/E_i) - log(q) + 0.5\sigma_i^2}{\sigma_i^2} \right) \tag{7.33}$$

$$\hat{q} = exp \frac{0.5n + S_3}{S_4} \tag{7.34}$$

where $S_3 = \sum_i (log(O_i/E_I)/\sigma_i^2)$ and $S_4 = \sum_i (1/\sigma_i^2)$.

Next, consider Bayesian estimation, where a prior for q must be specified.

The effects of the prior on the equations are to replace likelihood L by posterior P throughout, to add $-log(\pi(q))$ to the equation for $-log(P)$ and $\partial/\partial q(-log(-\pi(q)))$ to the equation for $\partial/\partial q(-log(P))$

This last term is 0 for a uniform prior on q, $1/q$ for a log-uniform prior, and $\frac{1}{q}\left(1.5 + \frac{log(q) - log(\mu_q)}{\sigma_q^2}\right.$ for a lognormal prior, where μ_q and c_q are the mean and c.v. of the prior on q, respectively, and $\sigma_q = \sqrt{log(1 + c_q^2)}$. Since the prior is uniform, the equation for $\hat{q}$ is the same as the maximum likelihood estimation.

When the (O_i) are assumed to be normally distributed and the prior is log-uniform equation (7.31) becomes,

$$\hat{q} = \frac{-S_1 + \sqrt{S_1^2 + 4(n+1)S_2}}{2(n+1)} \tag{7.35}$$

but $\hat{q}$ with either a normal or lognormal prior cannot be solved for.

When the O_i are assumed to be lognormally distributed and the prior is log-uniform, equation (7.34) becomes

$$\hat{q} = exp \frac{0.5n - 1 + S_3}{S_4} \tag{7.36}$$

and if the prior is lognormal,

$$\hat{q} = exp \frac{0.5n - 1.5 + log(\mu_q)/\sigma_q^2 + S_3}{S_4 + 1/\sigma_q^2} \tag{7.37}$$

However, it is not possible to solve for $\hat{q}$ with a normal prior.

An example of specifying the syntax and an equivalent additional prior

```
@catchability chatTANq
type nuisance
upper_bound 0.6
lower_bound 0.0001

@additional_prior chatTANq_prior
type lognormal
parameter catchabilityp[chatTANq].q
mu 0.3
cv 0.2
```

7.5. Ageing error

CASAL2 can apply ageing error to expected age frequencies estimated by the model. The ageing error is applied as a misclassification matrix, which has the effect of 'smearing' the expected age frequencies. This is mimicking the error involved in identifying the age of individuals. For example, fish species are aged by reading the ear bones (otoliths) which can be quite difficult depending on the species. These age frequencies are used in calculating the fits to the observed values, and hence the contribution to the total objective function.

Ageing error is optional, and if it is used, it may be omitted for any individual time series. Different ageing error models may be applied for different observation commands. See Section 8.14 for reporting the misclassification matrix at the end of model run.

The ageing error models implemented are

- None: The default model is to apply no ageing error.
- Off by one: Proportion p_1 of individuals of each age a are misclassified as age $a-1$ and proportion p_2 are misclassified as age $a+1$. Individuals of age $a < k$ are not misclassified. If there is no plus group in the population model, then proportion p_2 of the oldest age class will 'fall off the edge and disappear'.
- Normal: Individuals of age a are classified as ages which are normally distributed with mean a and constant c.v. c. As above, if there is no plus group in the population model, some individuals of the older age classes may disappear. If c is high enough, some of the younger age classes may 'fall off the other edge'. Individuals of age $a < k$ are not misclassified.

The expected values (fits) reported by CASAL2 for observations with ageing error will have had the ageing error applied.

7.6. Simulating observations

CASAL2 can generate simulated observations for a given model with a set of parameter values using `casal2 -s 1` (To simulate one set of simulated observations). Simulated observations are randomly generated values, which are generated with the error distributions defined for each observation, around fits calculated from one or more sets of the 'true' parameter values. Simulating from a set of parameters can be used to generate observations from an operating model or as a form of parametric bootstrap.

The procedure CASAL2 uses for simulating observations is to use the 'true' parameter values and generate the expected values. Then, if a set of observations use ageing error, ageing error is applied. Finally, a random value for each observed value is generated based on (i) the expected values, (ii) the type of likelihood specified, and (iii) the variability parameters (e.g., `error_value` and `process_error`).

Methods for generating the random error, and hence the simulated values, depend on the specific likelihood type of each observation.

- Normal likelihood parameterised by c.v.: Let E_i be the fitted value for observation i, and c_i be the corresponding c.v. (adjusted by the process error if applicable). Each simulated observation value S_i is generated as an independent normal deviate with mean E_i and standard deviation $E_i c_i$.
- Log-normal likelihood: Let E_i be the fitted value for observation i and c_i be the corresponding c.v. (adjusted by the process error if applicable). Each simulated observation value S_i is generated as an independent lognormal deviate with mean and standard deviation (on the natural scale, not the log-scale) of E_i and $E_i c_i$ respectively. The robustification parameter δ is ignored.
- Multinomial likelihood: Let E_i be the fitted value for observation i, for i between 1 and n, and let N be the sample size (adjusted by process error if applicable, and then rounded up to the next whole number). The robustification parameter δ is ignored. Then,
  1. A sample of N values from 1 to n is generated using the multinomial distribution, using sample probabilities proportional to the values of E_i.
  2. Each simulated observation value S_i is calculated as the proportion of the N sampled values equalling i
  3. The simulated observation values S_i are then rescaled so that their sum is equal to 1
- Binomial and the normal approximation to the binomial likelihoods: Let E_i be the fitted value for observation i, for i between 1 and n, and N_i the corresponding equivalent sample size (adjusted by process error if applicable, and then rounded up to the next whole number). The robustification parameter δ is ignored. Then,
  1. A sample of N_i independent binary variates is generated, equalling 1 with probability E_i
  2. The simulated observation value S_i is calculated as the sum of these binary variates divided by N_i

An important note when simulating: CASAL2 will **not** automatically report simulated observations when using a `casal2 -s 1` run. A report must be defined using the `simulated_observation` report (`@report[label].type=observation`). See Section 8 for more information on how to write this report.

7.7. Pseudo-observations

CASAL2 can generate expected values for observations without them contributing to the total objective function. These are called pseudo-observations, and can be used to either generate the expected values from CASAL2 for reporting or diagnostic purposes. To define an observation as a pseudo-observation, use the command `@observation[label].likelihood=none`. Any observation type can be used as a pseudo-observation. CASAL2 can also generate simulated observations from pseudo-observations. Note that

- Output will be generated only if a report command `@report[label].type=observation` is specified.
- The observed values should be supplied (even if they are 'dummy' observations). These observation values will be processed by CASAL2 as if they were actual observation values, and must be in the same format as actual observation values.
- The subcommands `likelihood`, `obs`, `error_value`, and `process_error` have no effect when generating the expected values for the pseudo-observation.
- When simulating observations, the subcommand `simulation_likelihood` to indicate the likelihood to use. In this case, the `obs`, `error_value`, and `process_error` are used to determine the appropriate terms to use for the likelihood when simulating.

7.8. Residuals

CASAL2 will print the default residual values (i.e., observed less fitted) only when the report type `@report.type=observation` is used. For an observation O and F the corresponding fit (=qE for relative observations), then

- Residuals = O - F

Pearson and normalised residuals can be generated using the CASAL2 **R** package. For specific **R** functions see Section 16.

The definitions used in the calculations are

- *Pearson residuals* attempt to express the residual relative to the variability of the observation, and are defined as (O-F)/std.dev.(O), where std.dev.(O) is calculated as
  - F $\times$ cv for normal, lognormal, robustified lognormal, and normal-log error distributions.
  - s for normal-by-standard deviation error distributions.
  - $\sqrt{\frac{Z(F,r)(1-Z(F,r))}{N}}$ for multinomial or binomial likelihoods.
  - $\sqrt{\frac{(F+r)(1-F+r)}{N}}$ for binomial-approx likelihood likelihoods.
- *Normalised residuals* to express the residual on a standard normal scale, and are defined as:
  - Equal to the Pearson residuals for normal error distributions.
  - (log(O/F)+0.5σ^2)/σ for lognormal (including robustified lognormal) error distributions, where $\sigma = \sqrt{log(1+cv^2)}$.
  - log(O/F)/σ for normal-log error distributions, again with $\sigma = \sqrt{log(1+cv^2)}$.
  - And are otherwise undefined.

where $Z(F,r)$ is the robustifying term on F (fit or expectation of the observation). This robustifying function is described earlier in the likelihood section.

8. The report section

The report section specifies the printouts and other output from the model. CASAL2 does not, in general, produce any output unless specified by a valid `@report` block.

8.1. Report command block format

Reports from CASAL2 can be defined to print partition and states objects at a particular point in time, observation summaries, estimated and derived parameter values, and objective function values.

```
@report observation_age ## label of report
type observation ## Type of report
observation age_1990 ## label corresponding to an @observation report, shown below

@observation age_1990
type proportion_at_age
year 1990
plus_group
etc ...
```

8.2. Report block output format

Reports from CASAL2 have a standard style (with one exception, the `output_parameters` report, see below). The standard style is that reports are prefixed with an asterix followed by a user-defined label and type of report in brackets (e.g., `*label (type)`), with the report ending with the line `*end`. For example,

```
*My_report(type)
...
*end
```

This report block output format should make it easier for other software packages to read and process CASAL2 output. The `extract` functions in the **R** CASAL2 package use this information to identify and read CASAL2 output.

The `output_parameters` report does not print either a header or `*end` at the end of the report block. This is because the `output_parameters` report is designed to provide a single line vector of the estimated parameter values, or multiple lines for more than one set, which can be read by CASAL2 with the command `casal2 -i`. This is a specialised report for the `casal2 -o filename` command.

For estimated values in standard output use the `type=estimate_value` report.

Reports can be defined in a `@report` command block but may not be output, e.g., a report to print the partition for a year and/or timestep that does not exist, or reporting the covariance matrix when not estimation run mode.

Certain reports are associated with certain CASAL2 run modes. These reports are ignored by CASAL2 and the program will not generate any output for these reports, although they must still conform to CASAL2 syntax requirements.

Not all reports will be generated in all run modes. Some reports are only available in some run modes. For example, when simulating, only the simulation reports will be output.

8.3. Print the partition at the end of an initialisation

This report prints the partition following the initialisation phase, which includes the numbers of individuals in each age class and category in the partition. This report will print out in run modes `-r, -e, -f`.

8.4. Print the partition

This report prints the numbers of individuals in each age class and category in the partition for each given year or given years and timestep. This report is evaluated at the end of the timestep in the given year(s). This report will print out in run modes `-r, -e, -f`.

8.5. Print the age length and length weight values

This report prints the length and weight value for each age class and category in the partition for each given year or given years and timestep. This report is evaluated at the end of the timestep in the given year(s). This report will print out in run modes `-r, -e, -f`.

```
@report length_weight_at_age
type partition_mean_weight
time_step step2
years 1900:2013
```

8.6. Print a process summary

Depending on the process, different summaries are produced. These reports typically detail the type of process, its parameters and other options, and any associated details. This report will print out in run modes `-r, -e, -f`.

8.7. Print derived quantities

This report prints the description of the derived quantity, and the values of the derived quantity as recorded in the model state, for each year of the model, and for all years in the initialisation phase. This report will print out in run modes `-r, -e, -f`.

8.8. Print the estimated parameters

This report prints a summary of the estimated parameters using the type `estimate_summary`, including the parameter name, lower and upper bounds, the label of the prior, and its value. This report will print out in run modes `-r, -e`.

8.9. Print the estimated parameters in a vector format

This report prints the estimated parameter values out as a vector. The `estimate_values` report prints the name of the parameter, followed by the value for that run. This report will print out in run modes `-r, -e`.

8.10. Print the objective function

This report prints the total objective function value, the value of all observation likelihood components, the values of all priors, and the value of any penalties that have been incurred. If an individual model run does not incur a penalty, then the penalty will not be reported. This report will print out in run modes `-r, -e, -f`.

8.11. Print the covariance matrix

This report prints the Hessian and covariance matrices if in estimation run mode and if the covariance has been requested by `@minimiser[label].covariance=true`.

8.12. Print observations, fits, and residuals

This report prints, for each category or combination of categories, the expected values, residuals (observed − expected), the error value, process error, the total error (i.e., the error value as modified by any additional process error), and the contribution to the total objective function of that individual datum in the observation.

Constants in the likelihood components are often ignored in the objective function score of individual observation values. Hence, the total score from an observation equals the contribution of the objective function scores from each individual observation value plus a constant term (if applicable). In likelihood components without a constant term, the total score from an observation will equal the contribution of the objective function scores from each individual observation value.

If CASAL2 is in simulation run mode, then the contribution to the objective function of each observation is reported as zero.

```
@report Tan_at_age_obs
type observation
observation TAN_AT_AGE
```

8.13. Print simulated observations

This report prints a complete set of observation values in the form specified by `@report[label].type=observation`, with observed values replaced by randomly generated simulated values. The output is in a form suitable for use within a CASAL2 input configuration file, reproducing the command and subcommands from the input configuration file. This report will print out in run mode `-s`.

8.14. Print the ageing error misclassification matrix

This report prints the ageing error misclassification matrix used to offset observations within during model the model fitting procedure.

8.15. Print selectivities

This report prints the values of a selectivity for each age in the partition, for a given year and at then end of a given timestep.

8.16. Print the random number seed

This report prints the random number seed used by CASAL2 to initialise the generated random number sequence. Additional runs which use the same random number seed and the same model will produce identical outputs.

8.17. Print the results of an MCMC

This report prints the MCMC samples, objective function values, and proposal covariance matrix following an MCMC. This report will print out in run mode `-m`.

8.18. Print the MCMC samples as they are calculated

This report prints the MCMC samples for each new *i*th sample as they are calculated while doing an MCMC. The output file will be appended with each new sample as it is calculated by CASAL2. This report will print out in run mode `-m`.

8.19. Print the MCMC objective function values as they are calculated

This report prints the MCMC objective function values, along with the proposal covariance matrix, for each new *i*th sample as they are calculated while doing an MCMC. The output file will be appended with each new set of objective function values as it is calculated by CASAL2. This report will print out in run mode `-m`.

8.20. Print time varying parameters

This report prints all `@time_varying` blocks with the values and years in which they were specified. This report will print out in run modes `-r, -e, -m`.

```
@report time_varying_parameters
type time_varying
```

8.21. Tabular reporting format

An alternative reporting framework to the standard output is the tabular reporting format. Tabular reporting is used with multi-line `-i` input files (like the MCMC sample or `-o` outputs). Tabular reports will print out a row that will correspond with each row of the `-i` input files.

Tabular reporting is specified using the `--tabular` argument (`casal2 -r --tabular -i file_name`).

Derived quantities, processes, observations, and `estimate_values` are the only report types that can be output with this format. For each input file the output will begin with the names of each column followed by a multi-line report ending with the `*end` syntax.

These tables can be read with **R** using the `CASAL2` package. An example usage is reading in files of MCMC posterior values of derived quantities, which can then be plotted. This command is the same as running `casal -v` in CASAL.

9. Population command and subcommand syntax

In the syntax sections, the sub-section headers use a notation like "**@observation[label].type=abundance**" which, in this case, represents the input command fragment

```
@observation label  #where label is a generic placeholder
type=abundance
...
```

and the sub-section will describe all of its own specific sub-commands. Common sub-commands are described after the command header.

9.1. Model structure

`@model` *`label`* Define an object of type *model*

`start_year` The first year of the model, immediately following initialisation
Type: non-negative integer
Default: No Default
Value: R,Defines the first year of the model, ≥ 1, e.g. 1990,

`final_year` The final year of the model, excluding years in the projection period
Type: non-negative integer
Default: No Default
Value: Defines the last year of the model, i.e., the model is run from start_year to final_year

`min_age` The minimum age of individuals in the population
Type: non-negative integer
Default: 0
Value: R,$0 \leq \text{age}_{\text{min}} \leq \text{age}_{\text{max}}$,

`max_age` The maximum age of individuals in the population
Type: non-negative integer
Default: 0
Value: R,$0 \leq \text{age}_{\text{min}} \leq \text{age}_{\text{max}}$,

`age_plus` The oldest age or extra length midpoint ,plus group size, as a plus group
Type: boolean
Default: true
Value: true, false

`initialisation_phases` The labels of the phases of the initialisation
Type: string vector
Default: true
Value: R,A list of valid labels defined by `@initialisation_phase`,

`time_steps` The labels of the time steps, in the order that they are applied, to form the annual

cycle
Type: string vector
Default: No Default
Value: R,A list of valid labels defined by `@time_step`,

`projection_final_year` The final year of the model in projection mode
Type: non-negative integer
Default: 0
Value: R,Defines the last year of the projection period, i.e., the projection period runs from `final_year`+1 to `projection_final_year`. For the default, 0, no projections are run.,

`type` The type of model ,the partition structure,: age, length, or hybrid
Type: string
Default: age
Allowed Values: age, length, hybrid

`length_bins` The minimum length in each length bin
Type: constant vector
Default: true
Value: R,$0 \leq \text{length}_{\text{min}} \leq \text{length}_{\text{max}}$,

`length_plus` Is there a length plus group or not?
Type: boolean
Default: true
Value: true, false

`length_plus_group` The mean length of length plus group
Type: constant
Default: 0
Value: R,$\text{length}_{\text{max}}$, length_plus_group,

`base_weight_units` The units for the base weight. This will be the default unit of any weight input parameters
Type: string
Default: tonnes
Value: grams, kgs or tonnes
Allowed Values: grams, tonnes, kgs

9.2. Initialisation

`@initialisation_phase` *`label`* Define an object of type *initialisation_phase*

`label` The label of the initialisation phase
Type: string
Default: No Default

`type` The type of initialisation
Type: string
Default: iterative

9.2.1. @initialisation__phase[label].type=cinitial

`categories` The list of categories for the Cinitial initialisation
Type: string vector
Default: No Default

9.2.2. @initialisation__phase[label].type=derived

`insert_processes` Additional processes not defined in the annual cycle that are to be inserted into this initialisation phase
Type: string vector
Default: true

`exclude_processes` Processes in the annual cycle to be excluded from this initialisation phase
Type: string vector
Default: true

`casal_initialisation_switch` Run an extra annual cycle to evalaute equilibrium SSBs. Warning - if true, this may not correctly evaluate the equilibrium state. Set to true if replicating a CASAL model
Type: boolean
Default: false

9.2.3. @initialisation__phase[label].type=iterative

`years` The number of iterations ,years, over which to execute this initialisation phase
Type: non-negative integer
Default: No Default

`insert_processes` The processes in the annual cycle to be included in this initialisation phase
Type: string vector
Default: true

`exclude_processes` The processes in the annual cycle to be excluded from this initialisation phase
Type: string vector
Default: true

`convergence_years` The iteration ,year, when the test for converegence ,lambda, is evaluated

Type: non-negative integer vector
Default: true

`lambda` The maximum value of the absolute sum of differences ,lambda, between the partition at year-1 and year that indicates successful convergence
Type: constant
Default: 0.0

9.2.4. `@initialisation__phase[label].type=state_category_by_age`

`categories` The list of categories for the category state initialisation
Type: string vector
Default: No Default

`min_age` The minimum age of values supplied in the definition of the category state
Type: non-negative integer
Default: No Default

`max_age` The maximum age of values supplied in the definition of the category state
Type: non-negative integer
Default: No Default

9.3. Categories

`@categories` *`label`* Define an object of type *categories*

`format` The format that the category names use
Type: string
Default: No Default

`names` The names of the categories to be used in the model
Type: string vector
Default: No Default

`years` The years that individual categories will be active for. This overrides the model values
Type: string vector
Default: true

`age_lengths` R,The labels of age_length objects that are assigned to categories,
Type: string vector
Default: true

`length_weight` R,The labels of the length_weight objects that are assigned to categories,
Type: string vector
Default: true

`age_weight` R,The labels of the age_weight objects that are assigned to categories,
Type: string vector
Default: true

9.4. Time-steps

`@time_step` *`label`* Define an object of type *time_step*

`label` The label of the timestep
Type: string
Default: No Default

`processes` The labels of the processes for this time step in the order that they occur
Type: string vector
Default: No Default

9.5. Processes

`@process` *`label`* Define an object of type *process*

`label` The label of the process
Type: string
Default: No Default

`type` The type of process
Type: string
Default: ""

9.5.1. `@process[label].type=ageing`

`categories` The labels of the categories
Type: string vector
Default: No Default

9.5.2. `@process[label].type=growth_basic`

`categories` The labels of the categories
Type: string vector
Default: No Default

`number_of_growth_episodes` The number of growth episodes per year
Type: non-negative integer
Default: No Default

`growth_time_steps` The time step in which each growth episode occurs
Type: string vector
Default: No Default

`cv` The CV for the growth model
Type: constant
Default: 0.0
Lower Bound: 0.0 (inclusive)

`sigma_min` The lower bound on sigma for the growth model
Type: constant
Default: 0.0
Lower Bound: 0.0 (inclusive)

9.5.3. `@process[label].type=maturation`

`from` The list of categories to mature from
Type: string vector
Default: No Default

`to` The list of categories to mature to
Type: string vector
Default: No Default

`selectivities` The list of selectivities to use for maturation
Type: string vector
Default: No Default

`years` The years to be associated with the maturity rates
Type: non-negative integer vector
Default: No Default

`rates` The rates to mature for each year
Type: estimable vector
Default: No Default

9.5.4. `@process[label].type=mortality_constant_rate`

`categories` The list of categories labels
Type: string vector
Default: No Default

`m` The mortality rates

Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`time_step_ratio` The time step ratios for the mortality rates
Type: constant vector
Default: true
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

9.5.5. @process[label].type=mortality_event

`categories` The categories
Type: string vector
Default: No Default

`years` The years in which to apply the mortality process
Type: non-negative integer vector
Default: No Default

`catches` The number of removals ,catches, to apply for each year
Type: estimable vector
Default: No Default

`u_max` The maximum exploitation rate ,U_{max},
Type: estimable
Default: 0.99
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`selectivities` The list of selectivities
Type: string vector
Default: No Default

`penalty` The label of the penalty to apply if the total number of removals cannot be taken
Type: string
Default: ""

9.5.6. @process[label].type=mortality_event_biomass

`categories` The category labels
Type: string vector
Default: No Default

`selectivities` The labels of the selectivities for each of the categories
Type: string vector
Default: No Default

`years` The years in which to apply the mortality process
Type: non-negative integer vector
Default: No Default

`catches` The biomass of removals ,catches, to apply for each year
Type: estimable vector
Default: No Default

`u_max` The maximum exploitation rate ,U_{max},
Type: estimable
Default: 0.99
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`penalty` The label of the penalty to apply if the total biomass of removals cannot be taken
Type: string
Default: ""

9.5.7. `@process[label].type=mortality_holling_rate`

`prey_categories` The prey categories labels
Type: string vector
Default: No Default

`predator_categories` The predator categories labels
Type: string vector
Default: No Default

`is_abundance` Is vulnerable amount of prey and predator an abundance [true] or biomass [false]
Type: boolean
Default: true

`a` Parameter a
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`b` Parameter b

Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`x` This parameter controls the functional form: Holling function type 2 ,x=2, or 3 ,x=3, or generalised ,Michaelis Menten, x,=1,
Type: estimable
Default: No Default
Lower Bound: 1.0 (inclusive)

`u_max` The maximum exploitation rate ,U_{max},
Type: constant
Default: 0.99
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`prey_selectivities` The selectivities for prey categories
Type: string vector
Default: true

`predator_selectivities` The selectivities for predator categories
Type: string vector
Default: true

`penalty` The label of penalty
Type: string
Default: ""

`years` The years in which to apply the mortality process
Type: non-negative integer vector
Default: No Default

9.5.8. `@process[label].type=mortality_initialisation_event`

`categories` The categories
Type: string vector
Default: No Default

`catch` The number of removals ,catches, to apply for each year
Type: estimable
Default: No Default

`u_max` The maximum exploitation rate ,U_{max},

Type: estimable
Default: 0.99
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`selectivities` The list of selectivities
Type: string vector
Default: No Default

`penalty` The label of the penalty to apply if the total number of removals cannot be taken
Type: string
Default: ""

9.5.9. `@process[label].type=mortality_initialisation_event_biomass`

`categories` The categories
Type: string vector
Default: No Default

`catch` The number of removals ,catches, to apply for each year
Type: estimable
Default: No Default

`u_max` The maximum exploitation rate ,U_{max},
Type: estimable
Default: 0.99
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`selectivities` The list of selectivities
Type: string vector
Default: No Default

`penalty` The label of the penalty to apply if the total number of removals cannot be taken
Type: string
Default: ""

9.5.10. `@process[label].type=mortality_instantaneous`

`categories` The categories for instantaneous mortality
Type: string vector
Default: No Default

`m` The natural mortality rates for each category

Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`time_step_ratio` The time step ratios for natural mortality
Type: constant vector
Default: true
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`relative_m_by_age` The M-by-age ogives to apply on the categories for natural mortality
Type: string vector
Default: No Default

9.5.11. `@process[label].type=mortality_instantaneous_retained`

`categories` The categories for instantaneous mortality
Type: string vector
Default: No Default

`m` The natural mortality rates for each category
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`time_step_ratio` The time step ratios for natural mortality
Type: constant vector
Default: true
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`relative_m_by_age` The M-by-age ogives to apply on the categories for natural mortality
Type: string vector
Default: No Default

9.5.12. `@process[label].type=mortality_prey_suitability`

`prey_categories` The prey categories labels
Type: string vector
Default: No Default

`predator_categories` The predator categories labels
Type: string vector
Default: No Default

`consumption_rate` The predator consumption rate
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`electivities` The prey electivities
Type: estimable vector
Default: No Default
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`u_max` The maximum exploitation rate ,U_{max},
Type: constant
Default: 0.99
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`prey_selectivities` The selectivities for prey categories
Type: string vector
Default: No Default

`predator_selectivities` The selectivities for predator categories
Type: string vector
Default: No Default

`penalty` The label of the penalty
Type: string
Default: ""

`years` The year that process occurs
Type: non-negative integer vector
Default: No Default

9.5.13. `@process[label].type=recruitment_beverton_holt`

`categories` The category labels
Type: string vector
Default: No Default

`r0` R0
Type: estimable
Default: false
Lower Bound: 0.0 (inclusive)

`b0` B0
Type: estimable
Default: false
Lower Bound: 0.0 (inclusive)

`proportions` The proportion for each category
Type: estimable
Default: No Default

`age` The age at recruitment
Type: non-negative integer
Default: true

`ssb_offset` The spawning biomass year offset
Type: non-negative integer
Default: true

`steepness` Steepness ,h,
Type: estimable
Default: 1.0
Lower Bound: 0.2 (inclusive)
Upper Bound: 1.0 (inclusive)

`ssb` The SSB label ,derived quantity,
Type: string
Default: No Default

`b0_initialisation_phase` The initialisation phase label that B0 is from
Type: string
Default: ""

`ycs_values` The YCS values
Type: estimable vector
Default: No Default

`ycs_years` The recruitment years. A vector of years that relates to the year of the spawning event that created this cohort
Type: non-negative integer vector
Default: false

`standardise_ycs_years` The years that are included for year class standardisation
Type: non-negative integer vector
Default: true

9.5.14. @process[label].type=recruitment_beverton_holt_with_deviations

`categories` The category labels
Type: string vector
Default: No Default

`r0` R0
Type: estimable
Default: false
Lower Bound: 0.0 (inclusive)

`b0` B0
Type: estimable
Default: false
Lower Bound: 0.0 (inclusive)

`proportions` The proportion for each category
Type: estimable
Default: No Default

`age` The age at recruitment
Type: non-negative integer
Default: true

`ssb_offset` The spawning biomass year offset
Type: non-negative integer
Default: true

`steepness` Steepness ,h,
Type: estimable
Default: 1.0
Lower Bound: 0.2 (inclusive)

`ssb` The SSB Label ,derived quantity,
Type: string
Default: No Default

`sigma_r` sigma R
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`b_max` The maximum bias adjustment

Type: estimable
Default: 0.85
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`last_year_with_no_bias` The last year with no bias adjustment
Type: non-negative integer
Default: false

`first_year_with_bias` The first year with full bias adjustment
Type: non-negative integer
Default: false

`last_year_with_bias` The last year with full bias adjustment
Type: non-negative integer
Default: false

`first_recent_year_with_no_bias` The first recent year with no bias adjustment
Type: non-negative integer
Default: false

`b0_initialisation_phase` The initialisation phase label that B0 is from
Type: string
Default: ""

`deviation_values` The recruitment deviation values
Type: estimable vector
Default: No Default

`deviation_years` The recruitment years. A vector of years that relates to the year of the spawning event that created this cohort
Type: non-negative integer vector
Default: false

9.5.15. `@process[label].type=recruitment_constant`

`categories` The categories
Type: string vector
Default: No Default

`proportions` The proportions
Type: estimable
Default: true

`length_bins` The length bins that recruits are uniformly distributed over at the time of

recruitment
Type: constant vector
Default: No Default

`r0` R0
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

9.5.16. `@process[label].type=survival_constant_rate`

`categories` The list of categories
Type: string vector
Default: No Default

`s` The survival rates
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`time_step_ratio` The time step ratios for S
Type: constant vector
Default: true
Lower Bound: 0.0 (exclusive)
Upper Bound: 1.0 (inclusive)

`selectivities` The selectivity label
Type: string vector
Default: No Default

9.5.17. `@process[label].type=tag_by_age`

`from` The categories to transition from
Type: string vector
Default: No Default

`to` The categories to transition to
Type: string vector
Default: No Default

`min_age` The minimum age to transition
Type: non-negative integer
Default: No Default

`max_age` The maximum age to transition
Type: non-negative integer
Default: No Default

`penalty` The penalty label
Type: string
Default: ""

`u_max` The maximum exploitation rate ,U_{max},
Type: constant
Default: 0.99
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`years` The years to execute the transition in
Type: non-negative integer vector
Default: No Default

`initial_mortality` The initial mortality value
Type: constant
Default: 0.0
Lower Bound: 0.0 (inclusive)

`initial_mortality_selectivity` The initial mortality selectivity label
Type: string
Default: ""

`loss_rate` The loss rate
Type: constant vector
Default: No Default

`loss_rate_selectivities` The loss rate selectivity label
Type: string vector
Default: true

`selectivities` The selectivity labels
Type: string vector
Default: No Default

`n` N
Type: constant vector
Default: true

9.5.18. @process[label].type=tag_by_length

`from` The categories to transition from
Type: string vector
Default: No Default

`to` The categories to transition to
Type: string vector
Default: No Default

`penalty` The penalty label
Type: string
Default: ""

`u_max` The maximum exploitation rate ,U_{max},
Type: constant
Default: 0.99
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`initial_mortality`
Type: constant
Default: 0.0
Lower Bound: 0.0 (inclusive)

`initial_mortality_selectivity`
Type: string
Default: ""

`selectivities`
Type: string vector
Default: No Default

`n`
Type: constant vector
Default: true

9.5.19. @process[label].type=tag_loss

`categories` The list of categories
Type: string vector
Default: No Default

`tag_loss_rate` The tag loss rates

Type: constant vector
Default: No Default
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`time_step_ratio` The time step ratios for tag loss
Type: constant vector
Default: true
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`tag_loss_type` The type of tag loss
Type: string
Default: No Default

`selectivities` The selectivities
Type: string vector
Default: No Default

`year` The year the first tagging release process was executed
Type: non-negative integer
Default: No Default

9.5.20. `@process[label].type=transition_category`

`from` The from category
Type: string vector
Default: No Default

`to` The to category
Type: string vector
Default: No Default

`proportions` The proportions
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`selectivities` The selectivity names
Type: string vector
Default: No Default

9.5.21. `@process[label].type=transition_category_by_age`

`from` The categories to transition from
Type: string vector
Default: No Default

`to` The categories to transition to
Type: string vector
Default: No Default

`min_age` The minimum age to transition
Type: non-negative integer
Default: No Default

`max_age` The maximum age to transition
Type: non-negative integer
Default: No Default

`penalty` The penalty label
Type: string
Default: ""

`u_max` The maximum exploitation rate ,U_{max},
Type: constant
Default: 0.99
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`years` The years to execute the transition in
Type: non-negative integer vector
Default: No Default

9.6. Time varying parameters

`@time_varying` *`label`* Define an object of type *time_varying*

`label` The time-varying label
Type: string
Default: No Default

`type` The time-varying type
Type: string
Default: ""

`years` The years in which to vary the values

Type: non-negative integer vector
Default: No Default

`parameter` The name of the parameter to time vary
Type: string
Default: No Default

9.6.1. @time__varying[label].type=annual_shift

`values` The values
Type: constant vector
Default: No Default

`a` Parameter A
Type: constant
Default: No Default

`b` Parmeter B
Type: constant
Default: No Default

`c` Parameter C
Type: constant
Default: No Default

`scaling_years` The scaling years
Type: non-negative integer vector
Default: true

9.6.2. @time__varying[label].type=constant

`values` The value to assign to addressable
Type: estimable vector
Default: No Default

9.6.3. @time__varying[label].type=exogenous

`a` The shift parameter
Type: estimable
Default: No Default

`exogeneous_variable` The values of exogeneous variable for each year

Type: constant vector
Default: No Default

9.6.4. @time__varying[label].type=linear

`slope` The slope of the linear trend ,additive unit per year,
Type: estimable
Default: No Default

`intercept` The intercept of the linear trend value for the first year
Type: estimable
Default: No Default

9.6.5. @time__varying[label].type=random_draw

`mean` The mean ,mu,
Type: estimable
Default: 0

`sigma` The standard deviation ,sigma,
Type: estimable
Default: 1

`distribution` The distribution
Type: string
Default: normal
Allowed Values: normal, lognormal

9.6.6. @time__varying[label].type=random_walk

`mean` The mean ,mu,
Type: estimable
Default: 0

`sigma` The standard deviation ,sigma,
Type: estimable
Default: 1

`upper_bound` The upper bound for the random walk
Type: constant
Default: 1

`upper_bound` The lower bound for the random walk

Type: constant
Default: 1

`rho` The autocorrelation parameter ,rho,
Type: constant
Default: 1

`distribution` The distribution
Type: string
Default: normal

9.7. Derived quantities

`@derived_quantity` *`label`* Define an object of type *derived_quantity*

`label` The label of the derived quantity
Type: string
Default: No Default

`type` The type of derived quantity
Type: string
Default: No Default

`time_step` The time step in which to calculate the derived quantity after
Type: string
Default: No Default

`categories` The list of categories to use when calculating the derived quantity
Type: string vector
Default: No Default

`selectivities` A list of one selectivity
Type: string vector
Default: No Default

`time_step_proportion` The proportion through the mortality block of the time step when calculated
Type: constant
Default: 0.5
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`time_step_proportion_method` The method for interpolating for the proportion through the

mortality block
Type: string
Default: weighted_sum
Allowed Values: weighted_sum, weighted_product

`values`
Type: Addressable vector
Default: No Default

9.7.1. `@derived__quantity[label].type=abundance`

9.7.2. `@derived__quantity[label].type=biomass`

9.8. Age-length relationship

`@age_length` *`label`* Define an object of type *age_length*

`label` The label of the age length relationship
Type: string
Default: No Default

`type` The type of age length relationship
Type: string
Default: No Default

`time_step_proportions` The fraction of the year applied in each time step that is added to the age for the purposes of evaluating the length, i.e., a value of 0.5 for a time step will evaluate the length of individuals at age+0.5 in that time step
Type: constant vector
Default: true
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`distribution` The assumed distribution for the growth curve
Type: string
Default: normal

`cv_first` The CV for the first age class
Type: estimable
Default: 0.0
Lower Bound: 0.0 (inclusive)

`cv_last` The CV for last age class
Type: estimable
Default: 0.0
Lower Bound: 0.0 (inclusive)

`casal_switch` If true, use the ,less accurate, equation for the cumulative normal function as was used in the legacy version of CASAL.
Type: boolean
Default: false

`by_length` Specifies if the linear interpolation of CVs is a linear function of mean length at age. Default is by age only
Type: boolean
Default: true

9.8.1. @age__length[label].type=data

`external_gaps` The method to use for external data gaps
Type: string
Default: mean
Allowed Values: mean, nearest_neighbour

`internal_gaps` The method to use for internal data gaps
Type: string
Default: mean
Allowed Values: mean, nearest_neighbour, interpolate

`length_weight` The label from an associated length-weight block
Type: string
Default: No Default

`time_step_measurements_were_made` The time step label for which size-at-age data are provided
Type: string
Default: No Default

9.8.2. @age__length[label].type=none

9.8.3. @age__length[label].type=schnute

`y1` The y_1 parameter
Type: estimable
Default: No Default

`y2` The y_2 parameter
Type: estimable
Default: No Default

`tau1` The τ_1 parameter

Type: estimable
Default: No Default

`tau2` The τ_2 parameter
Type: estimable
Default: No Default

`a` The a parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`b` The b parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`length_weight` The label of the associated length-weight relationship
Type: string
Default: No Default

9.8.4. `@age__length[label].type=von_bertalanffy`

`linf` The $L_{infinity}$ parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`k` The k parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`t0` The t_0 parameter
Type: estimable
Default: No Default

`length_weight` The label of the associated length-weight relationship
Type: string
Default: No Default

9.9. Length-weight

`@length_weight` *`label`* Define an object of type *length_weight*

`label` The label of the length-weight relationship
Type: string
Default: No Default

`type` The type of the length-weight relationship
Type: string
Default: No Default

9.9.1. `@length__weight[label].type=basic`

`a` The a parameter ,$W = aL^b$,
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`b` The b parameter ,$W = aL^b$,
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`units` The units of measure ,tonnes, kgs, grams,
Type: string
Default: No Default
Allowed Values: tonnes, kgs, grams

9.9.2. `@length__weight[label].type=none`

9.10. Selectivities

`@selectivity` *`label`* Define an object of type *selectivity*

`label` The label for this selectivity
Type: string
Default: No Default

`type` The type of selectivity
Type: string
Default: No Default

`length_based` Is the selectivity length based?
Type: boolean
Default: false

`intervals` The number of quantiles to evaluate a length-based selectivity over the age-length

distribution
Type: non-negative integer
Default: 5

`partition_type` The type of partition that this selectivity will support. Defaults to the same as the model
Type: string
Default: model
Allowed Values: model, age, length, hybrid

`values`
Type: Addressable vector
Default: No Default

`length_values`
Type: Addressable vector
Default: No Default

9.10.1. `@selectivity[label].type=all_values`

`v` The v parameter
Type: estimable vector
Default: No Default

9.10.2. `@selectivity[label].type=all_values_bounded`

`l` The low value ,L,
Type: non-negative integer
Default: No Default

`h` The high value ,H,
Type: non-negative integer
Default: No Default

`v` The v parameter
Type: estimable vector
Default: No Default

9.10.3. `@selectivity[label].type=constant`

`c` The constant value
Type: estimable
Default: No Default

9.10.4. @selectivity[label].type=double_exponential

`x0` The X0 parameter
Type: estimable
Default: No Default

`x1` The X1 parameter
Type: constant
Default: No Default

`x2` The X2 parameter
Type: constant
Default: No Default

`y0` The Y0 parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`y1` The Y1 parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`y2` The Y2 parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

`alpha` alpha
Type: estimable
Default: 1.0
Lower Bound: 0.0 (exclusive)

9.10.5. @selectivity[label].type=double_normal

`mu` The mean ,mu,
Type: estimable
Default: No Default

`sigma_l` The sigma L parameter

Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`sigma_r` The sigma R parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`alpha` alpha
Type: estimable
Default: 1.0
Lower Bound: 0.0 (exclusive)

9.10.6. `@selectivity[label].type=increasing`

`l` The low value ,L,
Type: non-negative integer
Default: No Default

`h` The high value ,H,
Type: non-negative integer
Default: No Default

`v` The v parameter
Type: estimable vector
Default: No Default

`alpha` alpha
Type: constant
Default: 1.0
Lower Bound: 0.0 (exclusive)

9.10.7. `@selectivity[label].type=inverse_logistic`

`a50` a50
Type: estimable
Default: No Default

`ato95` ato95
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`alpha` alpha
Type: estimable
Default: 1.0
Lower Bound: 0.0 (exclusive)

9.10.8. `@selectivity[label].type=knife_edge`

`e` The edge value
Type: estimable
Default: No Default

`alpha` alpha
Type: estimable
Default: 1.0

9.10.9. `@selectivity[label].type=logistic`

`a50` a50
Type: estimable
Default: No Default

`ato95` ato95
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`alpha` alpha
Type: estimable
Default: 1.0
Lower Bound: 0.0 (exclusive)

9.10.10. `@selectivity[label].type=logistic_producing`

`l` The low value ,L,
Type: non-negative integer
Default: No Default

`h` The high value ,H,
Type: non-negative integer
Default: No Default

`a50` a50

Type: estimable
Default: No Default

`ato95` ato95
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`alpha` alpha
Type: estimable
Default: 1.0
Lower Bound: 0.0 (exclusive)

9.11. Projections

`@project` *`label`* Define an object of type *project*

`label` The projection label
Type: string
Default: No Default

`type` The projection type
Type: string
Default: ""

`years` The years to recalculate the values
Type: non-negative integer vector
Default: true

`parameter` The parameter to project
Type: string
Default: No Default

`multiplier` The multiplier applied to the projected value
Type: constant
Default: 1.0
Lower Bound: 0.0 (exclusive)

9.11.1. `@project[label].type=constant`

`values` The values to assign to the addressable
Type: constant vector
Default: No Default

9.11.2. @project[label].type=empirical_sampling

`start_year` The start year of sampling
Type: non-negative integer
Default: false

`final_year` The final year of sampling
Type: non-negative integer
Default: false

9.11.3. @project[label].type=log_normal

`mean` The mean of the lognormal process
Type: constant
Default: 0.0

`sigma` The standard deviation ,sigma, of the lognormal process
Type: constant
Default: No Default
Lower Bound: 0.0 (inclusive)

9.11.4. @project[label].type=log_normal_empirical

`mean` The mean of the Gaussian process
Type: constant
Default: 0.0

`start_year` The start year of sampling
Type: non-negative integer
Default: false

`final_year` The final year of sampling
Type: non-negative integer
Default: false

9.11.5. @project[label].type=user_defined

`equation` The equation to do a test run of
Type: string vector
Default: No Default

9.12. Time-varying Parameters

`@time_varying` *`label`* Define an object of type *time_varying*

`label` The time-varying label
Type: string
Default: No Default

`type` The time-varying type
Type: string
Default: ""

`years` The years in which to vary the values
Type: non-negative integer vector
Default: No Default

`parameter` The name of the parameter to time vary
Type: string
Default: No Default

9.12.1. `@time__varying[label].type=annual_shift`

`values` The values
Type: constant vector
Default: No Default

`a` Parameter A
Type: constant
Default: No Default

`b` Parmeter B
Type: constant
Default: No Default

`c` Parameter C
Type: constant
Default: No Default

`scaling_years` The scaling years
Type: non-negative integer vector
Default: true

9.12.2. `@time__varying[label].type=constant`

`values` The value to assign to addressable
Type: estimable vector
Default: No Default

9.12.3. `@time__varying[label].type=exogenous`

`a` The shift parameter
Type: estimable
Default: No Default

`exogeneous_variable` The values of exogeneous variable for each year
Type: constant vector
Default: No Default

9.12.4. `@time__varying[label].type=linear`

`slope` The slope of the linear trend ,additive unit per year,
Type: estimable
Default: No Default

`intercept` The intercept of the linear trend value for the first year
Type: estimable
Default: No Default

9.12.5. `@time__varying[label].type=random_draw`

`mean` The mean ,mu,
Type: estimable
Default: 0

`sigma` The standard deviation ,sigma,
Type: estimable
Default: 1

`distribution` The distribution
Type: string
Default: normal
Allowed Values: normal, lognormal

9.12.6. `@time__varying[label].type=random_walk`

`mean` The mean ,mu,
Type: estimable
Default: 0

`sigma` The standard deviation ,sigma,
Type: estimable
Default: 1

`upper_bound` The upper bound for the random walk
Type: constant
Default: 1

`upper_bound` The lower bound for the random walk
Type: constant
Default: 1

`rho` The autocorrelation parameter ,rho,
Type: constant
Default: 1

`distribution` The distribution
Type: string
Default: normal

10. Estimation command and subcommand syntax

In the syntax sections, the sub-section headers use a notation like "**@observation[label].type=abundance**" which, in this case, represents the input command fragment

```
@observation label  #where label is a generic placeholder
type=abundance
...
```

and the sub-section will describe all of its own specific sub-commands. Common sub-commands are described after the command header.

10.1. Estimation methods

`@estimate` *`label`* Define an object of type *estimate*

`label` The label of the estimate
Type: string
Default: ""

`type` The prior type for the estimate
Type: string
Default: No Default

`parameter` The name of the parameter to estimate in the model
Type: string
Default: No Default

`lower_bound` The lower bound for the parameter
Type: constant
Default: No Default

`upper_bound` The upper bound for the parameter
Type: constant
Default: No Default

`same` List of parameters that are constrained to have the same value as this parameter
Type: string vector
Default: ""

`estimation_phase` The first estimation phase to allow this to be estimated
Type: non-negative integer
Default: 1

`mcmc` Indicates if this parameter is estimated at the point estimate but fixed during MCMC estimation run
Type: boolean
Default: false

`transformation` Type of simple transformation to apply to estimate
Type: string
Default: ""

`transform_with_jacobian` Apply jacobian during transformation
Type: boolean
Default: false

`prior_applies_to_transform` Does the prior apply to the transformed parameter? a legacy switch, see Manual for more information
Type: boolean
Default: false

10.1.1. `@estimate[label].type=beta`

`mu` Beta prior mean ,mu, parameter
Type: estimable
Default: No Default

`sigma` Beta prior variance ,sigma, parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`a` Beta prior lower bound of the range ,A, parameter
Type: constant
Default: No Default

`b` Beta prior upper bound of the range ,B, parameter
Type: constant
Default: No Default

10.1.2. `@estimate[label].type=lognormal`

`mu` The lognormal prior mean ,mu, parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`cv` The lognormal variance ,cv, parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

10.1.3. `@estimate[label].type=normal`

`mu` The normal prior mean ,mu, parameter
Type: estimable
Default: No Default

`cv` The normal variance ,standard devation, parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

10.1.4. `@estimate[label].type=normal_by_stdev`

`mu` The normal prior mean ,mu, parameter
Type: estimable
Default: No Default

`sigma` The normal variance ,sigma, parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`lognormal_transfomration` Add a Jacobian if the derived outcome of the estimate is assumed to be lognormal, e.g., used for rectuitment deviations in the recruitment process. See the User Manual for more information
Type: boolean
Default: false

10.1.5. `@estimate[label].type=normal_log`

`mu` The normal-log prior mean ,mu, parameter
Type: estimable
Default: No Default

`sigma` The normal-log prior variance ,standard deviation, parameter
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

10.1.6. `@estimate[label].type=uniform`

10.1.7. `@estimate[label].type=uniform_log`

10.2. Point estimation

`@minimiser` *`label`* Define an object of type *minimiser*

`label` The minimiser label
Type: string
Default: No Default

`type` The type of minimiser to use
Type: string
Default: No Default

`active` Indicates if this minimiser is active

Type: boolean
Default: false

`covariance` Indicates if a covariance matrix should be generated
Type: boolean
Default: true

10.2.1. `@minimiser[label].type=adolc`

`iterations` The maximum number of iterations
Type: integer
Default: 1000
Lower Bound: 1 (inclusive)

`evaluations` The maximum number of evaluations
Type: integer
Default: 4000
Lower Bound: 1 (inclusive)

`tolerance` The tolerance of the gradient for convergence
Type: constant
Default: 0.02
Lower Bound: 0.0 (exclusive)

`step_size` The minimum step size before minimisation fails
Type: constant
Default: 1e-7
Lower Bound: 0.0 (exclusive)

10.2.2. `@minimiser[label].type=betadiff`

`iterations` The maximum number of iterations
Type: integer
Default: 1000
Lower Bound: 1 (inclusive)

`evaluations` The maximum number of evaluations
Type: integer
Default: 4000
Lower Bound: 1 (inclusive)

`tolerance` The tolerance of the gradient for convergence
Type: constant
Default: 2e-3
Lower Bound: 0.0 (exclusive)

10.2.3. `@minimiser[label].type=cppad`

`retape` Retape?
Type: string
Default: "true"
Allowed Values: true, false

`print_level` The level of debug to stdout
Type: non-negative integer
Default: 5u
Lower Bound: 0u (inclusive)
Upper Bound: 12u (inclusive)

`sb` String buffer output?
Type: string
Default: "yes"
Allowed Values: yes, no

`pidi` Print iteration diagnostic information?
Type: string
Default: "yes"
Allowed Values: yes, no

`iterations` The maximum number of iterations
Type: non-negative integer
Default: 100
Lower Bound: 1u (inclusive)

`tolerance` The tolerance for convergence
Type: constant
Default: 1e-9
Lower Bound: 0.0 (exclusive)

`acceptable_tol` The acceptable tolerance
Type: constant
Default: 1e-6
Lower Bound: 0.0 (exclusive)

`acceptable_obj_change_tol`
Type: constant
Default: 1e+20
Lower Bound: 0.0 (exclusive)

`derivative_test` How to test for derivatives

Type: string
Default: "first-order"
Allowed Values: none, first-order, second-order, only-second-order

`point_perturbation_radius` The point perturbation radius
Type: constant
Default: 0.0
Lower Bound: 0.0 (inclusive)

10.2.4. @minimiser[label].type=de_solver

`population_size` The number of candidate solutions to have in the population
Type: non-negative integer
Default: No Default

`crossover_probability` The minimiser's crossover probability
Type: constant
Default: 0.9
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`difference_scale` The scale to apply to new solutions when comparing candidates
Type: constant
Default: 0.02

`max_generations` The maximum number of iterations to run
Type: non-negative integer
Default: No Default

`tolerance` The total variance between the population and best candidate before acceptance
Type: constant
Default: 0.01
Lower Bound: 0.0 (exclusive)

`method` The type of candidate generation method to use
Type: string
Default: ""
Value: not_yet_implemented

10.2.5. @minimiser[label].type=d_lib

`minimisation_type` The type of minimisation to use
Type: string
Default: min_using_approximate_derivatives
Allowed Values: min_using_approximate_derivatives, minimisation, min_box_constrained,

min_trust_region, min_bobyqa, min_global

`search_strategy` The type of search strategy to use
Type: string
Default: bfgs
Allowed Values: bfgs, lbfgs, cg, newton

`tolerance` The tolerance of the gradient for convergence
Type: constant
Default: 1e-7
Lower Bound: 0.0 (exclusive)

`lbfgs_max_size` The maximum size for the LBFGS search strategy
Type: non-negative integer
Default: 1

`bobyqa_interpolation_points` BOBYQA interpolation points
Type: non-negative integer
Default: 5u

`bobyqa_initial_trust_radius` BOBYQA initial trust radius
Type: constant
Default: 1e-2

`bobyqa_stopping_trust_radius` BOBYQA stopping trust radius
Type: constant
Default: 1e-6

`bobyqa_max_evaluations` BOBYQA max objective evaluations
Type: constant
Default: 4000
Lower Bound: 1 (inclusive)

`verbose` Print debug of objective function calls?
Type: boolean
Default: false

10.2.6. `@minimiser[label].type=numerical_differences`

`iterations` The maximum number of iterations
Type: integer
Default: 1000
Lower Bound: 1 (inclusive)

`evaluations` The maximum number of evaluations

Type: integer
Default: 4000
Lower Bound: 1 (inclusive)

`tolerance` The tolerance of the gradient for convergence
Type: constant
Default: 0.02
Lower Bound: 0.0 (exclusive)

`step_size` The minimum step size before minimisation fails
Type: constant
Default: 1e-7
Lower Bound: 0.0 (exclusive)

10.3. Markov chain Monte Carlo (MCMC)

`@mcmc` *`label`* Define an object of type *mcmc*

`label` The label of the MCMC
Type: string
Default: No Default

`type` The type of MCMC
Type: string
Default: ""

`length` The number of iterations in for the MCMC chain
Type: non-negative integer
Default: No Default

`active` Indicates if this is the active MCMC algorithm
Type: boolean
Default: true

`step_size` Initial stepsize ,as a multiplier of the approximate covariance matrix,
Type: constant
Default: 0.02

10.3.1. `@mcmc[label].type=independence_metropolis`

`start` The covariance multiplier for the starting point of the MCMC
Type: constant
Default: 0.0
Lower Bound: 0.0 (inclusive)

`keep` The spacing between recorded values in the MCMC
Type: non-negative integer
Default: 1u
Lower Bound: 1u (inclusive)

`max_correlation` The maximum absolute correlation in the covariance matrix of the proposal distribution
Type: constant
Default: 0.8
Lower Bound: 0.0 (exclusive)
Upper Bound: 1.0 (inclusive)

`covariance_adjustment_method` The method for adjusting small variances in the covariance proposal matrix
Type: string
Default: correlation
Allowed Values: covariance, correlation, none

`correlation_adjustment_diff` The minimum non-zero variance times the range of the bounds in the covariance matrix of the proposal distribution
Type: constant
Default: 0.0001
Lower Bound: 0.0 (exclusive)

`proposal_distribution` The shape of the proposal distribution ,either the t or the normal distribution,
Type: string
Default: t

`df` The degrees of freedom of the multivariate t proposal distribution
Type: non-negative integer
Default: 4
Lower Bound: 0 (exclusive)

`adapt_stepsize_at` The iteration numbers in which to check and resize the MCMC stepsize
Type: non-negative integer vector
Default: true
Lower Bound: 0 (inclusive)

`adapt_covariance_matrix_at` The iteration numbers in which to adapt the covariance matrix
Type: non-negative integer vector
Default: true
Lower Bound: 0 (inclusive)

`adapt_stepsize_method` The method to use to adapt the step size

Type: string
Default: ratio
Allowed Values: ratio, double_half

10.4. Profiles

@profile *label* Define an object of type *profile*

`label` The label of the profile
Type: string
Default: ""

`steps` The number of steps between the lower and upper bound
Type: non-negative integer
Default: No Default

`lower_bound` The lower bounds
Type: constant
Default: No Default

`upper_bound` The upper bounds
Type: constant
Default: No Default

`parameter` The system parameter to profile
Type: string
Default: No Default

`same` A parameter that is constrained to have the same value as the parameter being profiled
Type: string
Default: ""

10.5. Defining catchability constants

@catchability *label* Define an object of type *catchability*

`label` Label of the catchability
Type: string
Default: No Default

`type` The type of catchability
Type: string
Default: No Default

10.5.1. `@catchability[label].type=free`

`q` The value of the catchability
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)

10.5.2. `@catchability[label].type=nuisance`

`lower_bound` The upper bound for nuisance catchability
Type: constant
Default: No Default

`upper_bound` The lower bound for nuisance catchability
Type: constant
Default: No Default

`q`
Type: Addressable
Default: No Default

10.6. Defining penalties

`@penalty` *`label`* Define an object of type *penalty*

`label` The label of the penalty
Type: string
Default: No Default

`type` The type of penalty
Type: string
Default: No Default

10.6.1. `@penalty[label].type=process`

`multiplier` The penalty multiplier
Type: constant
Default: 1.0
Lower Bound: 0.0 (exclusive)

`log_scale` Indicates if the sums of squares is calculated on the log scale
Type: boolean
Default: false

10.7. Defining priors on parameter ratios, differences, and means

`@additional_prior` *`label`* Define an object of type *additional_prior*

`parameter` The name of the parameter to generate additional prior on
Type: string
Default: No Default

`label` The label for the additional prior
Type: string
Default: No Default

`type` The type of additional prior
Type: string
Default: No Default

10.7.1. `@additional__prior[label].type=beta`

`mu` Beta distribution mean ,mu, parameter
Type: constant
Default: No Default

`sigma` Beta distribution variance ,sigma, parameter
Type: constant
Default: No Default
Lower Bound: 0.0 (inclusive)

`a` Beta distribution lower bound of the range ,A, parameter
Type: constant
Default: No Default

`b` Beta distribution upper bound of the range ,B, parameter
Type: constant
Default: No Default

10.7.2. `@additional__prior[label].type=element_difference`

`second_parameter` The name of the second parameter for comparing
Type: string
Default: No Default

`multiplier` Multiply the penalty by this factor
Type: constant
Default: 1

10.7.3. @additional__prior[label].type=log_normal

`mu` The lognormal prior mean ,mu, parameter
Type: constant
Default: No Default
Lower Bound: 0.0 (exclusive)

`cv` The lognormal variance ,cv, parameter
Type: constant
Default: No Default
Lower Bound: 0.0 (exclusive)

10.7.4. @additional__prior[label].type=uniform_log

10.7.5. @additional__prior[label].type=vector_average

`method` Which calculation method to use: k, l, or m
Type: string
Default: k

`k` The k value to use in the calculation
Type: constant
Default: No Default

`multiplier` The multiplier for the penalty amount
Type: constant
Default: 1

10.7.6. @additional__prior[label].type=vector_smoothing

`log_scale` Should the sums of squares be calculated on the log scale?
Type: boolean
Default: false

`multiplier` Multiply the penalty by this factor
Type: constant
Default: 1

`lower_bound` The first element to apply the penalty to in the vector
Type: non-negative integer
Default: 0u

`upper_bound` The last element to apply the penalty to in the vector
Type: non-negative integer
Default: 0u

`r` Penalty applied to rth differences
Type: non-negative integer
Default: 2u

10.8. Defining estimation of transformations

`@estimate_transformation` *`label`* Define an object of type *estimate_transformation*

`label` Label for the transformation block
Type: string
Default: No Default

`type` The type of transformation
Type: string
Default: No Default

`transform_with_jacobian` Apply Jacobian during transformation
Type: boolean
Default: true

10.8.1. `@estimate__transformation[label].type=average_difference`

`theta2` The label of the @estimate block relating to the θ_2 parameter in the transformation. See the User Manual for more information
Type: string
Default: No Default

`theta1` The label of @estimate block relating to the θ_1 parameter in the transformation. See the User Manual for more information
Type: string
Default: No Default

10.8.2. `@estimate__transformation[label].type=inverse`

`estimate_label` The label of estimate block to apply transformation. Defined as θ_1 in the documentation
Type: string
Default: No Default

10.8.3. @estimate__transformation[label].type=log

`estimate_label` Label of estimate block to apply transformation. Defined as θ_1 in the documentation
Type: string
Default: No Default

10.8.4. @estimate__transformation[label].type=log_sum

`theta2` The label of the @estimate block relating to the θ_2 parameter in the transformation. See the User Manual for more information
Type: string
Default: No Default

`theta1` The label of @estimate block relating to the θ_1 parameter in the transformation. See the User Manual for more information
Type: string
Default: No Default

10.8.5. @estimate__transformation[label].type=orthogonal

`theta2` The label of the @estimate block relating to the θ_2 parameter in the transformation. See the User Manual for more information
Type: string
Default: No Default

`theta1` The label of @estimate block relating to the θ_1 parameter in the transformation. See the User Manual for more information
Type: string
Default: No Default

10.8.6. @estimate__transformation[label].type=simplex

`lower_bound` The empirical lower bound for the simplex transformed. This should be -Inf but some of the minimisers do not allow that
Type: constant
Default: true

`upper_bound` The empirical upper bound for the simplex transformed. This should be Inf but some of the minimisers do not allow that
Type: constant
Default: true

10.8.7. `@estimate__transformation[label].type=square_root`

`estimate_label` The label of the estimate block to apply transformation. Defined as θ_1 in the documentation
Type: string
Default: No Default

10.8.8. `@estimate__transformation[label].type=sum_to_one`

`estimate_labels` The label for the estimates for the sum to one transformation
Type: string vector
Default: No Default

`upper_bound` The empirical upper bounds for the transformed parameters. There should be one less bound than parameters
Type: constant vector
Default: true

`lower_bound` The empirical lower bound for the transformed parameters. There should be one less bound than parameters
Type: constant vector
Default: true

11. Observation command and subcommand syntax

In the syntax sections, the sub-section headers use a notation like "**@observation[label].type=abundance**" which, in this case, represents the input command fragment

```
@observation label  #where label is a generic placeholder
type=abundance
...
```

and the sub-section will describe all of its own specific sub-commands. Common sub-commands are described after the command header.

11.1. Observation types

The observation types available are:

Observations of proportions of individuals by age class

Observations of proportions of individuals by category and age class

Relative and absolute abundance observations

Relative and absolute biomass observations

Each type of observation requires a set of subcommands and arguments specific to that process.

`@observation` *`label`* Define an object of type *observation*

`label` The label of the observation
Type: string
Default: No Default

`type` The type of observation
Type: string
Default: No Default

`likelihood` The type of likelihood to use
Type: string
Default: No Default

`categories` The category labels to use
Type: string vector
Default: true

`delta` The robustification value ,delta, for the likelihood
Type: constant
Default: DELTA
Lower Bound: 0.0 (inclusive)

`simulation_likelihood` The simulation likelihood to use
Type: string
Default: ""

`likelihood_multiplier` The likelihood score multiplier
Type: constant
Default: 1.0
Lower Bound: 0.0 (inclusive)

`error_value_multiplier` The error value multiplier for likelihood
Type: constant
Default: 1.0
Lower Bound: 0.0 (inclusive)

11.1.1. `@observation[label].type=abundance`

`time_step` The label of the time step that the observation occurs in
Type: string
Default: No Default

`catchability` The label of the catchability coefficient ,q,

Type: string
Default: No Default

`selectivities` The labels of the selectivities
Type: string vector
Default: true

`process_error` The process error
Type: estimable
Default: 0.0
Lower Bound: 0.0 (inclusive)

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

11.1.2. @observation[label].type=biomass

`time_step` The label of the time step that the observation occurs in
Type: string
Default: No Default

`catchability` The label of the catchability coefficient ,q,
Type: string
Default: No Default

`selectivities` The labels of the selectivities
Type: string vector
Default: true

`process_error` The process error
Type: estimable
Default: 0.0
Lower Bound: 0.0 (inclusive)

`age_weight_labels` R,The labels for the `@`*age_weight* block which corresponds to each category, to use the weight calculation method for biomass calculations,
Type: string vector
Default: ""

`years` The years of the observed values
Type: non-negative integer vector
Default: No Default

11.1.3. @observation[label].type=process_removals_by_age

`min_age` The minimum age
Type: non-negative integer
Default: No Default

`max_age` The maximum age
Type: non-negative integer
Default: No Default

`plus_group` Is the maximum age the age plus group
Type: boolean
Default: true

`time_step` The label of time-step that the observation occurs in
Type: string vector
Default: No Default

`tolerance` The tolerance
Type: constant
Default: double
Lower Bound: 0.0 (exclusive)
Upper Bound: 1.0 (exclusive)

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`process_errors` The label of process error to use
Type: estimable vector
Default: true

`ageing_error` The label of the ageing error to use
Type: string
Default: ""

`method_of_removal` The label of the observed method of removals
Type: string vector
Default: ""

`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation
Type: string
Default: No Default

11.1.4. @observation[label].type=process_removals_by_age_retained

`min_age` The minimum age
Type: non-negative integer
Default: No Default

`max_age` The maximum age
Type: non-negative integer
Default: No Default

`plus_group` Is the maximum age the age plus group?
Type: boolean
Default: true

`time_step` The label of the time step that the observation occurs in
Type: string vector
Default: No Default

`tolerance` The tolerance
Type: constant
Default: double
Lower Bound: 0.0 (exclusive)
Upper Bound: 1.0 (exclusive)

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`process_errors` The label of the process error to use
Type: estimable vector
Default: true

`ageing_error` The label of the ageing error to use
Type: string
Default: ""

`method_of_removal` The label of observed method of removals
Type: string vector
Default: ""

`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation
Type: string
Default: No Default

11.1.5. @observation[label].type=process_removals_by_age_retained_total

`min_age` The minimum age
Type: non-negative integer
Default: No Default

`max_age` The maximum age
Type: non-negative integer
Default: No Default

`plus_group` Is the maximum age the age plus group?
Type: boolean
Default: true

`time_step` The label of the time step that the observation occurs in
Type: string vector
Default: No Default

`tolerance` The tolerance
Type: constant
Default: double
Lower Bound: 0.0 (exclusive)
Upper Bound: 1.0 (exclusive)

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`process_errors` The label of the process error to use
Type: estimable vector
Default: true

`ageing_error` The label of the ageing error to use
Type: string
Default: ""

`method_of_removal` The label of observed method of removals
Type: string vector
Default: ""

`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation
Type: string
Default: No Default

11.1.6. @observation[label].type=process_removals_by_length

`time_step` The time step to execute in
Type: string
Default: No Default

`tolerance` The tolerance for rescaling proportions
Type: constant
Default: double
Lower Bound: 0.0 (exclusive)
Upper Bound: 1.0 (exclusive)

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`process_errors` The process error
Type: estimable vector
Default: true

`method_of_removal` The label of observed method of removals
Type: string
Default: ""

`length_bins` The length bins
Type: constant vector
Default: No Default

`length_plus` Is the last length bin a plus group? ,defaults to @model value,
Type: boolean
Default: model

`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation
Type: string
Default: No Default

11.1.7. @observation[label].type=process_removals_by_length_retained

`time_step` The time step to execute in
Type: string
Default: No Default

`tolerance` The tolerance for rescaling proportions

Type: constant
Default: double
Lower Bound: 0.0 (exclusive)
Upper Bound: 1.0 (exclusive)

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`process_errors` The process error
Type: estimable vector
Default: true

`method_of_removal` The label of observed method of removals
Type: string
Default: ""

`length_bins` The length bins
Type: constant vector
Default: No Default

`length_plus` Is the last length bin a plus group? ,defaults to @model value,
Type: boolean
Default: model

`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation
Type: string
Default: No Default

11.1.8. `@observation[label].type=process_removals_by_length_retained_total`

`time_step` The time step to execute in
Type: string
Default: No Default

`tolerance` The tolerance for rescaling proportions
Type: constant
Default: double
Lower Bound: 0.0 (exclusive)
Upper Bound: 1.0 (exclusive)

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`process_errors` The process error
Type: estimable vector
Default: true

`method_of_removal` The label of observed method of removals
Type: string
Default: ""

`length_bins` The length bins
Type: constant vector
Default: No Default

`length_plus` Is the last length bin a plus group? ,defaults to @model value,
Type: boolean
Default: model

`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation
Type: string
Default: No Default

11.1.9. `@observation[label].type=proportions_at_age`

`min_age` The minimum age
Type: non-negative integer
Default: No Default

`max_age` The maximum age
Type: non-negative integer
Default: No Default

`plus_group` Is the maximum age the age plus group?
Type: boolean
Default: true

`time_step` The label of the time step that the observation occurs in
Type: string
Default: No Default

`tolerance` The tolerance on the constraint that for each year the sum of proportions in each

age must equal 1, e.g., if tolerance = 0.1 then 1 - Sum,Proportions, can be as great as 0.1
Type: constant
Default: double
Lower Bound: 0.0 (exclusive)
Upper Bound: 1.0 (exclusive)

`years` The years of the observed values
Type: non-negative integer vector
Default: No Default

`selectivities` The labels of the selectivities
Type: string vector
Default: true

`process_errors` The process error
Type: constant vector
Default: true

`ageing_error` The label of ageing error to use
Type: string
Default: ""

11.1.10. `@observation[label].type=proportions_at_length`

`time_step` The label of the time step that the observation occurs in
Type: string
Default: No Default

`tolerance` The tolerance for rescaling proportions
Type: constant
Default: double
Lower Bound: 0.0 (exclusive)

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`selectivities` The labels of the selectivities
Type: string vector
Default: true

`process_errors` The process error
Type: constant vector
Default: true

`length_bins` The length bins
Type: constant vector
Default: true

11.1.11. @observation[label].type=proportions_by_category

`min_age` The minimum age
Type: non-negative integer
Default: No Default

`max_age` The maximum age
Type: non-negative integer
Default: No Default

`time_step` The label of the time step that the observation occurs in
Type: string
Default: No Default

`plus_group` Use the age plus group?
Type: boolean
Default: true

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`selectivities` The labels of the selectivities
Type: string vector
Default: true

`categories2` The target categories
Type: string vector
Default: No Default

`selectivities2` The target selectivities
Type: string vector
Default: No Default

11.1.12. @observation[label].type=proportions_mature_by_age

`min_age` The minimum age
Type: non-negative integer
Default: No Default

`max_age` The maximum age
Type: non-negative integer
Default: No Default

`time_step` The label of time-step that the observation occurs in
Type: string
Default: No Default

`plus_group` Use the age plus group?
Type: boolean
Default: true

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`ageing_error` The label of ageing error to use
Type: string
Default: ""

`total_categories` All category labels that were vulnerable to sampling at the time of this observation ,not including the categories already given,
Type: string vector
Default: true

`time_step_proportion` The proportion through the mortality block of the time step when the observation is evaluated
Type: constant
Default: double
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

11.1.13. `@observation[label].type=proportions_migrating`

`min_age` The minimum age
Type: non-negative integer
Default: No Default

`max_age` The maximum age
Type: non-negative integer
Default: No Default

`time_step` The label of the time step that the observation occurs in
Type: string
Default: No Default

`plus_group` Is the maximum age the age plus group?
Type: boolean
Default: true

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`process_errors` The process error
Type: constant vector
Default: true

`ageing_error` The label of the ageing error to use
Type: string
Default: ""

`process` The process label
Type: string
Default: No Default

11.1.14. `@observation[label].type=tag_recapture_by_age`

`min_age` The minimum age
Type: non-negative integer
Default: No Default

`max_age` The maximum age
Type: non-negative integer
Default: No Default

`plus_group` Is the maximum age the age plus group?
Type: boolean
Default: true

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`categories2` The available categories in the partition
Type: string vector
Default: No Default

`selectivities` The labels of the selectivities

Type: string vector
Default: true

`time_step` The label of the time step that the observation occurs in
Type: string
Default: No Default

`selectivities2` The categories of tagged individuals for the observation
Type: string vector
Default: No Default

`detection` The probability of detecting a recaptured individual
Type: constant
Default: No Default
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`time_step_proportion` The proportion through the mortality block of the time step when the observation is evaluated
Type: constant
Default: double
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

11.1.15. `@observation[label].type=tag_recapture_by_length`

`years` The years for which there are observations
Type: non-negative integer vector
Default: No Default

`time_step` The time step to execute in
Type: string
Default: No Default

`length_bins` The length bins
Type: constant vector
Default: true

`selectivities` The labels of the selectivities used for untagged categories
Type: string vector
Default: true

`tagged_selectivities` The labels of the tag category selectivities
Type: string vector
Default: No Default

`detection` The probability of detecting a recaptured individual
Type: constant
Default: No Default
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`dispersion` The overdispersion parameter ,phi,
Type: constant
Default: double
Lower Bound: 0.0 (inclusive)

`time_step_proportion` The proportion through the mortality block of the time step when the observation is evaluated
Type: constant
Default: double
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

11.2. Likelihoods

`@likelihood` *`label`* Define an object of type *likelihood*

11.2.1. `@likelihood[label].type=binomial`

11.2.2. `@likelihood[label].type=binomial_approx`

11.2.3. `@likelihood[label].type=dirichlet`

11.2.4. `@likelihood[label].type=log_normal`

11.2.5. `@likelihood[label].type=log_normal_with_q`

11.2.6. `@likelihood[label].type=multinomial`

11.2.7. `@likelihood[label].type=normal`

11.2.8. `@likelihood[label].type=pseudo`

11.3. Defining ageing error

The methods for including ageing error into estimation with observations are:

- None
- Data
- Normal

- Off-by-one

Each type of ageing error requires a set of subcommands and arguments specific to its type.

`@ageing_error` *`label`* Define an object of type *ageing_error*

`label` The label of the ageing error
Type: string
Default: No Default

`type` The type of ageing error
Type: string
Default: No Default

11.3.1. `@ageing__error[label].type=data`

11.3.2. `@ageing__error[label].type=none`

11.3.3. `@ageing__error[label].type=normal`

`cv` CV of the misclassification matrix
Type: estimable
Default: No Default
Lower Bound: 0.0 (exclusive)

`k` k defines the minimum age of individuals which can be misclassified, i.e., individuals of age less than k have no ageing error
Type: non-negative integer
Default: 0u
Lower Bound: 0u (inclusive)

11.3.4. `@ageing__error[label].type=off_by_one`

`p1` The proportion misclassified as one year younger, e.g., the proportion of age 3 individuals that were misclassified as age 2
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`p2` The proportion misclassified as one year older, e.g., the proportion of age 3 individuals that were misclassified as age 4
Type: estimable
Default: No Default
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

`k` The minimum age of animals which can be misclassified, i.e., animals of age less than k are assumed to be correctly classified
Type: non-negative integer
Default: 0u
Lower Bound: 0.0 (inclusive)
Upper Bound: 1.0 (inclusive)

11.4. Simulating observations

`@simulate` *`label`* Define an object of type *simulate*

`label` Label
Type: string
Default: No Default

`type` Type
Type: string
Default: ""

`years` Years to recalculate the values
Type: non-negative integer vector
Default: true

`parameter` Parameter to simulate
Type: string
Default: No Default

11.4.1. `@simulate[label].type=constant`

`value` The value to assign to the addressable
Type: constant
Default: No Default

12. Report command and subcommand syntax

12.1. Report commands and subcommands

`@report` *`label`* Define an object of type *report*

`label` The label for the report
Type: string
Default: No Default

`type` The type of report

Type: string
Default: No Default

`file_name` The filename for this report to be in a separate file
Type: string
Default: ""

`write_mode` The write mode
Type: string
Default: overwrite
Allowed Values: overwrite, append, incremental_suffix

12.1.1. `@report[label].type=addressable`

`parameter` The parameter to print
Type: string
Default: No Default

`years` The years to print the addressable for
Type: non-negative integer vector
Default: No Default

`time_step` The time Step label
Type: string
Default: ""

12.1.2. `@report[label].type=age_length`

`time_step` The time step label
Type: string
Default: ""

`years` The years for the report
Type: non-negative integer vector
Default: true

`age_length` The age-length label
Type: string
Default: No Default

`category` The category label
Type: string
Default: No Default

12.1.3. @report[label].type=ageing_error_matrix

`ageing_error` The ageing error label
Type: string
Default: No Default

12.1.4. @report[label].type=catchability

12.1.5. @report[label].type=category_info

12.1.6. @report[label].type=category_list

12.1.7. @report[label].type=correlation_matrix

12.1.8. @report[label].type=covariance_matrix

12.1.9. @report[label].type=derived_quantity

12.1.10. @report[label].type=equation_test

`equation` The equation to do a test run of
Type: string vector
Default: No Default

12.1.11. @report[label].type=estimate_summary

12.1.12. @report[label].type=estimate_value

12.1.13. @report[label].type=estimation_result

12.1.14. @report[label].type=hessian_matrix

12.1.15. @report[label].type=initialisation_partition

12.1.16. @report[label].type=initialisation_partition_mean_weight

12.1.17. @report[label].type=mcmc_covariance

12.1.18. @report[label].type=mcmc_objective

12.1.19. @report[label].type=mcmc_sample

12.1.20. @report[label].type=m_p_d

12.1.21. @report[label].type=objective_function

12.1.22. @report[label].type=observation

`observation` The observation label
Type: string
Default: No Default

`normalised_residuals` Print Normalised Residuals?
Type: boolean
Default: false

`pearsons_residuals` Print Pearsons Residuals?
Type: boolean
Default: false

12.1.23. @report[label].type=output_parameters

12.1.24. @report[label].type=partition

`time_step` The time step label
Type: string
Default: ""

`years` The years for the report
Type: non-negative integer vector
Default: true

12.1.25. `@report[label].type=partition_biomass`

`time_step` The time step label
Type: string
Default: ""

`years` The years for the report
Type: non-negative integer vector
Default: true

12.1.26. `@report[label].type=partition_mean_length`

`time_step` The time step label
Type: string
Default: ""

`years` The years for the report
Type: non-negative integer vector
Default: true

12.1.27. `@report[label].type=partition_mean_weight`

`time_step` The time step label
Type: string
Default: ""

`years` The years for the report
Type: non-negative integer vector
Default: true

12.1.28. `@report[label].type=partition_year_cross_age_matrix`

12.1.29. `@report[label].type=process`

`process` The process label that is reported
Type: string
Default: ""

12.1.30. `@report[label].type=project`

`project` The project label that is reported
Type: string
Default: ""

12.1.31. `@report[label].type=random_number_seed`

12.1.32. `@report[label].type=selectivity`

`selectivity` The selectivity name
Type: string
Default: No Default

12.1.33. `@report[label].type=simulated_observation`

`observation` The observation label
Type: string
Default: No Default

12.1.34. `@report[label].type=standard_header`

12.1.35. `@report[label].type=time_varying`

13. Including commands from other files

`@include` *`file`* Include an external file

`file` The name of the external file to include
Type: string
Default: No default
Value: A valid external file
Condition: The file name must be enclosed in double quotes
Example: `@include "my_file.csl2"`
Note: `@include` does not denote the end of the previous command block as is the case for all other commands

14. Tips for setting up Casal2 model based on an existing CASAL model

Many users of CASAL2 may be be starting with a functioning CASAL model. This section focuses on transitioning from CASAL to CASAL2.

There are a range of reasons why CASAL2 will output different values when comparing model output to CASAL models. There are also reasons why values will differ that are not so obvious such as, reasons caused from using different compilers on different machines where over/underflow might occur. It is assumed that the latter reasons should be rare, and the 'overall' behaviour when it comes to estimation will be the same between CASAL and CASAL2.

Reasons why there may be different values reported between CASAL and CASAL2 include:

- Report rounding. There are settings with respect to output in CASAL that set the number of significant figures for writing to files. So if values look truncated, this might be the reason.
- Priors on parameters that are turned off with `upper_bound` = `lower_bound`. In both CASAL and CASAL2 the estimation of parameters can be turned off by setting the bounds equal. CASAL will evaluate the prior value and add this to the objective function. This contribution is a constant value so it will not affect parameter inference. It may however be confusing when comparing output between the two models.
- Default values. There are a lot of switches in these programs, and options like the `delta` in CASAL2 or `r` parameter in CASAL for robustifying likelihoods can cause differences.
- The order of processes. CASAL has a predefined sequence in which it executes processes within a time step (i.e., ageing, recruitment, maturation, migration, growth, natural and fishing mortality, disease mortality, tag release events, tag shedding rate, and semelparous mortality), where as CASAL2 is completely user defined.
- Length-based processes or observations. CASAL2 has updated the cumulative normal distribution calculation (CASAL used the approximated no closed form solution) with better approximations.
- Age-based observations. CASAL2 does not have the `sum_to_one` subcommand implemented, and CASAL makes this adjustment implicitly. Check that this flag is set to 'false' in the CASAL model for a more accurate comparison.
- Tag penalties. CASAL applies a penalty to the sum of squares on total tagged fish in a 'tagging episode' from the model compared to observed number of tagged fish. CASAL2 applies a penalty on the transition rate by length. If tags are applied in a length bin that does not have individuals, e.g., a model configuration which tags 2 individuals of length l when there are no individuals in that length bin will include a penalty.

Many of the flags and options in CASAL and CASAL2 are the same or similar. The syntax section of this document (Sections 9) provides more details about the CASAL2 functionality and behaviour. Check that the programs produce the same results with a **range** of parameter values using the deterministic run command (`casal2 -r`), before doing an estimation run (`casal2 -e`).

The first outputs to check when comparing CASAL2 and CASAL versions of the same model are the stock dynamics outputs, ignoring the fits to observations. That is, check the initial age structure, the SSB and YCS values and patterns, R0, B0, etc. If these outputs differ, then the fits to the observations will likely also be different.

There are a few linkages with certain stock dynamics outputs to check to determine if processes are misspecified. Differences between the proportions in the initial age structure, assuming an equilibrium state, are due to M, natural mortality. Differences in the initial equilibrium recruitment

value, R_0, are due to growth (`@age_length` or `@length_weight`). Many models estimate B_0 so that R_0 is a back calculation through the growth curve.

If the initial age structure is the same, next check the derived quantities such as the SSB values. Differences in these values are generally caused by how fishing and recruitment processes are specified. Check which *YCS* values are estimated or standardised, the definition and designation of selectivities, etc.

Once the stock dynamics outputs match, check the results with a few different sets of starting parameter values by using the `-i` command line option. Next, check the fits to the observation data by comparing the expected values. Assuming the observations in both models match, the differences in the objective function value come from the expected values and the likelihood configurations. This is where subcommands such as the robustification values and the default values may differ between CASAL and CASAL2.

Once the stock dynamics outputs and the fits to the observation data are the same, do an estimation run (`casal2 -e`). If CASAL and CASAL2 do not optimise to the same parameter values, then use the parameter values from CASAL and do a deterministic run with CASAL2 using the CASAL estimated parameter values (`casal2 -r -i CASAL_mpd_pars.txt`). Then check the stock dynamics outputs and the fits to the observation data and determine where the differences in the parameter estimates and outputs are.

The next question is, how close do the parameter estimates, expected values, and objective function values have to be to say that the models are equivalent? This is an ongoing topic of discussion. Previously, subjective qualitative measures have been used to decide whether the models are equivalent. A recorded comparison for the hake stock assessment can be found at Appendix B in Horn (2017).

15. Syntax conventions, examples and niceties

15.1. Input File Specification

The file format used for CASAL2 is based on the command block formats used in CASAL and SPM. It is a text file that contains definitions organised into blocks.

Every object specified in a configuration file is part of a block. At the top level blocks have a one-to-one relationships with components in the system.

Example:

```
@block1 label
parameter value
parameter value_1 value 2

@block2 label
parameter value
table table_name
column_1 column_2
data_1 data_2
data_3 data_4
end_table
```

Some general notes about configuration files:

- Whitespace can be used freely. Tabs and spaces are both valid.
- A block ends only at the beginning of a new block or at the end of the final configuration file.
- Configuration files can include other configuration files.
- Included files are placed in-line, so a block can be continued in a new file.
- The configuration files support in-line declarations of objects.

15.1.1. Keywords And Reserved Characters

In order to allow efficient creation of input files, the CASAL2 file format has special keywords and characters that cannot be used for labels.

`@`Block Definitions

Each block in the configuration file must start with the block definition character, which is the "`@`" character.

Example:

```
@block1 <label>
type <type>

@block2 <label>
type <type>
```

The 'type' Keyword

The 'type' keyword is used for declaring the sub-type of a defined block. Any block object that has multiple sub-types will use the `type` keyword.

Example:

```
@block1 <label>
type <sub_type>

@block2 <label>
type <sub_type>
```

(Single Line Comments)

Comments are supported in the configuration file on one line (to the end of that line) or over multiple lines. Comments on single lines start with the "#" character.

Example:

```
@block <label>
type <sub_type> # Descriptive comment
# parameter <value_1> *** This whole line is commented out
parameter <value_1> # <value_2> *** value_2 is commented out
```

/* */ (Multiple Line Comments)

Multiple line comments are supported by surrounding the comments in /* and */

Example:

```
@block <label>
type <sub_type>
parameter <value_1>
parameter <value_1> <value_2>

/*
Do not load this process
@block <label>
type <sub_type>
parameter <value_1>
parameter <value_1> <value_2>
*/
```

{} (Indexing Parameters)

Individual elements of a vector can be referenced using the `{ }` syntax. For example, when estimating `ycs_values` a range or block of YCS values can be referenced.

Example:

```
@estimate YCS
parameter process[Recruitment].ycs_values{1975:2012}
type uniform
lower_bound
upper_bound
```

':' (Range Specifier)

The range specifier ":" allows specifying a range of values instead of specifying each value explicitly. Ranges can be either incremental or decremental.

Example:

```
@process my_recruitment_process
type constant_recruitment
# With the range specifier
years_to_run 1999:2009

@process my_mortality_process
type natural_mortality
# Without the range specifier
years_to_run 2000 2001 2002 2003 2004 2005 2006 2007
```

',' (List Specifier)

When a parameter supports multiple values in a single entry, the list specifier "," can be used to define multiple values as a single parameter.

Example:

```
@categories
format sex.stage
# With the list specifier
names male,female.immature,mature

@categories
format sex.stage
# Without the list specifier
names male.immature male.mature female.immature female.mature
```

'table' and 'end_table' Keyword

The table keyword `table` is used to define a tabular block of values used as a parameter. The first line following the `table` declaration must contain a list of columns to be used. The subsequent lines are rows of the table. Each row must have the same number of values as the number of columns specified. The table definition must end with the "`end_table`" keyword on its own line.

Example:

```
@block <label>
type <sub_type>
parameter <value_1>
table <table_label>
<column_label_1> <column_label_2> ...  <column_label_N>
<row1_value_1>   <row1_value_2>   ...  <row1_value_N>
<row2_value_1>   <row2_value_2>   ...  <row2_value_N>
end_table
```

[ ] (in-line Declarations)

When an object takes the label of a target object as a parameter, the label can be replaced with an in-line declaration. An in-line declaration "[ ]" is a complete declaration of an object on one line. This feature is designed to allow simplifying the configuration definition.

Example:

```
@model
# With in-line declaration with label specified for time step
time_steps step_one=[type=iterative; processes=recruitment ageing]

@model
# With in-line declaration with default label (model.1)
time_steps [type=iterative; processes=recruitment ageing]


# Without in-line declaration
@model
time_steps step_one

@time_step step_one
processes recruitment ageing
```

Categories

The CASAL2 population representation is essentially a 2-dimensional structure. The partition is:

Categories x Ages or Lengths

Each category allows for a different range of ages or lengths and accessibility during different time periods.

Because each category can be quite complicated, the syntax for defining categories has been structured to allow for complex definitions using a simple shorthand structure.

The "format" parameter allows for defining the structure of the category labels. Using a "." (period) character between each segment allows for shorthand lookups of categories.

The "names" parameter is a list of the category names. The syntax of these names is required to match the "format" parameter so CASAL2 can organise and search on them. Using the "list specifier" and range characters this parameter can be shortened.

Example:

```
@categories
format sex.stage.tag
names male.immature.notag male.immature.2001 male.mature.notag male.mature.2001

names male.immature # Invalid: No tag information
names female # Invalid: no stage of tag information
names female.immature.notag.1 # Invalid: Additional format segment not defined

names male,female.immature,mature.notag,2001:2005 # Valid
# Without the shorthand syntax these categories would be written:
names male.immature.notag male.immature.2001 male.immature.2002 male.immature.2003 male.immature.200
```

Specific data for a year in a category can be set up so that this category is not to be processed during specific years or in the initialisation phases. A list of years can be provided for each category to indicate in which year(s) it is to be available. These years which will override the default setting of all years in the model. Any category with the default years overridden will no longer be accessible in the initialisation phases.

Examples:

```
@model
start_year 1998
final_year 2010

@categories
format sex.stage.tag
names male,female.immature,mature.notag,2001:2005 # Valid
# Categories with the tag value "2001" are available in years 1999, 2000, 2001, 2002, 2003
# Categories with the tag value "2005" are available in years 2003, 2004, 2005, 2006, 2007
years tag=2001=1999:2003  tag=2005=2003:2007
```

15.2. More examples of shorthand syntax and use of reserved and key characters

Categories

CASAL2 allows for many user-defined categories so shorthand syntax has been added to aid in the definition of complex configuration labelling and partition structures. For example, when defining categories a comma "`,`" can be used to shorten lists of categories.

This syntax is the long way:

```
@categories
format sex.stage
names male.immature male.mature female.immature female.mature
```

For the exact same partition structure specified in a shorter way:

```
@categories
format sex.stage
names male,female.immature,mature
```

CASAL2 requires categories in processes and observations so that the correct model dynamics can be applied to the correct elements of the partition.

An example of a process where categories are required as an input command is for ageing

```
# 1. The standard way
@ageing my_ageing
categories male.immature male.mature female.immature female.mature

# 2. The first shorthand way
@ageing my_ageing
categories male,female.immature,mature

# 3. Wild Card (all categories)
```

```
@ageing my_ageing
categories *

# 4. The second shorthand way
@ageing my_ageing
categories sex=male sex=female
```

To combine/aggregate categories together, use the "+" special character. For example, this feature can be used to specify that the total biomass of the population is made up of both males and females.

For example,

```
@observation CPUE
type biomass
catchability Fishq
time_step one
categories male+female
selectivities FishSel
likelihood lognormal
time_step_proportion 1.0
years 1992:2001
table obs
1992    1.50    0.35
1993    1.10    0.35
1994    0.93    0.35
1995    1.33    0.35
1996    1.53    0.35
1997    0.90    0.35
1998    0.68    0.35
1999    0.75    0.35
2000    0.57    0.35
2001    1.23    0.35
end_table
```

This combination/aggregation functionality can be used to compare an observation to the total combined population:

```
@observation CPUE
type biomass
catchability Fishq
time_step one
categories *+
selectivities FishSel
likelihood lognormal
time_step_proportion 1.0
years 1992:2001
table obs
1992    1.50    0.35
1993    1.10    0.35
1994    0.93    0.35
1995    1.33    0.35
1996    1.53    0.35
1997    0.90    0.35
1998    0.68    0.35
1999    0.75    0.35
```

```
2000    0.57    0.35
2001    1.23    0.35
end_table
```

If male and female are the only categories in a population, then this is the same syntax as the command block above it.

Shorthand syntax can be useful when applying processes to a select group of categories from the partition.

For example, to apply a spawning migration to the mature categories in the partition and the partition was defined:

```
@categories
format area.maturity.tag
names north.immature.notag,2011 north.mature.notag,2011 south.immature.notag,2011
south.mature.notag,2011
```

Then, to migrate a portion of the mature population from the southern area to the northern area:

```
@process spawn_migration
type transition_category
from format=south.mature.*
to format=north.mature.*
proportions 1.0
selectivities One
```

Parameters

CASAL2 also allows parameters that are of type vector or map to be referenced and estimated fully or partially. An example of a parameter that is type vector is ycs_values in a recruitment process.

For example, a recruitment block:

```
@process WestRecruitment
type recruitment_beverton_holt
r0 400000
years
ycs_values 1 1 1 1 1 1 1 1
ycs_years 1975:1983
# An alternative method to specify a sequence of values
# use an asterix to represent a vector of repeating integers
ycs_values 1*8
steepness 0.9
age 1
```

To estimate the last four years of the parameter process[WestRecruitment].ycs_values only can be specified as

```
@estimate
parameter process[WestRecruitment].ycs_values{1980:1983}
type uniform
lower_bound 0.1 0.1 0.1 0.1
upper_bound  10  10  10  10
```

Note that the first element of a vector is indexed by 1. This syntax can be applied to parameters that are of type map as well. For information on what type a parameter is see the syntax section.

An example of a parameter that is of type map is `@time_varying[label].type=constant`.

For a `@time_varying` block

```
@time_varying q_step1
type constant
parameter catchability[Fishq].q
years  1992 1993 1994 1995
value  0.2 0.2 0.2 0.2
```

For example, to estimate only one element of the map (say 1992), and force all other years to be the same as the one estimate, can be done in the `@estimate` block using `same`:

```
@estimate
parameter time_varying[q_step1].value{1992}
same time_varying[q_step1].value{1993:1995}
type uniform
lower_bound 0.1 0.1 0.1 0.1
upper_bound  10  10  10  10
```

In-line declaration

In-line declarations can help shorten models by passing `@` blocks.

For example,

```
@observation chatCPUE
type biomass
catchability [q=6.52606e-005]
time_step one
categories male+female
selectivities chatFselMale chatFselFemale
likelihood lognormal
time_step_proportion 1.0
years 1992:2001
table obs
1992    1.50    0.35
1993    1.10    0.35
1994    0.93    0.35
1995    1.33    0.35
1996    1.53    0.35
1997    0.90    0.35
1998    0.68    0.35
1999    0.75    0.35
2000    0.57    0.35
2001    1.23    0.35
end_table

@estimate
parameter catchability[chatTANbiomass.one].q
type uniform_log
lower_bound 1e-2
upper_bound 1
```

In the above code catchability is defined and estimated without explicitly creating a `@catchability` block.

When an in-line declaration is made, the new object will be created with the name of the creator's `label.index`, where `index` is the word "`one`" through "`nine`" if it is 1 through 9, and the number if it is 10+.

For example,

```
@mortality halfm
selectivities [type=constant; c=1]

would create
@selectivity halfm.one
```

If there are 10 categories, each with its own selectivity, the 10th selectivity is labelled

```
@selectivity halfm.10
```

15.3. Processes

Processes are special in how they can be defined. Throughout this document specifying a process has been

```
@process Recruitment
type recruitment_beverton_holt
```

However, for convenience and clarity, this block can also be specified as

```
@recruitment Recruitment
type beverton_holt
```

The difference is that the keyword `process` can be replaced with the first word of the process type. In the example above this is the `recruitment` process. This option can be used to create more succinct model configurations.

More examples:

```
@mortality Fishing_and_M
type instantaneous

@transition Migration
type category
```

15.4. An example of a simple model

This example describes a single species and area model, with recruitment, maturation, natural and fishing mortality, and an annual age increment. The population structure has ages $1-30^{+}$ with a single category.

The default CASAL2 configuration filename is `config.csl2`. In this example, `config.csl2` specifies the files to include to run the CASAL2 model from the current directory using the `!include` command.

```
!include "population.csl2"
!include "reports.csl2"
!include "observation.csl2"
!include "estimation.csl2"
```

It is recommended to separate the sections of a CASAL2 model for enhancing readability and error checking, and including the files in a version control system.

The file `population.csl2` contains the population information. The model years are from 1975 through 2012, with 3 time steps. The model is initialised over a 120 year period prior to 1975 and applies the following processes

- A Beverton-Holt recruitment process, recruiting a constant number of individuals to the first age class (i.e., $age = 1$).
- A constant mortality process representing natural mortality(M). This process is repeated in all 3 time steps, so that a proportion of M is applied in each time step.
- An ageing process, where all individuals are aged by one year, and with a plus group accumulator age class at $age = 30$.

Following initialisation, the model runs from the years 1975 to 2012 iterating through 3 time steps.

The first time step applies processes of recruitment, and $\frac{1}{2}M_1 + F + \frac{1}{2}M_1$ processes, where M_1 is the proportion of M applied in the first time step. The exploitation process (fishing) is applied in the years 1975 - 2012. Catches are defined in the catches table and attributes for each fishery, such as selectivity and time step they are implemented, are in the fisheries table in the `@process` block.

The second time step applies an age increment and the remaining natural mortality.

The third time step applies TODO.

The first 28 lines of the main section of the `population.csl2`:

```
#THE MODEL constraints
@model
start_year 1975
final_year 2012
min_age 1
max_age 30
age_plus true
base_weight_units tonnes
initialisation_phases Equilibrium_state
time_steps Sep_Feb Mar_May Jun_Aug ##
length_bins 10 20 30 40 50 60 70 80 90 100 110 120 130 140 150 160 170 180 190 200 210 220 230
     240 250

#CATEGORIES
@categories
format stock ## Single sex and area population
names HAK4
age_lengths age_size

@initialisation_phase Equilibrium_state
type derived

## Define the processes in the Annual Cycle
## This is a list of labels that correspond to a process
@time_step Sep_Feb
processes Recruitment Instantaneous_Mortality

@time_step Mar_May
```

```
processes Instantaneous_Mortality
```

To run the model to verify that the model runs without any syntax errors, use the command `casal2 -r`. Since CASAL2 reads in the default filename `config.csl2`, this filename can be overridden. For example, if the model is in file `Mymodel.txt`, then this filename would be specified using the `-c` option, `casal2 -r -c Mymodel.txt`.

To estimate the parameters defined in the file `estimation.csl2` (the catchability constant q, recruitment R_0, and the selectivity parameters a_{50} and a_{to95}), use `casal2 -e`. The output has been redirected to file `estimate.log` using the command `casal2 -e > estimate.log`. Reports for the user-defined reports `reports.csl2` from the final iteration of the estimation are output to the file `estimate.log`, and successful convergence is printed to the screen

```
Total elapsed time: 1 second
Completed
```

The main output from the estimation run is summarised in the file `estimate.log`, and the final MPD parameter values can also be redirected as a separate report, in this case named `paramaters.out`, using the command `casal2 -e -o paramaters.out > estimate.log`.

A profile on the R_0 parameter can be run, using `casal2 -p > profile.log`. See the examples folder for the example of the output.

16. Post-processing output using R

R (https://www.r-project.org/) is the main application used to process and visualise output from a CASAL2 model. **R** is free and can be downloaded from https://cran.r-project.org/. Once you have installed **R** you can install the casal2 **R** package from the file (casal2_1.0.tar.gz) which is part of the CASAL2 download.

The CASAL2 **R** package has functionality to parse CASAL2 output into a list. It also has diagnostic, plotting, and summarising functions.

There are three types of output that CASAL2 can produce, depending on the type of analysis run. These outputs are: Standard, MCMC, and Derived Quantity.

The Standard outputs are the reports that are produced in most CASAL2 run modes, with the exception of -s and -m. The Standard output can be split into two additional categories, a single parameter run (casal2 -r) or a multi-parameter run (casal2 -r -i many_pars.out), or running in projection mode (-f 1). The Standard outputs can be read into **R** using the extract.mpd() function.

The second type of output is generated when doing an MCMC analysis (casal2 -m), which can generate two files, mcmc_objective.out and mcmc_samples.out. The MCMC outputs can be used to summarise convergence properties or chain behaviour, and can also be used to view marginal posteriors and quantify parameter uncertainty.

The third output type is the Derived Quantity outputs, also referred to as tabular output. The Derived Quantity output can be generated after an MCMC analysis is done, to produce the marginal posteriors for derived quantities. A commonly reported derived quantity in fisheries stock assessment modelling is the time series of spawning stock biomass. To get the posterior distributions for these derived quantities use the --tabular flag (e.g., casal2 -r -i mcmc_samples.out --tabular > Tabular_report.out). This output can then be read into **R** using the extract.tabular() function.

CASAL2's reported output is written so that each @report will start with a '*' and end with '*end'. This format can be used as the basis to construct functions that read CASAL2 output to identify and read individual reports for post-processing.

The CASAL2 **R** extract() functions differ by how the expected output is structured and they each create a different casal2 object. The summary() and plot() functions will generate different plots for the different casal2 objects. Objects produced by the extract() function can be queried with class(object).

The list of casal2 **R** functions include:

- extract.mpd(), which parses the CASAL2 default output into a list
- extract.mcmc(), which parses the CASAL2 MCMC output into a list
- extract.tabular(), which parses the CASAL2 tabular output into a list
- extract.parameters(), which parses the CASAL2 parameter files into a list
- generate.starting.pars(), which reads in a file that contains the @estimate blocks and generates 'N' starting values to test convergence (???)
- burn.in.tabular(), which omits the first 'N' rows from a casal2TAB object
- plot.derived_quantities(), which plots the derived quantities
- plot.selectivities(), which plots the selectivities

- `plot.ycs()`, which plots the true YCS strengths
- `plot.pressure()`, which plots the fishing pressures
- `summary()`, which summarises a model run
- `extract.csl2.file()`, which reads a CASAL2 .csl2 (configuration) file into a list
- `write.csl2.file()`, which writes a CASAL2 .csl2 (configuration) file from (???)
- `ReadSimulatedData()`, which parses CASAL2 output from a `casal2 -s` run
- `Method.TA1.8()`, which returns a weighting factor for age or length composition data. See Francis (2011) for more detail.
- `apply.dataweighting.to.csl2()`, which parses a CASAL2 .csl2 (configuration) file that contains `@observation` blocks, applies a weighting factor to an age or length composition data set, and generates a new `.csl2` file with modified effective sample size values

The required and optional arguments for these functions can be queried after loading the CASAL2 **R** library with `library(casal2)` and using the standard **R** help syntax `?` (e.g., `?param.profile()`). Many of the help files have example code and data to demonstrate function syntax.

Standard diagnostic functions and plots for model output

TODO (functionality description)

`plot.derived_quantity()`

When comparing model output either: different parameters for the same model structure are being compared (Situation 1), or outputs from multiple model structures are being compared (Situation 2). These functions can be useful for both comparison types.

- `plot.selectivities()`
- `plot.pressure()`
- `plot.fit()`
- `plot.ycs()`

Data weighting

An important component of fisheries stock assessment modelling is addressing data conflicts through the use of data weighting. There are a range of methods that can be used (Francis (2011)). The CASAL2 **R** function is `Method.TA1.8()`. An additional function `apply.dataweighting.to.csl2()` automatically applies a weighting factor to a specific age or length composition data in an `@observation` block, and generates a new `.csl2` file with modified effective sample size values.

```
library(casal2)

## read in the reported output from a "casal2 -e" run
## ensure there is a @report block for the observation of interest.
mpd <- extract.mpd(file = "estimate.log")

## calculate weighting factor from Francis method
WeightingFactor <- Method.TA1.8(model = mpd, observation_labels = "chatTANage")

## Apply the weighting factor to the block in the Observation.csl2 file
```

```
## this call generates a new file (Observation.csl2.0) with the re-weighted effective sample
    sizes
apply.dataweighting.to.csl2(weighting_factor = WeightingFactor,
                            Observation_csl2_file = "Observations.csl2",
                            Observation_label = "chatTANage",
                            Observation_out_filename = "Observation.csl2.0")
```

Automating the data weighting process:

```
library(casal2)

mpd <- extract.mpd(file = "estimate.log")

ModelFactor <- Method.TA1.8(mpd, observation_labels = c("ObserverProportionsAtAge"))

## make a back-up copy of the file Observation.csl2 before running this section

while(abs(ModelFactor - 1) > 0.01) {
  shell("betadiff & casal2 -e > estimate.log 2> log.out")

  new_mpd <- extract.mpd(file = "estimate.log")

  ModelFactor <- Method.TA1.8(new_mpd, observation_labels = c("ObserverProportionsAtAge"))

  apply.dataweighting.to.csl2(weighting_factor = ModelFactor,
                              Observation_csl2_file = "Observation.csl2",
                              Observation_out_filename = "Observation.csl2",
                              Observation_label = c("ObserverProportionsAtAge"))
  print(ModelFactor)
}
```

Troubleshooting the `casal2` R package

If you get this error when using one of the `extract()` functions

```
Read 1 item
Warning messages:
1: In scan(filename, what = "", sep = "\n", fileEncoding = fileEncoding) :
embedded nul(s) found in input
2: In extract.mpd(file = "results.txt", fileEncoding = "") :
File is empty, no reports found
```

You may be able to resolve this issue by using an alternative UTF format by specifying this format with the `fileEncoding` parameter

```
MyOutput <- extract.mpd(file = "Estimate.log", path = getwd(), fileEncoding = "UTF-16LE")
```

17. Troubleshooting

This section is to aid users in debugging models. If you cannot resolve an issue using these guidelines then please contact the development team. To report an issue please follow the format described in Section 17.2.1.

Most user errors should be well documented and CASAL2 should produce informative error messages. There are runtime options that users can enable to attempt to resolve or at least isolate an error or bug, including different levels of logging.

17.1. Logging

CASAL2's internal logging system can be invoked at the command line with argument `-loglevel` followed by one of these options: `trace`, `finest`, `fine`, `medium`.

An example of logging with trace level output:

- On Windows: `casal2 -r -loglevel trace > output.log 2> log.out`
- On Linux: `casal2 -r -loglevel trace > output.log 2&> log.out`

This argument will output CASAL2's reports to the file "output.log", and the "`2>`" or "`2&>`" syntax will print the error logged information to the file "log.out". You should be able to see where CASAL2 is exiting by going to the end of the "log.out" file.

The optimal level of logging will depend on what run mode you are using and the granularity of information that you would like to see. There is an ordering in the options, with `medium` being the most coarse, and `trace` being the finest level, with `fine` and `finest` in between. We suggest that if you are running CASAL2 in an iterative state such as for estimation (`casal2 -e`) or MCMC you use `medium` level. This is because the logging can print a lot of information for a single model run, so an estimation which could comprise thousands of model runs can produce very large text files with the finer logging option specified. For a single iteration run such as `casal2 -r` each of the logging options can be useful during different phases of model development.

You can see how CASAL2 creates these reports by looking in the ".cpp" files in the Observation or Processes source code subdirectories and see code such as in `Model/Model.cpp`,

```
LOG_FINE() « "Model:  State change to Execute";
```

17.2. Reporting errors

If you find a bug or error in CASAL2, please submit an issue in the GitHub repository at `https://github.com/NIWAFisheriesModelling/CASAL2/issues`.

Please follow the guidelines below so that the bug or error can be reproduced. It is helpful to be as detailed and specific as possible when describing the observed behavior as well as the expected behaviour.

17.2.1. Guidelines for reporting an error with CASAL2

1. Ensure you are using the most recent version of CASAL2, as the bug or error you are having may have already been resolved.

2. Provide the version of CASAL2 you are using, e.g., "CASAL2 v2020-08-07 (rev. 2d6fe374)". The version is output by CASAL2 with the command `casal2 -v`.
3. Provide the operating system you are using, e.g., "IBM-PC Intel CPU with Microsoft Windows 10 Enterprise".
4. Provide a brief description of the problem, e.g., "a segmentation fault was produced".
5. If the problem is reproducible, please describe in detail the steps required to cause it, and include the CASAL2 configuration files, other input files, and any output files generated. Specify the *exact* command line arguments that were used, e.g., "Using the command `casal2 -e -q` produced a segmentation fault. The input configuration files are attached."
6. If the problem is not reproducible (it happened only once, or occasionally for no apparent reason), please describe in detail the circumstances in which it occurred and the behaviour observed, e.g., "CASAL2 crashed, but I have not been able to reproduce the issue. It seemed to be related to a local network crash but I cannot be sure."
7. If the problem produced any error messages, please give the *exact* text displayed, e.g., "`segmentation fault (core dumped)`".
8. Attach all relevant input and output files so that the problem can be reproduced; these files can be compressed into a single file e.g., a zip file, and uploaded to GitHub.

18. CASAL2 software license

GNU GENERAL PUBLIC LICENSE

Copyright © 1989, 1991 Free Software Foundation, Inc.

51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change free software—to make sure the software is free for all its users. This General Public License applies to most of the Free Software Foundation's software and to any other program whose authors commit to using it. (Some other Free Software Foundation software is covered by the GNU Library General Public License instead.) You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights. These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must give the recipients all the rights that you have. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

We protect your rights with two steps: (1) copyright the software, and (2) offer you this license which gives you legal permission to copy, distribute and/or modify the software.

Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations.

Finally, any free program is threatened constantly by software patents. We wish to avoid the danger that redistributors of a free program will individually obtain patent licenses, in effect making the program proprietary. To prevent this, we have made it clear that any patent must be licensed for everyone's free use or not licensed at all.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public License.

The "Program", below, refers to any such program or work, and a "work based on the Program" means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or translated into another language. (Hereinafter, translation is included without limitation in the term "modification".) Each licensee is addressed as "you".

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program). Whether that is true depends on what the Program does.

1. You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence of any warranty; and give any other recipients of the Program a copy of this License along with the Program.

   You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

   a) You must cause the modified files to carry prominent notices stating that you changed the files and the date of any change.
   b) You must cause any work that you distribute or publish, that in whole or in part contains or is derived from the Program or any part thereof, to be licensed as a whole at no charge to all third parties under the terms of this License.
   c) If the modified program normally reads commands interactively when run, you must cause it, when started running for such interactive use in the most ordinary way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this License. (Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

   These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Program, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

   Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program.

   In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you also do one of the following:

   a) Accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,
   b) Accompany it with a written offer, valid for at least three years, to give any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corresponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or,
   c) Accompany it with the information you received as to the offer to distribute corresponding source code. (This alternative is allowed only for noncommercial distribution and only if you received the program in object code or executable form with such an offer, in accord with Subsection b above.)

   The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the executable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

   If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code.

4. You may not copy, modify, sublicense, or distribute the Program except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense or distribute the Program is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

5. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.

6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original licensor to copy, distribute or modify the Program subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties to this License.

7. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence

you may not distribute the Program at all. For example, if a patent license would not permit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Program.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

8. If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Program under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

9. The Free Software Foundation may publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

   Each version is given a distinguishing version number. If the Program specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this License, you may choose any version ever published by the Free Software Foundation.

10. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

11. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

19. Acknowledgements

We thank the developers of CASAL (Bull et al., 2012) for their ideas that led to the development of CASAL2. The CASAL2 logo was designed by Ian Doonan and Erika Mackay (NIWA).

Much of the structure of CASAL2, equations, and documentation in this manual draw heavily on similar components of the fisheries population modelling application CASAL (Bull et al., 2012) and the spatial population model SPM (Dunn et al., 2015). We thank the authors of CASAL and SPM for their permission to use their work as the basis for parts of CASAL2 and allow the use of the definitions, concepts, and documentation.

20. References

R J H Beverton and S J Holt. *On the dynamics of exploited fish populations*. Fishery investigations. HMSO, London, 1957.

B Bull, R I C C Francis, A. Dunn, A McKenzie, D J Gilbert, M H Smith, R Bian, and D Fu. CASAL C++ Algorithmic Stock Assessment Laboratory): CASAL user manual v2.30-2012/03/21. Technical Report 135, National Institute of Water and Atmospheric Research Ltd (NIWA), 2012.

J E Dennis Jr and R B Schnabel. *Numerical methods for unconstrained optimisation and nonlinear equations*. Classics in Applied Mathematics. Prentice Hall, 1996.

Ian Doonan, Kath Large, Alistair Dunn, Scott Rasmussen, Craig Marsh, and Sophie Mormede. Casal2: New zealand's integrated population modelling tool. *Fisheries Research*, 183:498–505, 2016.

A. Dunn, S. Rasmussen, and S. Mormede. Spatial population model user manual, spm v1.1-2016-03-04 (rev. 1278). Technical Report 138, National Institute of Water and Atmospheric Research Ltd (NIWA), 2015.

R I C C Francis, V Haist, and B Bull. Assessment of hoki (*Macruronus novaezelandiae*) in 2002 using a new model. *New Zealand Fisheries Assessment Report*, 6, 2003.

RIC Chris Francis. Data weighting in statistical fisheries stock assessment models. *Canadian Journal of Fisheries and Aquatic Sciences*, 68(6):1124–1138, 2011.

A B Gelman, J S Carlin, H S Stern, and D B Rubin. *Bayesian data analysis*. Chapman and Hall, London, 1995.

W R Gilks, A Thomas, and D J Spiegelhalter. A language and program for complex Bayesian modelling. *The Statistician*, 43(1):169–177, 1994.

Walter R Gilks, Sylvia Richardson, and David Spiegelhalter. *Markov chain Monte Carlo in practice*. CRC press, 1995.

C S Holling. The components of predation as revealed by a study of small-mammal predation of the european pine sawfly. *The Canadian Entomologist*, 91:293–320, 1959.

PL Horn. Stock assessment of hake (*Merluccius australis*) on the Chatham Rise (HAK 4) and off the west coast of South Island (HAK 7) for the 2016–17 fishing year. *New Zealand Fisheries Assessment Report*, 47, 2017.

J Jurado-Molina, P A Livingston, and J N Ianelli. Incorporating predation interactions in a statistical catch-at-age model for a predator-prey system in the eastern bering sea. *Canadian Journal of Fisheries and Aquatic Sciences*, 62:1865–1873, 2005.

Davis E King. Dlib-ml: A machine learning toolkit. *Journal of Machine Learning Research*, 10: 1755–1758, 2009.

P.M Mace and I.J Doonan. A generalised bioeconomic simulation model for fish population dynamics. *New Zealand Fisheries Assessment Report*, 4, 1988.

Makoto Matsumoto and Takuji Nishimura. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation: Special Issue on Uniform Random Number Generation*, 8(1):3–30, January 1998.

Richard D Methot Jr and Ian G Taylor. Adjusting for bias due to variability of estimated recruitments in fishery assessment models. *Canadian Journal of Fisheries and Aquatic Sciences*, 68(10):1744–1760, 2011.

M I Michaelis and L Menten. Die kinetik der invertinwirkung. *Biochemische Zeitschrift*, 49:333–369, 1913.

Andre E. Punt and Ray Hilborn. *BAYES-SA. Bayesian stock assessment methods in fisheries. User's manual. FAO Computerized information series (fisheries) 12*. Food and Agriculture Organisation of the United Nations, Rome (Italy), 2001.

R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2014. URL `http://www.R-project.org/`.

J Schnute. A versatile growth model with statistically stable parameters. *Canadian Journal of Fisheries and Aquatic Sciences*, 38(9):1128–1140, 1981.

C Sherlock and G Roberts. Optimal scaling of the random walk metropolis on elliptically symmetric unimodal targets. *Bernoulli*, 15(3), 2009.

Rainer Storn and Kenneth Price. Differential evolution - a simple and efficient adaptive scheme for global optimization over continuous spaces. Technical Report TR-95-012, International Computer Science Institute, Berkeley, CA, 1995. URL `http://citeseer.ist.psu.edu/182432.html`.

Andreas Wächter and Lorenz T Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical programming*, 106(1): 25–57, 2006.

C.J Walters and D Ludwig. Calculation of bayes posterior probability distributions for key population parameters. *Canadian Journal of Fisheries and Aquatic Sciences*, 1994.

Andrea Walther, Andreas Kowarz, and Andreas Griewank. Adol-c: a package for the automatic differentiation of algorithms written in c/c++. *ACM TOMS*, 22(2):131–167, 1996.

21. Index

CASAL2 source code, 3
CASAL2 Contributors, 1

A simple non-spatial model example, 201
About CASAL2, 5
About CASAL2, 1
Abundance or biomass observations, 88
Additional Priors
    Beta, 82
Age-length relationship, 54, 140
    Data, 55
Age-length relationshsip
    None, 54
    von Bertalanffy, 54
Age-weight relationship UNTESTED, 56
Ageing, 32, 38
Ageing error, 108
Annual cycle, 25

Bayesian estimation, 76
Beta additional prior, 82
Beta prior, 80
Beverton-Holt recruitment, 34
Beverton-Holt stock-recruitment relationship, 33
Binomial likelihood
    proportions-by-category, 105
    tag-recapture-by-length, 104
Binomial likelihood (normal approximation)
    proportions-by-category, 105
Biomass-event mortality, 41
Boost C++ library, 3
Bounds, 72

Calculating nuisance q parameters, 106
CASAL, 5
Categories, 120, 197
Category transition, 32
Cinitial Initialisation UNTESTED, 31
Citing CASAL2, 1
Command
    additional_prior, 164, 225
    age_length, 140, 226
    ageing_error, 183, 225
    catchability, 162, 226
    categories, 120, 227
    derived_quantity, 139, 227
    estimate, 152, 227
    estimate_transformation, 166, 228
    include, 189
    initialisation_phase, 118, 229
    length_weight, 142, 230
    likelihood, 182, 230
    mcmc, 160, 230
    minimiser, 155, 231
    model, 117, 232
    observation, 169, 232
    penalty, 163, 236
    process, 121, 236
    profile, 162, 241
    project, 148, 241
    report, 184, 241
    selectivity, 143, 243
    simulate, 184, 245
    time_step, 121, 245
    time_varying, 136, 150, 245
command
    include files, 12
Command block format, 12
Command line arguments, 7, 8
Commands, 10
Commands
    Subcommands, 11
Commenting out lines, 12
Comments, 12
Constant mortality, 40
Constant recruitment, 33
Convergence failure, 73
Correlation matrix, 73
Covariance matrix, 71, 73

Defining ageing error, 182
Defining catchability constants, 162
Defining estimation of transformations, 166
Defining penalties, 163
Defining priors on parameter ratios, differences, and means, 164
Density-dependent prey-suitability, 49
Derived Initialisation, 30
Derived quantities, 5, 53, 139
Determining parameter names, 13
Differential evolution minimiser, 3
Dirichlet likelihood, 102
Disk space, 2

Equation Parser UNTESTED, 65
Estimable parameters, 7

Estimating parameters, 72
Estimation methods, 152
Event mortality, 41
Exit status value, 15

Finite differences minimiser, 3

gcc, 3
General process observations, 96
Getting help, 2
GitHub, 2
Growth, 32

Hessian, 71, 73
Holling mortality, 47

In-line declaration, 200
In-line declaration, avoiding extra command blocks, 24
Include an external file, 189
Including external files, 10
Initialisation, 25, 118
Initialisation event mortality, 48
Input configuration file, 7
Input configuration file syntax, 9
Instantaneous mortality, 42
Instantaneous mortality retained, 44
Iterative Initialisation, 29

Length-weight, 142
Length-weight relationship, 55
    Basic, 56
    None, 56
Likelihoods, 101, 182
Linux, 2, 3, 7
Local minimums, 73
Lognormal likelihood, 102, 103
Lognormal prior, 79

Markov chain Monte Carlo (MCMC), 71, 76, 160
Maturation, 32
Maturity, in models without maturing in the partition, 57
Maximum exploitation rate, 42, 47, 50
Maximum posterior density estimate (MPD), 71
MCMC, 71, 76
Microsoft Windows, 2, 3, 7
Mingw, 3
Model
    About CASAL2, 5
    derived quantities, 5
    initialisation, 25
    partition, 5
    processes, 5
    state, 5
    time-steps, 5
Model overview, 5
Model scope and structure, 25
Model structure, 117
Mortality, 32, 38
Mortality block associated observations, 87
MPD (Maximum posterior density estimate), 71
Multi-phase iteration, 29
Multinomial likelihood, 101

Necessary files, 2
Normal likelihood, 103
Normal prior, 79
Notifying errors, 2

Objective function, 72
Objective function evaluations, 73
Observation types, 168
Observations, 87
Optional command line arguments, 9
Output header information, 8

Parameter names, 13
Parameter Transformations, 83
Parameters, 199
Partition, 5
Partition & Categories, 17
Point estimation, 72, 155
Population processes, 32
Population section, 24
Post processing, 205
Post-processing output using **R**, 205
Post-processing section, 205
Posterior profiles, 75
Print a process summary, 112
Print derived quantities, 112
Print observations, fits, and residuals, 113
Print selectivities, 114
Print simulated observations, 113
Print the age length and length weight values, 112
Print the ageing error misclassification matrix, 114
Print the covariance matrix, 113
Print the estimated parameters, 112
Print the estimated parameters in a vector format, 113

Print the MCMC objective function values as they are calculated, 114
Print the MCMC samples as they are calculated, 114
Print the objective function, 113
Print the partition, 112
Print the partition at the end of an initialisation, 112
Print the random number seed, 114
Print the results of an MCMC, 114
Print time varying parameters, 114
Priors, 72
Priors
    Beta, 80
    Lognormal, 79
    Normal, 79
    Uniform, 79
    Uniform-log, 79
Process error, 105
Process removals by age, 97
Process removals by age retained, 97
Process removals by length, 98
Process removals by length retained, 99
Processes, 5, 121
Profiles, 71, 162
Projection methods, 67
Projections, 148
    Catches, 70
    Constant, 67
    Empirical resampling, 68
    Lognormal, 68
    Lognormal-Empirical, 68
    User Defined, 69
Proportions migrating, 100
Proportions-at-age, 89
Proportions-at-length, 92
Proportions-by-category observations, 93
Pseudo-observations, 109

Quasi-Newton iterations, 73

Random number generator, 3
Reading a command block, 13
Reading command block, 13
Reading command block section, 13
Recruitment, 32, 33
Recruitment
    Beverton-Holt, 33
    Constant, 33
Redirecting standard error, 7
Redirecting standard out, 7
Redirecting standard output, 7
Referencing vector and map parameters, 22
Report commands and subcommands, 184
Reports, 111
Reports
    Ageing error misclassification matrix, 114
    Covariance Matrix, 113
    Derived quantities, 112
    Estimated parameters, 112, 113
    Hessian, 113
    Initialisation, 112
    MCMC, 114
    MCMC objective functions, 114
    MCMC samples, 114
    Objective function, 113
    Observations, 113
    Partition, 112
    Processes, 112
    Random number seed, 114
    Selectivities, 114
    Simulated observations, 113
    standard style, 111
    Tabular, 114
    time varying, 114
Reports section, 111
Residuals, 110
Role of the estimation section, 71
Running CASAL2, 7

Schnute, 55
Selectivities, 57, 143
    All-values, 59
    All-values-bounded, 59
    Double-exponential, 61
    Double-normal, 61
    Increasing, 59
    Inverse-logistic , 60
    Logistic, 60
    Logistic-producing, 60
Simulating observations, 108, 184
Single-stepping CASAL2 , 15
Single_stepping, 15
single_stepping section, 15
Software license, 2
Specific process observations, 96
Specifying projections UNTESTED, 66
Specifying the parameters to be estimated, 72
Specifying the population model, 24
standard error, 7
standard output, 7

State, 5
State_category_by_age UNTESTED, 32
Subcommand argument type, 11
Successful convergence, 73
Syntax conventions, examples and niceties, 193
System requirements, 2

Tabular reporting format, 114
Tag Loss UNTESTED, 53
Tag Recapture by length, 94
Tag Release events UNTESTED, 51
Tasks, 7
Technical specifications, 3
The ADOL-C minimiser, 74
The BetaDiff minimiser, 74
The CppAD minimiser, 75
The differential evolution minimiser, 73
The Dlib minimiser, 75
The estimation section, 71
The implicit annual cycle, 27
The initialisation phases, 28
The numerical differences minimiser, 73
The objective function, 71
The report section, 111
Time varying parameters, 136
Time-steps, 121
time-steps, 5
Time-varying Parameters, 150
    Annual shift, 64
    Constant, 63
    Exogenous, 64
    Random Walk, 64
Time-varying Parameters UNTESTED, 62
Timing evaluation interval; timing the point when observations are fitted or derived quantities are evaluated , 39
Tips for setting up Casal2 model based on an existing CASAL model, 191
Transition By Category, 51

Uniform prior, 79
Uniform-log prior, 79
Unit tests, 3
User assistance, 2
Using CASAL2, 7

Weightless model UNTESTED, 57
Where to get CASAL2, 2

22. Quick reference

@additional_prior *label* Define an object of type *additional_prior*

parameter The name of the parameter to generate additional prior on
label The label for the additional prior
type The type of additional prior

@additional__prior[label].type=beta

mu Beta distribution mean ,mu, parameter
sigma Beta distribution variance ,sigma, parameter
a Beta distribution lower bound of the range ,A, parameter
b Beta distribution upper bound of the range ,B, parameter

@additional__prior[label].type=element_difference

second_parameter The name of the second parameter for comparing
multiplier Multiply the penalty by this factor

@additional__prior[label].type=log_normal

mu The lognormal prior mean ,mu, parameter
cv The lognormal variance ,cv, parameter

@additional__prior[label].type=uniform_log

@additional__prior[label].type=vector_average

method Which calculation method to use: k, l, or m
k The k value to use in the calculation
multiplier The multiplier for the penalty amount

@additional__prior[label].type=vector_smoothing

log_scale Should the sums of squares be calculated on the log scale?
multiplier Multiply the penalty by this factor
lower_bound The first element to apply the penalty to in the vector
upper_bound The last element to apply the penalty to in the vector
r Penalty applied to rth differences

@ageing_error *label* Define an object of type *ageing_error*

label The label of the ageing error
type The type of ageing error

@ageing__error[label].type=data

@ageing__error[label].type=none

@ageing__error[label].type=normal

cv CV of the misclassification matrix
k k defines the minimum age of individuals which can be misclassified, i.e., individuals of age less than k have no ageing error

@ageing__error[label].type=off_by_one

p1 The proportion misclassified as one year younger, e.g., the proportion of age 3 individuals that were misclassified as age 2
p2 The proportion misclassified as one year older, e.g., the proportion of age 3 individuals that were misclassified as age 4
k The minimum age of animals which can be misclassified, i.e., animals of age less than k are assumed to be correctly classified
@age_length *label* Define an object of type *age_length*

label The label of the age length relationship
type The type of age length relationship
time_step_proportions The fraction of the year applied in each time step that is added to the age for the purposes of evaluating the length, i.e., a value of 0.5 for a time step will evaluate the length of individuals at age+0.5 in that time step
distribution The assumed distribution for the growth curve
cv_first The CV for the first age class
cv_last The CV for last age class
casal_switch If true, use the ,less accurate, equation for the cumulative normal function as was used in the legacy version of CASAL.
by_length Specifies if the linear interpolation of CVs is a linear function of mean length at age. Default is by age only

@age__length[label].type=data

external_gaps The method to use for external data gaps
internal_gaps The method to use for internal data gaps
length_weight The label from an associated length-weight block
time_step_measurements_were_made The time step label for which size-at-age data are provided

@age__length[label].type=none

@age__length[label].type=schnute

y1 The y_1 parameter
y2 The y_2 parameter
tau1 The τ_1 parameter
tau2 The τ_2 parameter
a The a parameter
b The b parameter
length_weight The label of the associated length-weight relationship

@age__length[label].type=von_bertalanffy

linf The $L_{infinity}$ parameter
k The k parameter
t0 The t_0 parameter
length_weight The label of the associated length-weight relationship
@catchability *label* Define an object of type *catchability*

`label` Label of the catchability
`type` The type of catchability

`@catchability[label].type=free`

`q` The value of the catchability

`@catchability[label].type=nuisance`

`lower_bound` The upper bound for nuisance catchability
`upper_bound` The lower bound for nuisance catchability
`q`
`@categories` *`label`* Define an object of type *categories*

`format` The format that the category names use
`names` The names of the categories to be used in the model
`years` The years that individual categories will be active for. This overrides the model values
`age_lengths` R,The labels of age_length objects that are assigned to categories,
`length_weight` R,The labels of the length_weight objects that are assigned to categories,
`age_weight` R,The labels of the age_weight objects that are assigned to categories,
`@derived_quantity` *`label`* Define an object of type *derived_quantity*

`label` The label of the derived quantity
`type` The type of derived quantity
`time_step` The time step in which to calculate the derived quantity after
`categories` The list of categories to use when calculating the derived quantity
`selectivities` A list of one selectivity
`time_step_proportion` The proportion through the mortality block of the time step when calculated
`time_step_proportion_method` The method for interpolating for the proportion through the mortality block
`values`

`@derived__quantity[label].type=abundance`

`@derived__quantity[label].type=biomass`

`@estimate` *`label`* Define an object of type *estimate*

`label` The label of the estimate
`type` The prior type for the estimate
`parameter` The name of the parameter to estimate in the model
`lower_bound` The lower bound for the parameter
`upper_bound` The upper bound for the parameter
`same` List of parameters that are constrained to have the same value as this parameter
`estimation_phase` The first estimation phase to allow this to be estimated
`mcmc` Indicates if this parameter is estimated at the point estimate but fixed during MCMC estimation run
`transformation` Type of simple transformation to apply to estimate
`transform_with_jacobian` Apply jacobian during transformation
`prior_applies_to_transform` Does the prior apply to the transformed parameter? a legacy switch, see Manual for more information

@estimate[label].type=beta

`mu` Beta prior mean ,mu, parameter
`sigma` Beta prior variance ,sigma, parameter
`a` Beta prior lower bound of the range ,A, parameter
`b` Beta prior upper bound of the range ,B, parameter

@estimate[label].type=lognormal

`mu` The lognormal prior mean ,mu, parameter
`cv` The lognormal variance ,cv, parameter

@estimate[label].type=normal

`mu` The normal prior mean ,mu, parameter
`cv` The normal variance ,standard devation, parameter

@estimate[label].type=normal_by_stdev

`mu` The normal prior mean ,mu, parameter
`sigma` The normal variance ,sigma, parameter
`lognormal_transfomration` Add a Jacobian if the derived outcome of the estimate is assumed to be lognormal, e.g., used for rectuitment deviations in the recruitment process. See the User Manual for more information

@estimate[label].type=normal_log

`mu` The normal-log prior mean ,mu, parameter
`sigma` The normal-log prior variance ,standard deviation, parameter

@estimate[label].type=uniform

@estimate[label].type=uniform_log

@estimate_transformation *`label`* Define an object of type *estimate_transformation*

`label` Label for the transformation block
`type` The type of transformation
`transform_with_jacobian` Apply Jacobian during transformation

@estimate__transformation[label].type=average_difference

`theta2` The label of the @estimate block relating to the θ_2 parameter in the transformation. See the User Manual for more information
`theta1` The label of @estimate block relating to the θ_1 parameter in the transformation. See the User Manual for more information

@estimate__transformation[label].type=inverse

`estimate_label` The label of estimate block to apply transformation. Defined as θ_1 in the documentation

@estimate__transformation[label].type=log

`estimate_label` Label of estimate block to apply transformation. Defined as θ_1 in the documentation

`@estimate__transformation[label].type=log_sum`

`theta2` The label of the @estimate block relating to the θ_2 parameter in the transformation. See the User Manual for more information
`theta1` The label of @estimate block relating to the θ_1 parameter in the transformation. See the User Manual for more information

`@estimate__transformation[label].type=orthogonal`

`theta2` The label of the @estimate block relating to the θ_2 parameter in the transformation. See the User Manual for more information
`theta1` The label of @estimate block relating to the θ_1 parameter in the transformation. See the User Manual for more information

`@estimate__transformation[label].type=simplex`

`lower_bound` The empirical lower bound for the simplex transformed. This should be -Inf but some of the minimisers do not allow that
`upper_bound` The empirical upper bound for the simplex transformed. This should be Inf but some of the minimisers do not allow that

`@estimate__transformation[label].type=square_root`

`estimate_label` The label of the estimate block to apply transformation. Defined as θ_1 in the documentation

`@estimate__transformation[label].type=sum_to_one`

`estimate_labels` The label for the estimates for the sum to one transformation
`upper_bound` The empirical upper bounds for the transformed parameters. There should be one less bound than parameters
`lower_bound` The empirical lower bound for the transformed parameters. There should be one less bound than parameters
`@initialisation_phase` *`label`* Define an object of type *initialisation_phase*

`label` The label of the initialisation phase
`type` The type of initialisation

`@initialisation__phase[label].type=cinitial`

`categories` The list of categories for the Cinitial initialisation

`@initialisation__phase[label].type=derived`

`insert_processes` Additional processes not defined in the annual cycle that are to be inserted into this initialisation phase
`exclude_processes` Processes in the annual cycle to be excluded from this initialisation phase
`casal_initialisation_switch` Run an extra annual cycle to evalaute equilibrium SSBs. Warning - if true, this may not correctly evaluate the equilibrium state. Set to true if replicating a

CASAL model

@initialisation__phase[label].type=iterative

`years` The number of iterations ,years, over which to execute this initialisation phase
`insert_processes` The processes in the annual cycle to be included in this initialisation phase
`exclude_processes` The processes in the annual cycle to be excluded from this initialisation phase
`convergence_years` The iteration ,year, when the test for converegence ,lambda, is evaluated
`lambda` The maximum value of the absolute sum of differences ,lambda, between the partition at year-1 and year that indicates successful convergence

@initialisation__phase[label].type=state_category_by_age

`categories` The list of categories for the category state initialisation
`min_age` The minimum age of values supplied in the definition of the category state
`max_age` The maximum age of values supplied in the definition of the category state
@length_weight *`label`* Define an object of type *length_weight*

`label` The label of the length-weight relationship
`type` The type of the length-weight relationship

@length__weight[label].type=basic

`a` The a parameter ,$W = aL^b$,
`b` The b parameter ,$W = aL^b$,
`units` The units of measure ,tonnes, kgs, grams,

@length__weight[label].type=none

@likelihood *`label`* Define an object of type *likelihood*

@likelihood[label].type=binomial

@likelihood[label].type=binomial_approx

@likelihood[label].type=dirichlet

@likelihood[label].type=log_normal

@likelihood[label].type=log_normal_with_q

@likelihood[label].type=multinomial

@likelihood[label].type=normal

@likelihood[label].type=pseudo

@mcmc *`label`* Define an object of type *mcmc*

`label` The label of the MCMC
`type` The type of MCMC
`length` The number of iterations in for the MCMC chain
`active` Indicates if this is the active MCMC algorithm
`step_size` Initial stepsize ,as a multiplier of the approximate covariance matrix,

@mcmc[label].type=independence_metropolis

`start` The covariance multiplier for the starting point of the MCMC
`keep` The spacing between recorded values in the MCMC
`max_correlation` The maximum absolute correlation in the covariance matrix of the proposal distribution
`covariance_adjustment_method` The method for adjusting small variances in the covariance proposal matrix
`correlation_adjustment_diff` The minimum non-zero variance times the range of the bounds in the covariance matrix of the proposal distribution
`proposal_distribution` The shape of the proposal distribution ,either the t or the normal distribution,
`df` The degrees of freedom of the multivariate t proposal distribution
`adapt_stepsize_at` The iteration numbers in which to check and resize the MCMC stepsize
`adapt_covariance_matrix_at` The iteration numbers in which to adapt the covariance matrix
`adapt_stepsize_method` The method to use to adapt the step size
`@minimiser` *`label`* Define an object of type *minimiser*

`label` The minimiser label
`type` The type of minimiser to use
`active` Indicates if this minimiser is active
`covariance` Indicates if a covariance matrix should be generated

`@minimiser[label].type=adolc`

`iterations` The maximum number of iterations
`evaluations` The maximum number of evaluations
`tolerance` The tolerance of the gradient for convergence
`step_size` The minimum step size before minimisation fails

`@minimiser[label].type=betadiff`

`iterations` The maximum number of iterations
`evaluations` The maximum number of evaluations
`tolerance` The tolerance of the gradient for convergence

`@minimiser[label].type=cppad`

`retape` Retape?
`print_level` The level of debug to stdout
`sb` String buffer output?
`pidi` Print iteration diagnostic information?
`iterations` The maximum number of iterations
`tolerance` The tolerance for convergence
`acceptable_tol` The acceptable tolerance
`acceptable_obj_change_tol`
`derivative_test` How to test for derivatives
`point_perturbation_radius` The point perturbation radius

`@minimiser[label].type=de_solver`

`population_size` The number of candidate solutions to have in the population

`crossover_probability` The minimiser's crossover probability
`difference_scale` The scale to apply to new solutions when comparing candidates
`max_generations` The maximum number of iterations to run
`tolerance` The total variance between the population and best candidate before acceptance
`method` The type of candidate generation method to use

`@minimiser[label].type=d_lib`

`minimisation_type` The type of minimisation to use
`search_strategy` The type of search strategy to use
`tolerance` The tolerance of the gradient for convergence
`lbfgs_max_size` The maximum size for the LBFGS search strategy
`bobyqa_interpolation_points` BOBYQA interpolation points
`bobyqa_initial_trust_radius` BOBYQA initial trust radius
`bobyqa_stopping_trust_radius` BOBYQA stopping trust radius
`bobyqa_max_evaluations` BOBYQA max objective evaluations
`verbose` Print debug of objective function calls?

`@minimiser[label].type=numerical_differences`

`iterations` The maximum number of iterations
`evaluations` The maximum number of evaluations
`tolerance` The tolerance of the gradient for convergence
`step_size` The minimum step size before minimisation fails
`@model` *`label`* Define an object of type *model*

`start_year` The first year of the model, immediately following initialisation
`final_year` The final year of the model, excluding years in the projection period
`min_age` The minimum age of individuals in the population
`max_age` The maximum age of individuals in the population
`age_plus` The oldest age or extra length midpoint ,plus group size, as a plus group
`initialisation_phases` The labels of the phases of the initialisation
`time_steps` The labels of the time steps, in the order that they are applied, to form the annual cycle
`projection_final_year` The final year of the model in projection mode
`type` The type of model ,the partition structure,: age, length, or hybrid
`length_bins` The minimum length in each length bin
`length_plus` Is there a length plus group or not?
`length_plus_group` The mean length of length plus group
`base_weight_units` The units for the base weight. This will be the default unit of any weight input parameters
`@observation` *`label`* Define an object of type *observation*

`label` The label of the observation
`type` The type of observation
`likelihood` The type of likelihood to use
`categories` The category labels to use
`delta` The robustification value ,delta, for the likelihood
`simulation_likelihood` The simulation likelihood to use
`likelihood_multiplier` The likelihood score multiplier
`error_value_multiplier` The error value multiplier for likelihood

@observation[label].type=abundance

`time_step` The label of the time step that the observation occurs in
`catchability` The label of the catchability coefficient ,q,
`selectivities` The labels of the selectivities
`process_error` The process error
`years` The years for which there are observations

@observation[label].type=biomass

`time_step` The label of the time step that the observation occurs in
`catchability` The label of the catchability coefficient ,q,
`selectivities` The labels of the selectivities
`process_error` The process error
`age_weight_labels` R,The labels for the `@`*age_weight* block which corresponds to each category, to use the weight calculation method for biomass calculations,
`years` The years of the observed values

@observation[label].type=process_removals_by_age

`min_age` The minimum age
`max_age` The maximum age
`plus_group` Is the maximum age the age plus group
`time_step` The label of time-step that the observation occurs in
`tolerance` The tolerance
`years` The years for which there are observations
`process_errors` The label of process error to use
`ageing_error` The label of the ageing error to use
`method_of_removal` The label of the observed method of removals
`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation

@observation[label].type=process_removals_by_age_retained

`min_age` The minimum age
`max_age` The maximum age
`plus_group` Is the maximum age the age plus group?
`time_step` The label of the time step that the observation occurs in
`tolerance` The tolerance
`years` The years for which there are observations
`process_errors` The label of the process error to use
`ageing_error` The label of the ageing error to use
`method_of_removal` The label of observed method of removals
`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation

@observation[label].type=process_removals_by_age_retained_total

`min_age` The minimum age

`max_age` The maximum age
`plus_group` Is the maximum age the age plus group?
`time_step` The label of the time step that the observation occurs in
`tolerance` The tolerance
`years` The years for which there are observations
`process_errors` The label of the process error to use
`ageing_error` The label of the ageing error to use
`method_of_removal` The label of observed method of removals
`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation

`@observation[label].type=process_removals_by_length`

`time_step` The time step to execute in
`tolerance` The tolerance for rescaling proportions
`years` The years for which there are observations
`process_errors` The process error
`method_of_removal` The label of observed method of removals
`length_bins` The length bins
`length_plus` Is the last length bin a plus group? ,defaults to @model value,
`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation

`@observation[label].type=process_removals_by_length_retained`

`time_step` The time step to execute in
`tolerance` The tolerance for rescaling proportions
`years` The years for which there are observations
`process_errors` The process error
`method_of_removal` The label of observed method of removals
`length_bins` The length bins
`length_plus` Is the last length bin a plus group? ,defaults to @model value,
`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation

`@observation[label].type=process_removals_by_length_retained_total`

`time_step` The time step to execute in
`tolerance` The tolerance for rescaling proportions
`years` The years for which there are observations
`process_errors` The process error
`method_of_removal` The label of observed method of removals
`length_bins` The length bins
`length_plus` Is the last length bin a plus group? ,defaults to @model value,
`mortality_instantaneous_process` The label of the mortality instantaneous process for the observation

`@observation[label].type=proportions_at_age`

`min_age` The minimum age

`max_age` The maximum age
`plus_group` Is the maximum age the age plus group?
`time_step` The label of the time step that the observation occurs in
`tolerance` The tolerance on the constraint that for each year the sum of proportions in each age must equal 1, e.g., if tolerance = 0.1 then 1 - Sum,Proportions, can be as great as 0.1
`years` The years of the observed values
`selectivities` The labels of the selectivities
`process_errors` The process error
`ageing_error` The label of ageing error to use

`@observation[label].type=proportions_at_length`

`time_step` The label of the time step that the observation occurs in
`tolerance` The tolerance for rescaling proportions
`years` The years for which there are observations
`selectivities` The labels of the selectivities
`process_errors` The process error
`length_bins` The length bins

`@observation[label].type=proportions_by_category`

`min_age` The minimum age
`max_age` The maximum age
`time_step` The label of the time step that the observation occurs in
`plus_group` Use the age plus group?
`years` The years for which there are observations
`selectivities` The labels of the selectivities
`categories2` The target categories
`selectivities2` The target selectivities

`@observation[label].type=proportions_mature_by_age`

`min_age` The minimum age
`max_age` The maximum age
`time_step` The label of time-step that the observation occurs in
`plus_group` Use the age plus group?
`years` The years for which there are observations
`ageing_error` The label of ageing error to use
`total_categories` All category labels that were vulnerable to sampling at the time of this observation ,not including the categories already given,
`time_step_proportion` The proportion through the mortality block of the time step when the observation is evaluated

`@observation[label].type=proportions_migrating`

`min_age` The minimum age

`max_age` The maximum age
`time_step` The label of the time step that the observation occurs in
`plus_group` Is the maximum age the age plus group?
`years` The years for which there are observations
`process_errors` The process error
`ageing_error` The label of the ageing error to use
`process` The process label

`@observation[label].type=tag_recapture_by_age`

`min_age` The minimum age
`max_age` The maximum age
`plus_group` Is the maximum age the age plus group?
`years` The years for which there are observations
`categories2` The available categories in the partition
`selectivities` The labels of the selectivities
`time_step` The label of the time step that the observation occurs in
`selectivities2` The categories of tagged individuals for the observation
`detection` The probability of detecting a recaptured individual
`time_step_proportion` The proportion through the mortality block of the time step when the observation is evaluated

`@observation[label].type=tag_recapture_by_length`

`years` The years for which there are observations
`time_step` The time step to execute in
`length_bins` The length bins
`selectivities` The labels of the selectivities used for untagged categories
`tagged_selectivities` The labels of the tag category selectivities
`detection` The probability of detecting a recaptured individual
`dispersion` The overdispersion parameter ,phi,
`time_step_proportion` The proportion through the mortality block of the time step when the observation is evaluated
`@penalty` *`label`* Define an object of type *penalty*

`label` The label of the penalty
`type` The type of penalty

`@penalty[label].type=process`

`multiplier` The penalty multiplier
`log_scale` Indicates if the sums of squares is calculated on the log scale
`@process` *`label`* Define an object of type *process*

`label` The label of the process
`type` The type of process

`@process[label].type=ageing`

`categories` The labels of the categories

@process[label].type=growth_basic

`categories` The labels of the categories
`number_of_growth_episodes` The number of growth episodes per year
`growth_time_steps` The time step in which each growth episode occurs
`cv` The CV for the growth model
`sigma_min` The lower bound on sigma for the growth model

@process[label].type=maturation

`from` The list of categories to mature from
`to` The list of categories to mature to
`selectivities` The list of selectivities to use for maturation
`years` The years to be associated with the maturity rates
`rates` The rates to mature for each year

@process[label].type=mortality_constant_rate

`categories` The list of categories labels
`m` The mortality rates
`time_step_ratio` The time step ratios for the mortality rates

@process[label].type=mortality_event

`categories` The categories
`years` The years in which to apply the mortality process
`catches` The number of removals ,catches, to apply for each year
`u_max` The maximum exploitation rate ,U_{max},
`selectivities` The list of selectivities
`penalty` The label of the penalty to apply if the total number of removals cannot be taken

@process[label].type=mortality_event_biomass

`categories` The category labels
`selectivities` The labels of the selectivities for each of the categories
`years` The years in which to apply the mortality process
`catches` The biomass of removals ,catches, to apply for each year
`u_max` The maximum exploitation rate ,U_{max},
`penalty` The label of the penalty to apply if the total biomass of removals cannot be taken

@process[label].type=mortality_holling_rate

`prey_categories` The prey categories labels
`predator_categories` The predator categories labels
`is_abundance` Is vulnerable amount of prey and predator an abundance [true] or biomass [false]
`a` Parameter a
`b` Parameter b
`x` This parameter controls the functional form: Holling function type 2 ,x=2, or 3 ,x=3, or

generalised ,Michaelis Menten, x,=1,
`u_max` The maximum exploitation rate ,U_{max},
`prey_selectivities` The selectivities for prey categories
`predator_selectivities` The selectivities for predator categories
`penalty` The label of penalty
`years` The years in which to apply the mortality process

`@process[label].type=mortality_initialisation_event`

`categories` The categories
`catch` The number of removals ,catches, to apply for each year
`u_max` The maximum exploitation rate ,U_{max},
`selectivities` The list of selectivities
`penalty` The label of the penalty to apply if the total number of removals cannot be taken

`@process[label].type=mortality_initialisation_event_biomass`

`categories` The categories
`catch` The number of removals ,catches, to apply for each year
`u_max` The maximum exploitation rate ,U_{max},
`selectivities` The list of selectivities
`penalty` The label of the penalty to apply if the total number of removals cannot be taken

`@process[label].type=mortality_instantaneous`

`categories` The categories for instantaneous mortality
`m` The natural mortality rates for each category
`time_step_ratio` The time step ratios for natural mortality
`relative_m_by_age` The M-by-age ogives to apply on the categories for natural mortality

`@process[label].type=mortality_instantaneous_retained`

`categories` The categories for instantaneous mortality
`m` The natural mortality rates for each category
`time_step_ratio` The time step ratios for natural mortality
`relative_m_by_age` The M-by-age ogives to apply on the categories for natural mortality

`@process[label].type=mortality_prey_suitability`

`prey_categories` The prey categories labels
`predator_categories` The predator categories labels
`consumption_rate` The predator consumption rate
`electivities` The prey electivities
`u_max` The maximum exploitation rate ,U_{max},
`prey_selectivities` The selectivities for prey categories
`predator_selectivities` The selectivities for predator categories
`penalty` The label of the penalty
`years` The year that process occurs

`@process[label].type=recruitment_beverton_holt`

`categories` The category labels
`r0` R0
`b0` B0
`proportions` The proportion for each category
`age` The age at recruitment
`ssb_offset` The spawning biomass year offset
`steepness` Steepness ,h,
`ssb` The SSB label ,derived quantity,
`b0_initialisation_phase` The initialisation phase label that B0 is from
`ycs_values` The YCS values
`ycs_years` The recruitment years. A vector of years that relates to the year of the spawning event that created this cohort
`standardise_ycs_years` The years that are included for year class standardisation

`@process[label].type=recruitment_beverton_holt_with_deviations`

`categories` The category labels
`r0` R0
`b0` B0
`proportions` The proportion for each category
`age` The age at recruitment
`ssb_offset` The spawning biomass year offset
`steepness` Steepness ,h,
`ssb` The SSB Label ,derived quantity,
`sigma_r` sigma R
`b_max` The maximum bias adjustment
`last_year_with_no_bias` The last year with no bias adjustment
`first_year_with_bias` The first year with full bias adjustment
`last_year_with_bias` The last year with full bias adjustment
`first_recent_year_with_no_bias` The first recent year with no bias adjustment
`b0_initialisation_phase` The initialisation phase label that B0 is from
`deviation_values` The recruitment deviation values
`deviation_years` The recruitment years. A vector of years that relates to the year of the spawning event that created this cohort

`@process[label].type=recruitment_constant`

`categories` The categories
`proportions` The proportions
`length_bins` The length bins that recruits are uniformly distributed over at the time of recruitment
`r0` R0

`@process[label].type=survival_constant_rate`

`categories` The list of categories
`s` The survival rates
`time_step_ratio` The time step ratios for S
`selectivities` The selectivity label

@process[label].type=tag_by_age

`from` The categories to transition from
`to` The categories to transition to
`min_age` The minimum age to transition
`max_age` The maximum age to transition
`penalty` The penalty label
`u_max` The maximum exploitation rate ,U_{max},
`years` The years to execute the transition in
`initial_mortality` The initial mortality value
`initial_mortality_selectivity` The initial mortality selectivity label
`loss_rate` The loss rate
`loss_rate_selectivities` The loss rate selectivity label
`selectivities` The selectivity labels
`n` N

@process[label].type=tag_by_length

`from` The categories to transition from
`to` The categories to transition to
`penalty` The penalty label
`u_max` The maximum exploitation rate ,U_{max},
`initial_mortality`
`initial_mortality_selectivity`
`selectivities`
`n`

@process[label].type=tag_loss

`categories` The list of categories
`tag_loss_rate` The tag loss rates
`time_step_ratio` The time step ratios for tag loss
`tag_loss_type` The type of tag loss
`selectivities` The selectivities
`year` The year the first tagging release process was executed

@process[label].type=transition_category

`from` The from category
`to` The to category
`proportions` The proportions
`selectivities` The selectivity names

@process[label].type=transition_category_by_age

`from` The categories to transition from

`to` The categories to transition to
`min_age` The minimum age to transition
`max_age` The maximum age to transition
`penalty` The penalty label
`u_max` The maximum exploitation rate ,U_{max},
`years` The years to execute the transition in
`@profile` *`label`* Define an object of type *profile*

`label` The label of the profile
`steps` The number of steps between the lower and upper bound
`lower_bound` The lower bounds
`upper_bound` The upper bounds
`parameter` The system parameter to profile
`same` A parameter that is constrained to have the same value as the parameter being profiled
`@project` *`label`* Define an object of type *project*

`label` The projection label
`type` The projection type
`years` The years to recalculate the values
`parameter` The parameter to project
`multiplier` The multiplier applied to the projected value

`@project[label].type=constant`

`values` The values to assign to the addressable

`@project[label].type=empirical_sampling`

`start_year` The start year of sampling
`final_year` The final year of sampling

`@project[label].type=log_normal`

`mean` The mean of the lognormal process
`sigma` The standard deviation ,sigma, of the lognormal process

`@project[label].type=log_normal_empirical`

`mean` The mean of the Gaussian process
`start_year` The start year of sampling
`final_year` The final year of sampling

`@project[label].type=user_defined`

`equation` The equation to do a test run of
`@report` *`label`* Define an object of type *report*

`label` The label for the report
`type` The type of report
`file_name` The filename for this report to be in a separate file
`write_mode` The write mode

@report[label].type=addressable

`parameter` The parameter to print
`years` The years to print the addressable for
`time_step` The time Step label

@report[label].type=age_length

`time_step` The time step label
`years` The years for the report
`age_length` The age-length label
`category` The category label

@report[label].type=ageing_error_matrix

`ageing_error` The ageing error label

@report[label].type=catchability

@report[label].type=category_info

@report[label].type=category_list

@report[label].type=correlation_matrix

@report[label].type=covariance_matrix

@report[label].type=derived_quantity

@report[label].type=equation_test

`equation` The equation to do a test run of

@report[label].type=estimate_summary

@report[label].type=estimate_value

@report[label].type=estimation_result

@report[label].type=hessian_matrix

@report[label].type=initialisation_partition

@report[label].type=initialisation_partition_mean_weight

@report[label].type=mcmc_covariance

@report[label].type=mcmc_objective

@report[label].type=mcmc_sample

@report[label].type=m_p_d

@report[label].type=objective_function

@report[label].type=observation

`observation` The observation label
`normalised_residuals` Print Normalised Residuals?
`pearsons_residuals` Print Pearsons Residuals?

@report[label].type=output_parameters

@report[label].type=partition

`time_step` The time step label
`years` The years for the report

@report[label].type=partition_biomass

`time_step` The time step label
`years` The years for the report

@report[label].type=partition_mean_length

`time_step` The time step label
`years` The years for the report

@report[label].type=partition_mean_weight

`time_step` The time step label
`years` The years for the report

@report[label].type=partition_year_cross_age_matrix

@report[label].type=process

`process` The process label that is reported

@report[label].type=project

`project` The project label that is reported

@report[label].type=random_number_seed

@report[label].type=selectivity

`selectivity` The selectivity name

@report[label].type=simulated_observation

`observation` The observation label

@report[label].type=standard_header

@report[label].type=time_varying

@selectivity *`label`* Define an object of type *selectivity*

`label` The label for this selectivity
`type` The type of selectivity
`length_based` Is the selectivity length based?
`intervals` The number of quantiles to evaluate a length-based selectivity over the age-length distribution
`partition_type` The type of partition that this selectivity will support. Defaults to the same as

the model
`values`
`length_values`

`@selectivity[label].type=all_values`

`v` The v parameter

`@selectivity[label].type=all_values_bounded`

`l` The low value ,L,
`h` The high value ,H,
`v` The v parameter

`@selectivity[label].type=constant`

`c` The constant value

`@selectivity[label].type=double_exponential`

`x0` The X0 parameter
`x1` The X1 parameter
`x2` The X2 parameter
`y0` The Y0 parameter
`y1` The Y1 parameter
`y2` The Y2 parameter
`alpha` alpha

`@selectivity[label].type=double_normal`

`mu` The mean ,mu,
`sigma_l` The sigma L parameter
`sigma_r` The sigma R parameter
`alpha` alpha

`@selectivity[label].type=increasing`

`l` The low value ,L,
`h` The high value ,H,
`v` The v parameter
`alpha` alpha

`@selectivity[label].type=inverse_logistic`

`a50` a50
`ato95` ato95
`alpha` alpha

`@selectivity[label].type=knife_edge`

`e` The edge value
`alpha` alpha

@selectivity[label].type=logistic

`a50` a50
`ato95` ato95
`alpha` alpha

@selectivity[label].type=logistic_producing

`l` The low value ,L,
`h` The high value ,H,
`a50` a50
`ato95` ato95
`alpha` alpha

@simulate *label* Define an object of type *simulate*

`label` Label
`type` Type
`years` Years to recalculate the values
`parameter` Parameter to simulate

@simulate[label].type=constant

`value` The value to assign to the addressable

@time_step *label* Define an object of type *time_step*

`label` The label of the timestep
`processes` The labels of the processes for this time step in the order that they occur

@time_varying *label* Define an object of type *time_varying*

`label` The time-varying label
`type` The time-varying type
`years` The years in which to vary the values
`parameter` The name of the parameter to time vary

@time__varying[label].type=annual_shift

`values` The values
`a` Parameter A
`b` Parmeter B
`c` Parameter C
`scaling_years` The scaling years

@time__varying[label].type=constant

`values` The value to assign to addressable

@time__varying[label].type=exogenous

`a` The shift parameter
`exogeneous_variable` The values of exogeneous variable for each year

@time__varying[label].type=linear

`slope` The slope of the linear trend ,additive unit per year,
`intercept` The intercept of the linear trend value for the first year

`@time__varying[label].type=random_draw`

`mean` The mean ,mu,
`sigma` The standard deviation ,sigma,
`distribution` The distribution

`@time__varying[label].type=random_walk`

`mean` The mean ,mu,
`sigma` The standard deviation ,sigma,
`upper_bound` The upper bound for the random walk
`upper_bound` The lower bound for the random walk
`rho` The autocorrelation parameter ,rho,
`distribution` The distribution